NexGenEMR - Complete Project Documentation

> **A Comprehensive Electronic Medical Records (EMR) System**
> Built with FastAPI (Backend) + Next.js (Frontend)

Table of Contents

System Overview

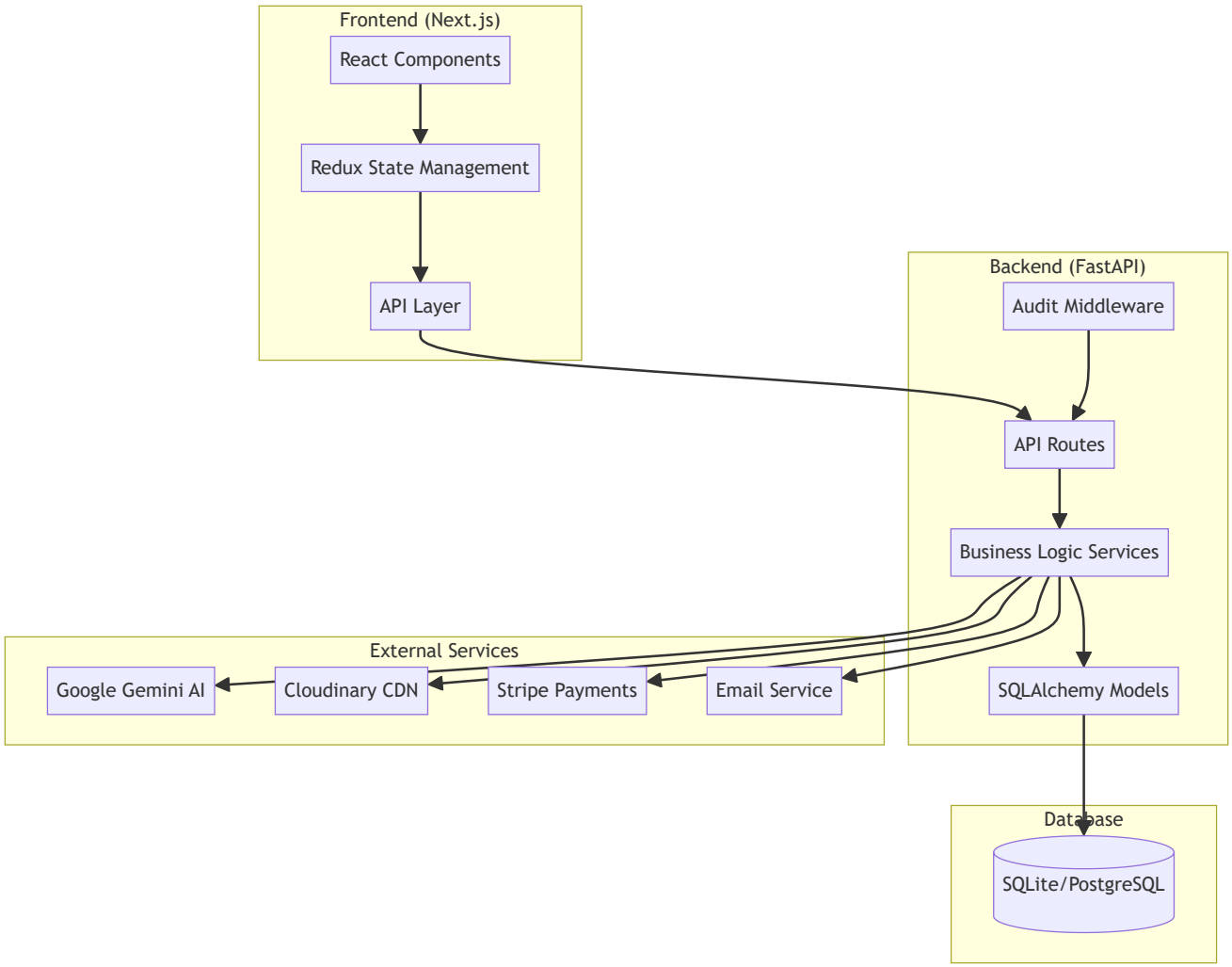
NexGenEMR is a modern, full-featured Electronic Medical Records system designed for multi-hospital environments. It provides comprehensive patient care management, appointment scheduling, billing, clinical documentation, and AI-powered features.

Key Highlights

- 🏥 **Multi-Hospital Support** - Single platform managing multiple hospitals
- 👥 **Role-Based Access Control** - 7+ distinct user roles with granular permissions
- 📅 **Advanced Scheduling** - Recurring sessions, slot management, waitlist integration
- 💰 **Revenue Cycle Management** - Billing, claims, invoices, and Stripe integration
- 🤖 **AI-Powered Features** - SOAP notes from audio, handoff notes, patient summaries
- 🧪 **Lab Integration** - Lab requests with AI-driven image analysis (brain tumor detection)
- 📱 **Responsive Design** - Mobile-optimized interfaces for all user types
- 🔐 **HIPAA-Ready Security** - Encryption, audit logging, secure authentication

---

Architecture



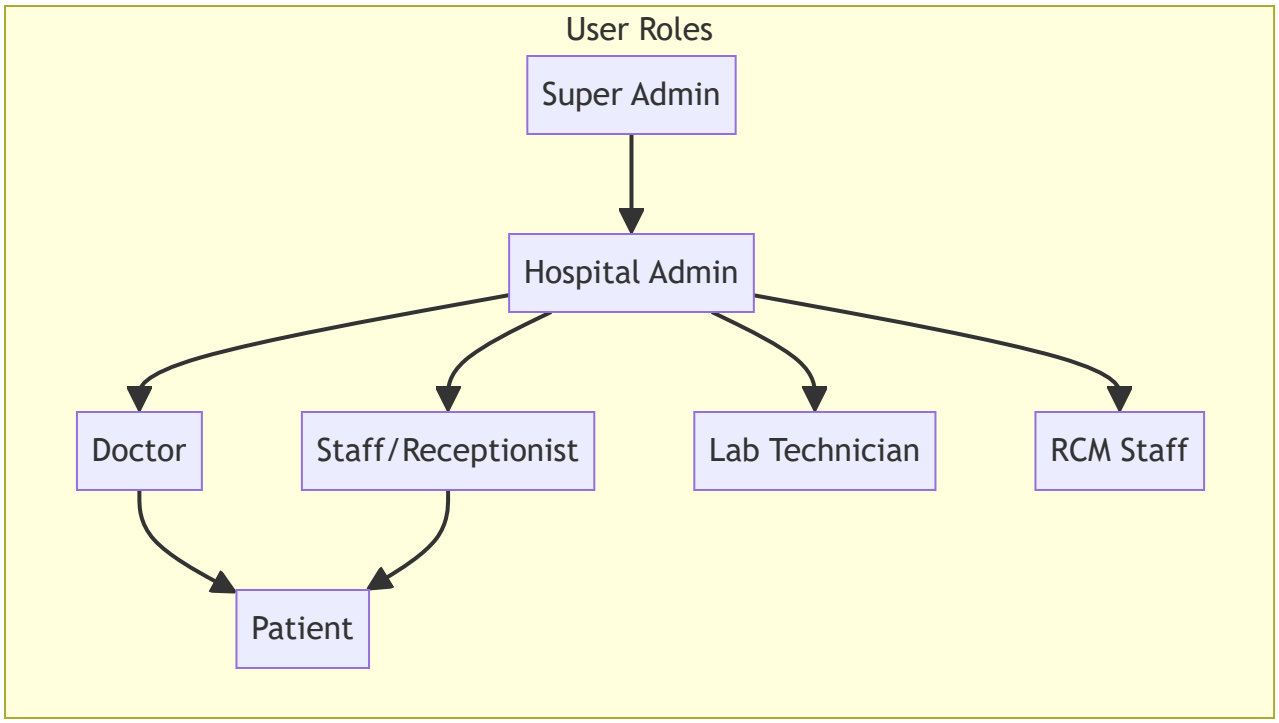
Technology Stack

Layer	Technology
Frontend	Next.js 14+, React, TypeScript, TailwindCSS, Redux Toolkit
Backend	FastAPI, Python 3.10+, SQLAlchemy, Pydantic
Database	SQLite (development), PostgreSQL (production)
AI	Google Gemini 2.5 Flash
File Storage	Cloudinary

Layer	Technology
Payments	Stripe
Real-time	Socket.IO
Deployment	Docker, Docker Compose, Nginx

User Roles & Access Control

The system implements a comprehensive role-based access control (RBAC) system with the following roles:



Role Descriptions

Role	Description	Key Permissions
Super Admin	Platform administrator	Create hospitals, manage subscriptions, view all analytics
Hospital Admin	Hospital-level administrator	Manage staff, doctors, departments, hospital settings

Role	Description	Key Permissions
Doctor	Medical practitioner	View patients, manage appointments, create clinical notes
Staff/Receptionist	Front desk staff	Register patients, manage appointments, handle waitlist
Lab Technician	Laboratory staff	Process lab requests, upload results, run AI analysis
RCM Staff	Revenue cycle management	Handle billing, claims, invoices, payments
Patient	End user/patient	View appointments, billing, lab results, summaries

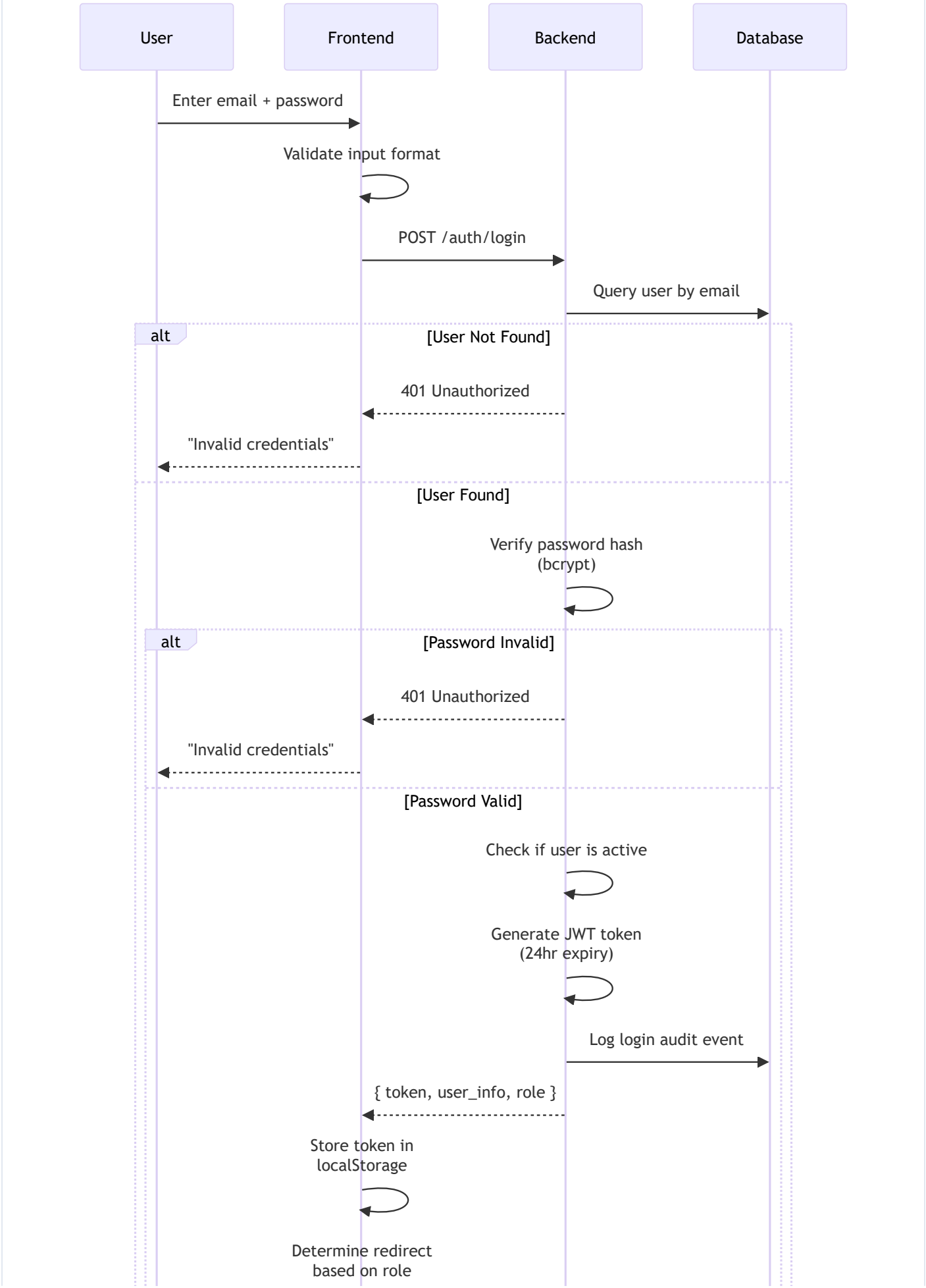
---

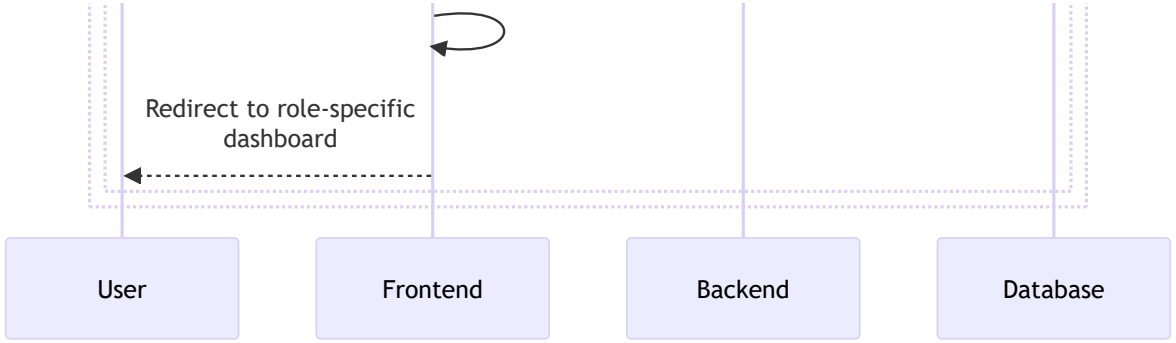
Core Features & Flows

1. Authentication & Authorization

The system uses JWT-based authentication with role-specific access control.

1.1 Login Flow

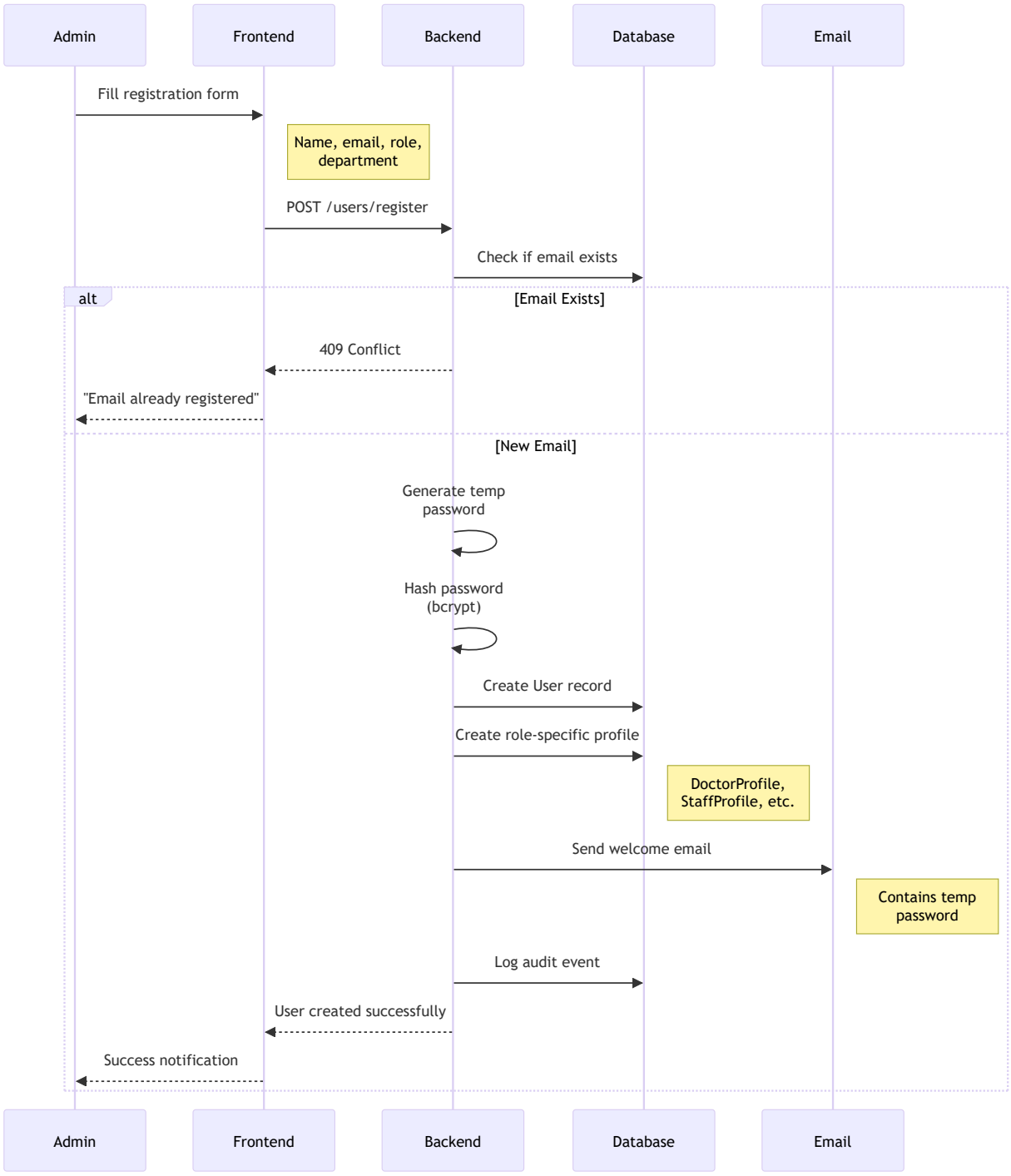

User
Frontend
Backend
Database
Enter email + password
Validate input format
POST /auth/login
Query user by email
alt
[User Not Found]
401 Unauthorized
"Invalid credentials"
[User Found]
Verify password hash
(bcrypt)
alt
[Password Invalid]
401 Unauthorized
"Invalid credentials"
[Password Valid]
Check if user is active
Generate JWT token
(24hr expiry)
Log login audit event
{ token, user_info, role }
Store token in
localStorage
Determine redirect
based on role



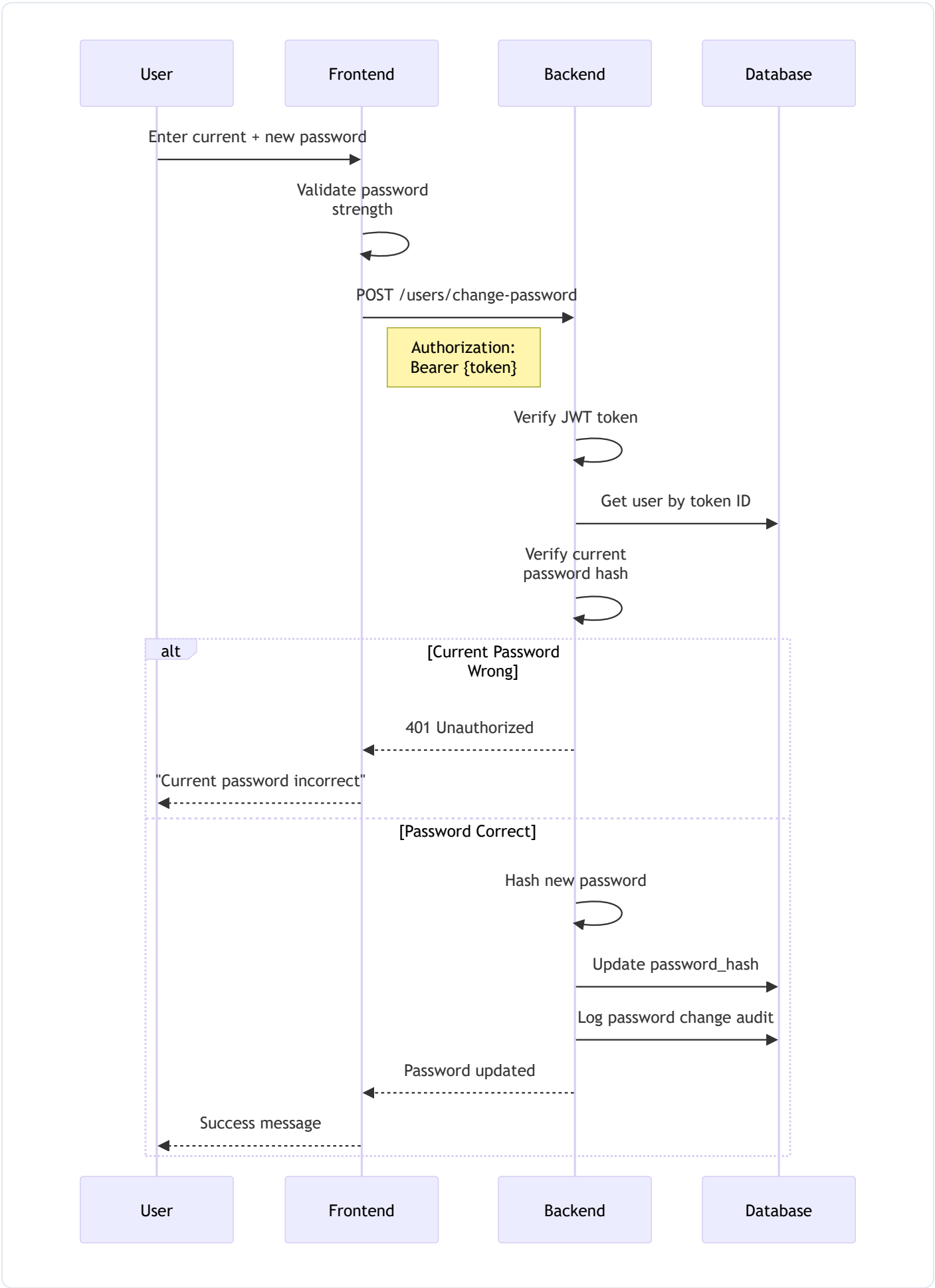
1.2 Role-Based Redirects

Role	Dashboard Redirect
Super Admin	`/super-admin/dashboard`
Hospital Admin	`/hospital-admin/dashboard`
Doctor	`/doctor/dashboard`
Receptionist	`/receptionist/dashboard`
Lab Technician	`/lab-technician/dashboard`
RCM Staff	`/rcm/dashboard`
Patient	`/patient/dashboard`

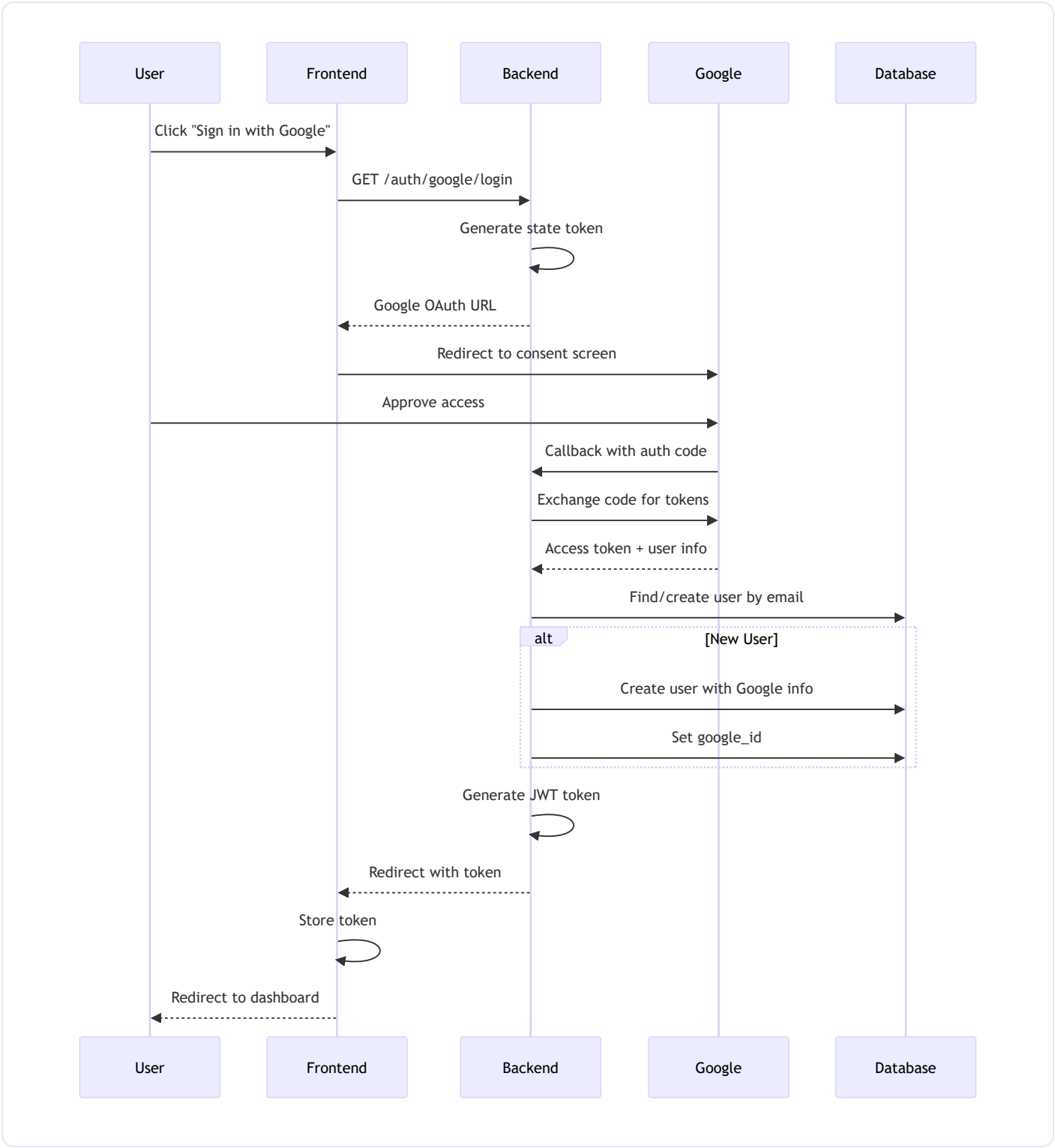
1.3 User Registration Flow (Staff/Doctor Creation)



1.4 Password Change Flow


User
Frontend
Backend
Database
Enter current + new password
Validate password strength
POST /users/change-password
Authorization: Bearer {token}
Verify JWT token
Get user by token ID
Verify current password hash
alt
[Current Password Wrong]
401 Unauthorized
"Current password incorrect"
[Password Correct]
Hash new password
Update password_hash
Log password change audit
Password updated
Success message
User
Frontend
Backend
Database

1.5 Google OAuth Flow



Key Features:

- **Email/Password Login** - Traditional authentication
- **Google OAuth** - Social login integration
- **JWT Tokens** - Secure token-based sessions (24hr expiry)
- **Password Hashing** - Bcrypt encryption

- **Role-based Redirects** - Different dashboards per role
- **Audit Logging** - Track login attempts

Key Files:

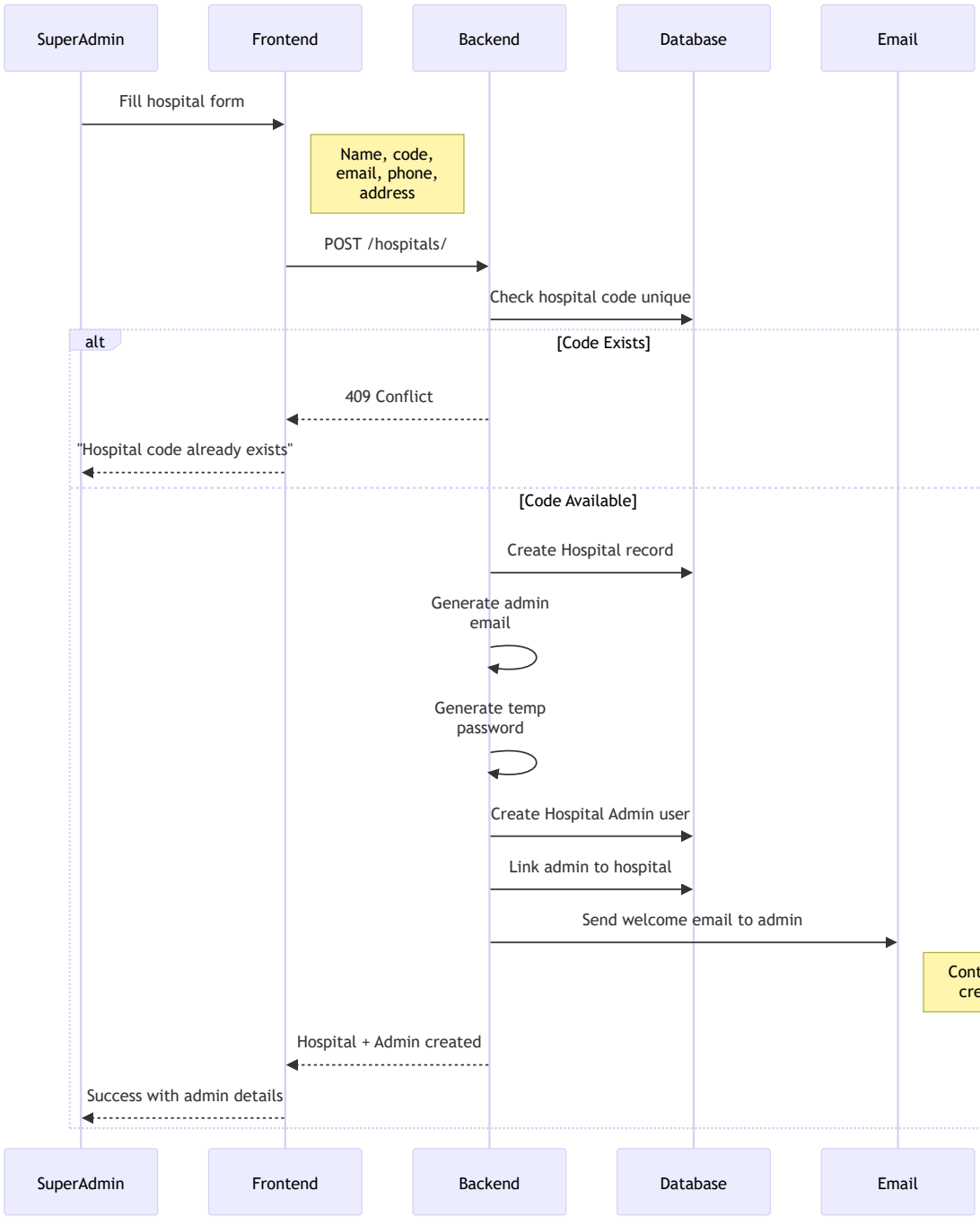
- Backend: `routes/user_routes.py`, `services/user_service.py`
- Frontend: `store/slices/authSlice.ts`, `app/auth/`

---

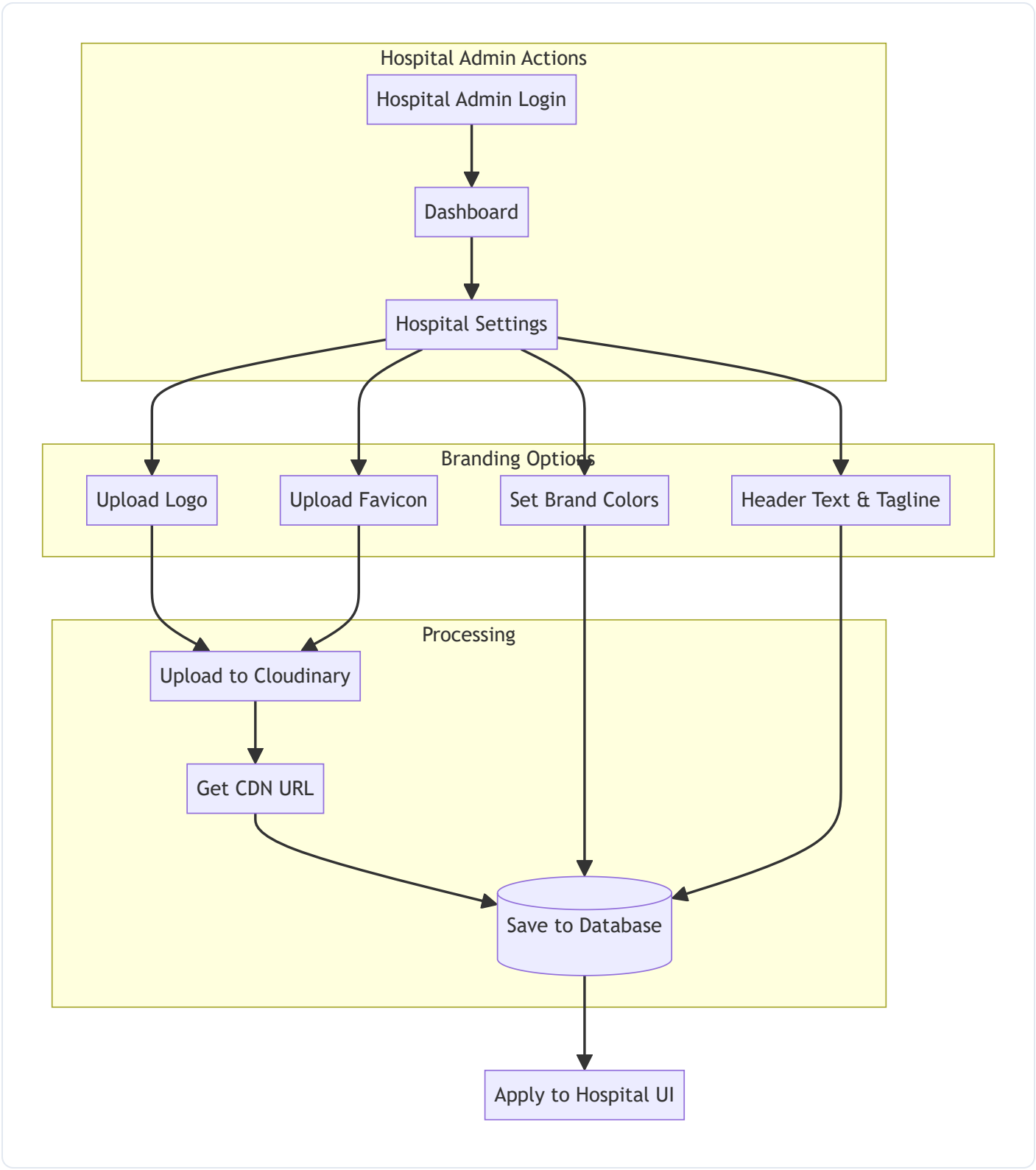
2. Hospital Management

Super admins can create and manage multiple hospitals, each operating as an independent entity.

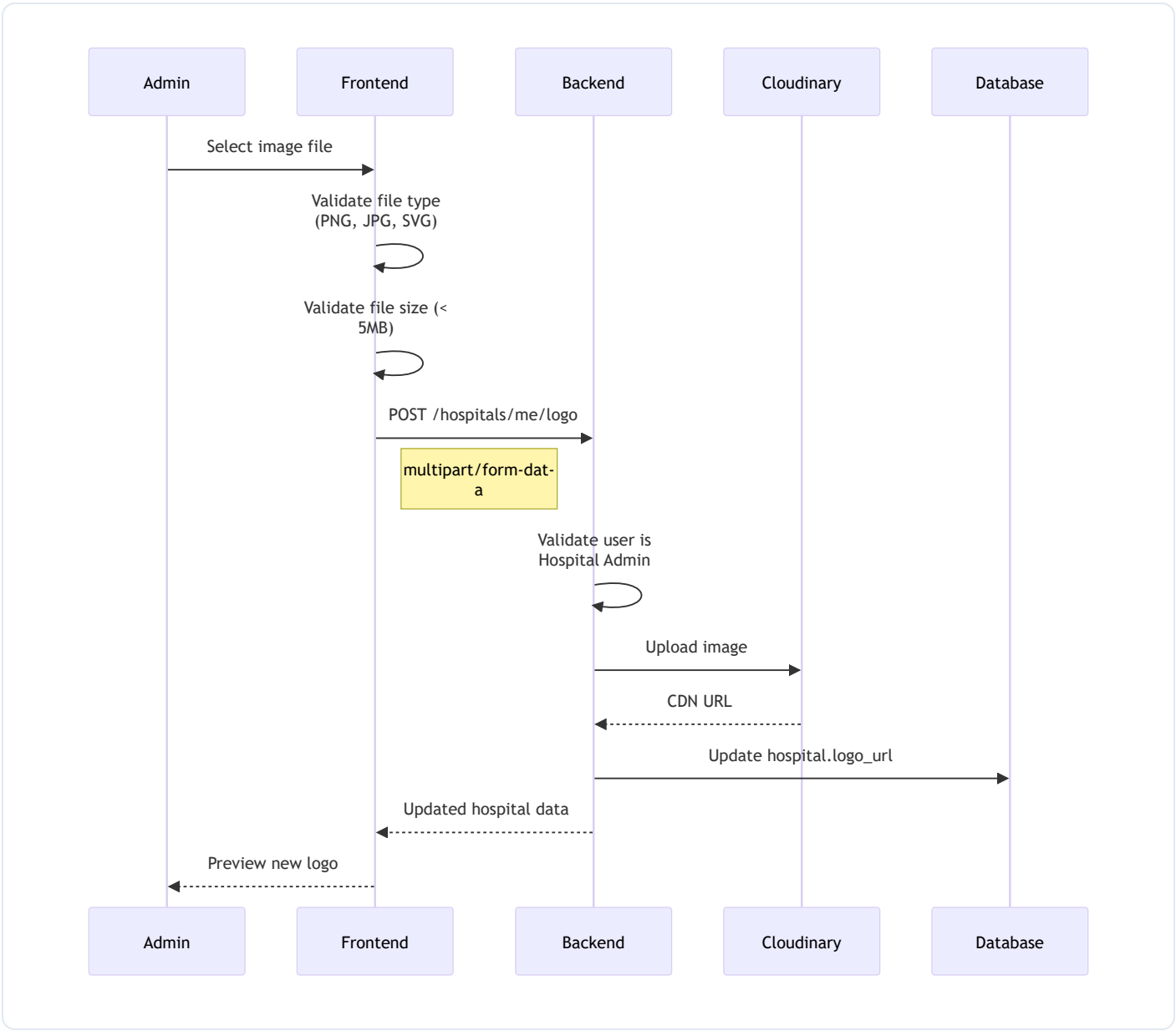
2.1 Hospital Creation Flow



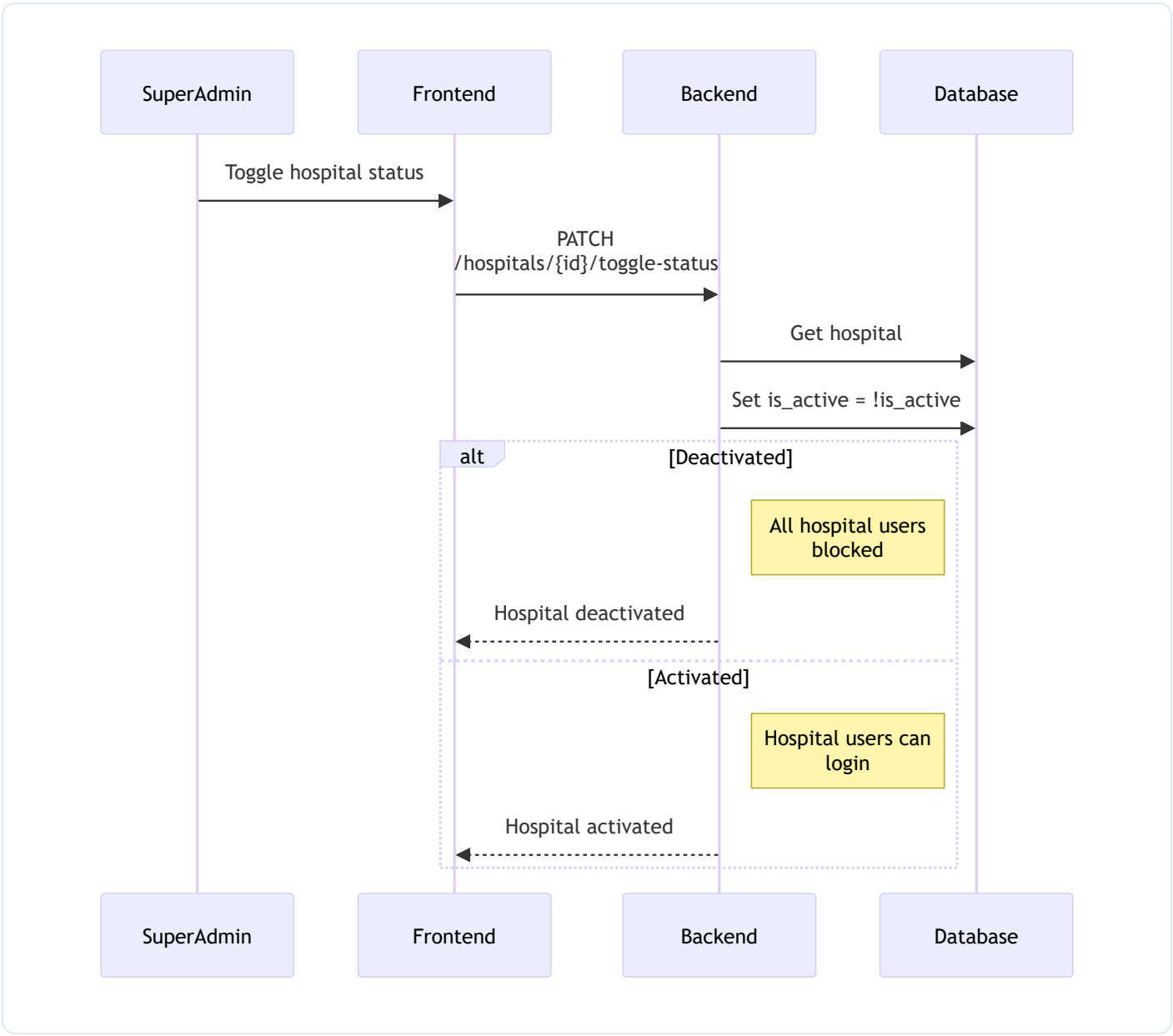
2.2 Hospital Branding Flow



2.3 Logo/Favicon Upload Flow



2.4 Hospital Activation Toggle



Hospital Features:

- **Hospital Creation** - Name, code, email, phone, address, timezone
- **Admin Auto-Creation** - Automatic hospital admin account with temp password
- **Branding** - Custom logo, favicon, header text, tagline, colors
- **Activation Toggle** - Enable/disable hospitals
- **Multi-tenant Isolation** - Complete data separation between hospitals

Key Endpoints:

```
POST   /hospitals/                    # Create hospital (Super Admin)
GET    /hospitals/all                 # List all hospitals
GET    /hospitals/me                  # Get current user's hospital
PATCH  /hospitals/me                  # Update hospital settings
```

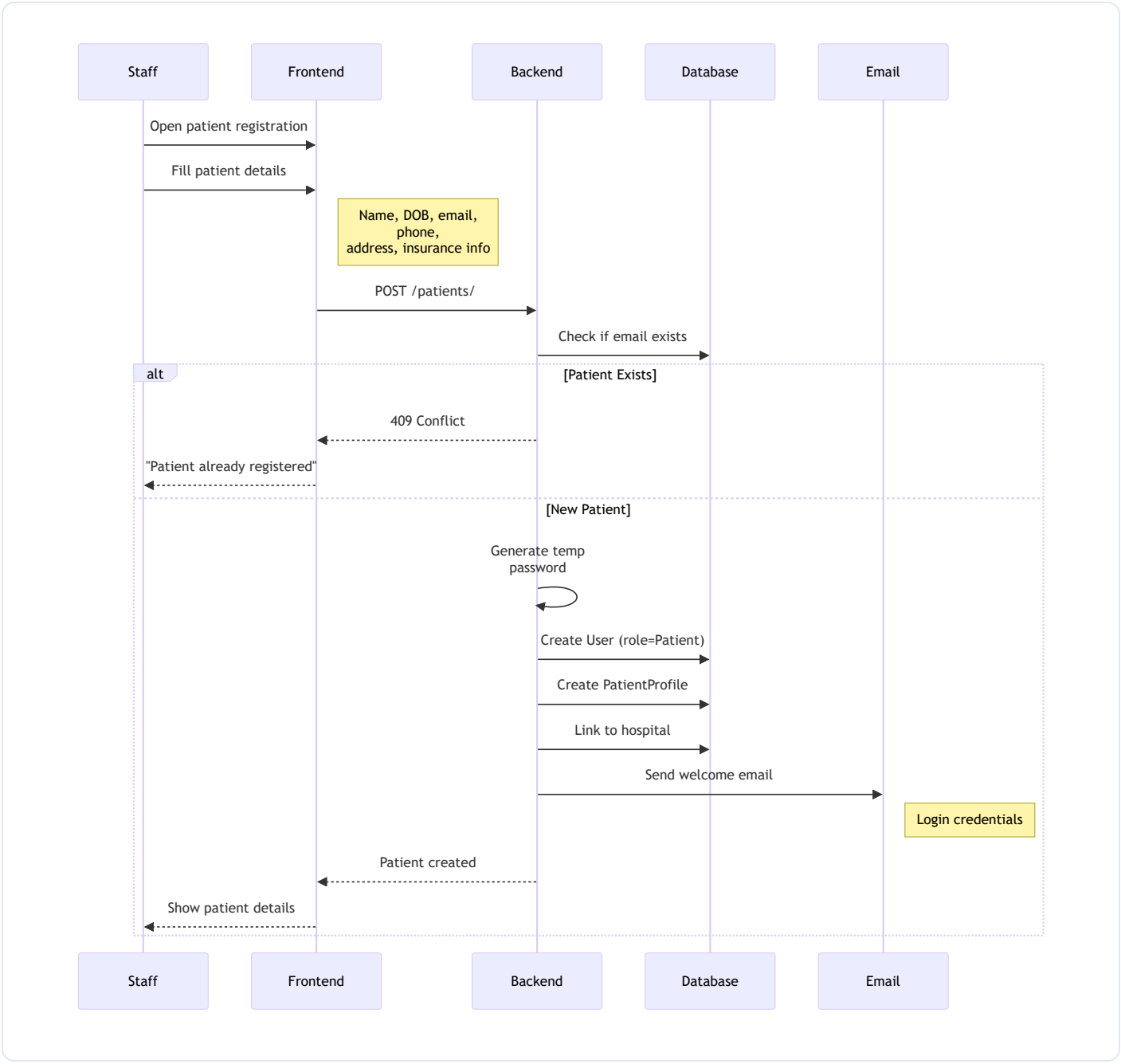
```
POST   /hospitals/me/logo             # Upload hospital logo
POST   /hospitals/me/favicon          # Upload hospital favicon
PATCH  /hospitals/{id}/toggle-status  # Activate/deactivate hospital
```

---

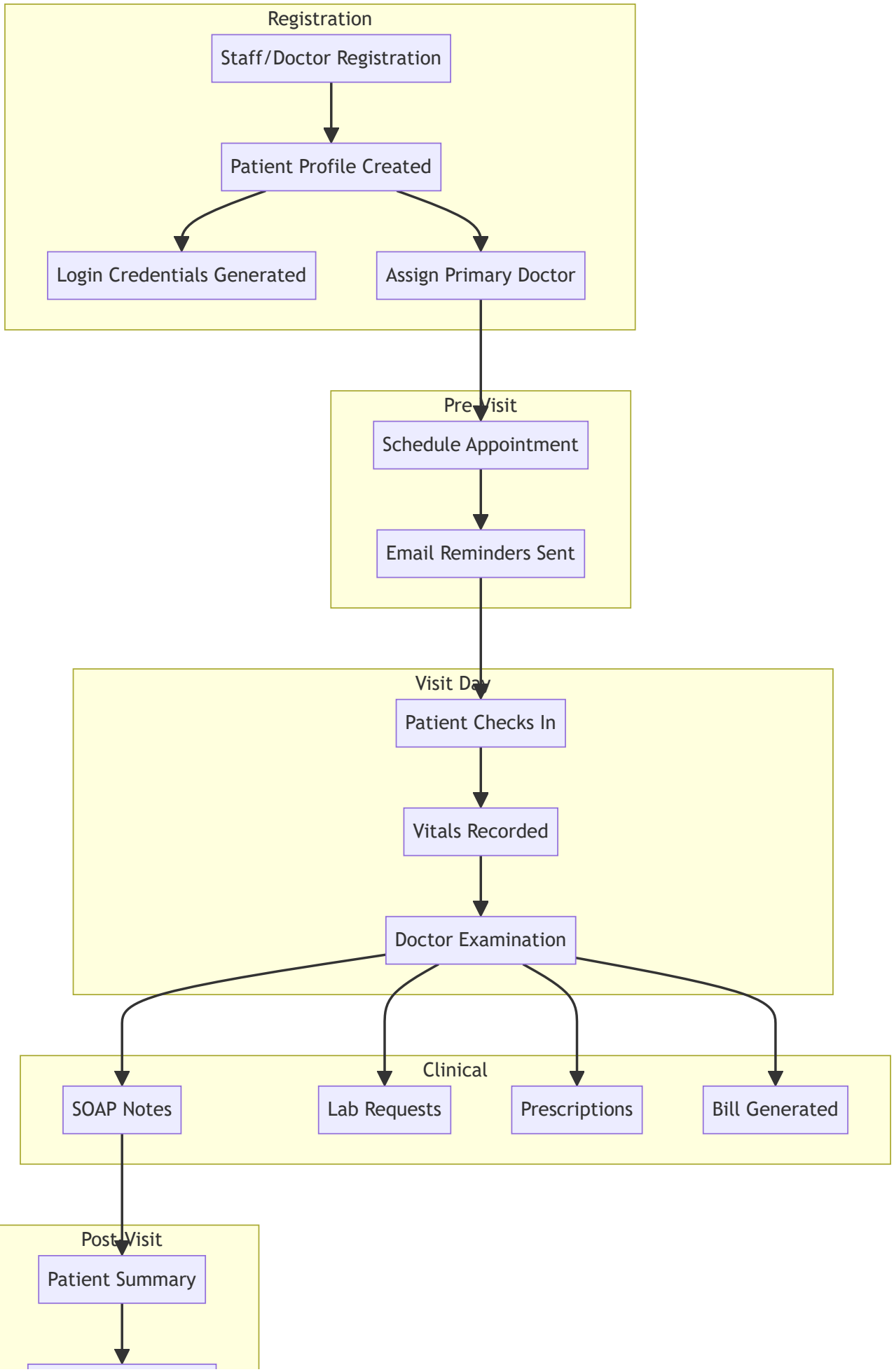
3. Patient Management

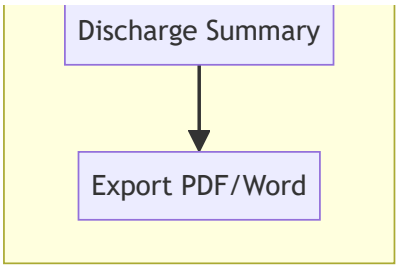
Comprehensive patient lifecycle management from registration to discharge.

3.1 Patient Registration Flow

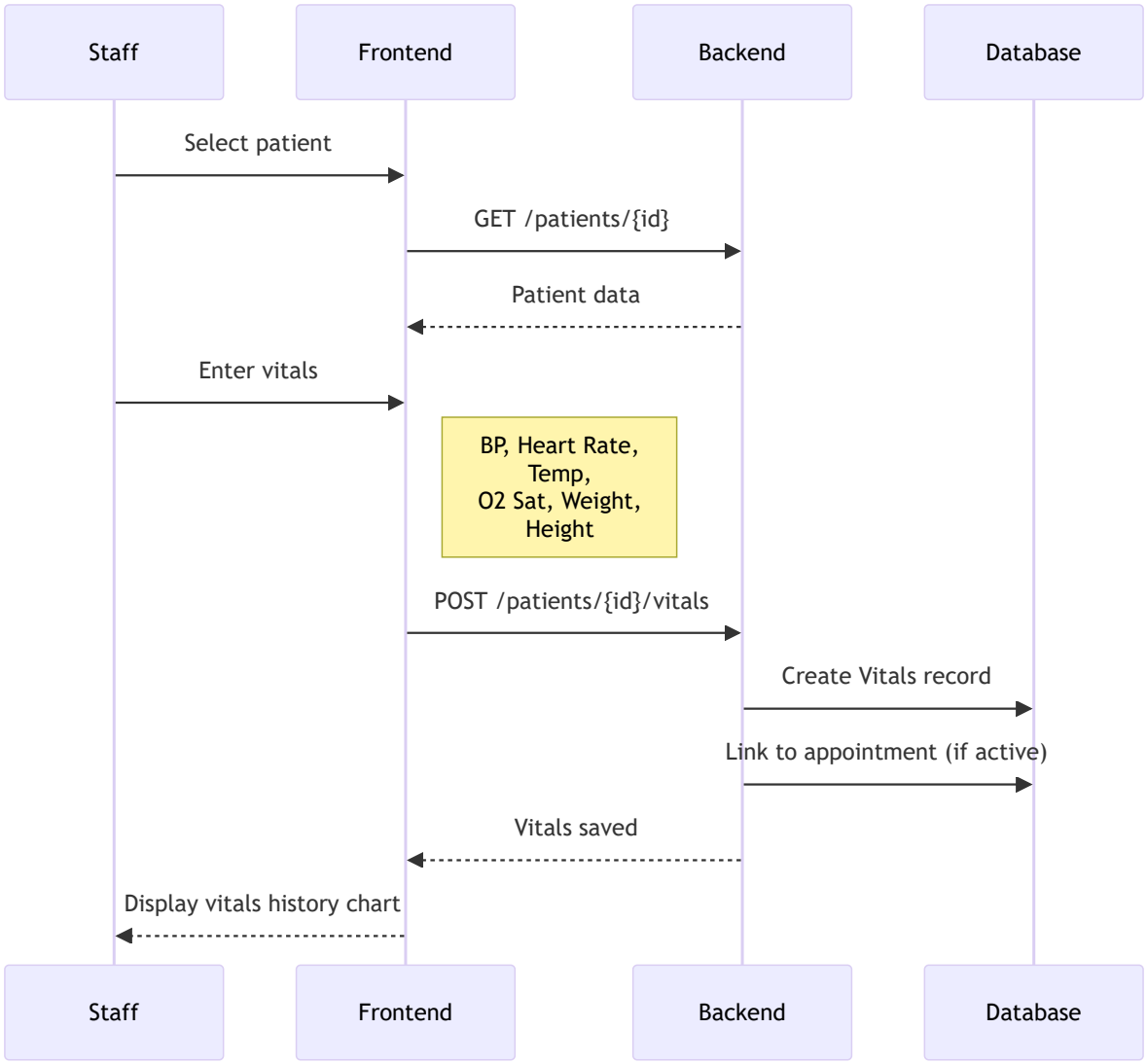


3.2 Complete Patient Lifecycle

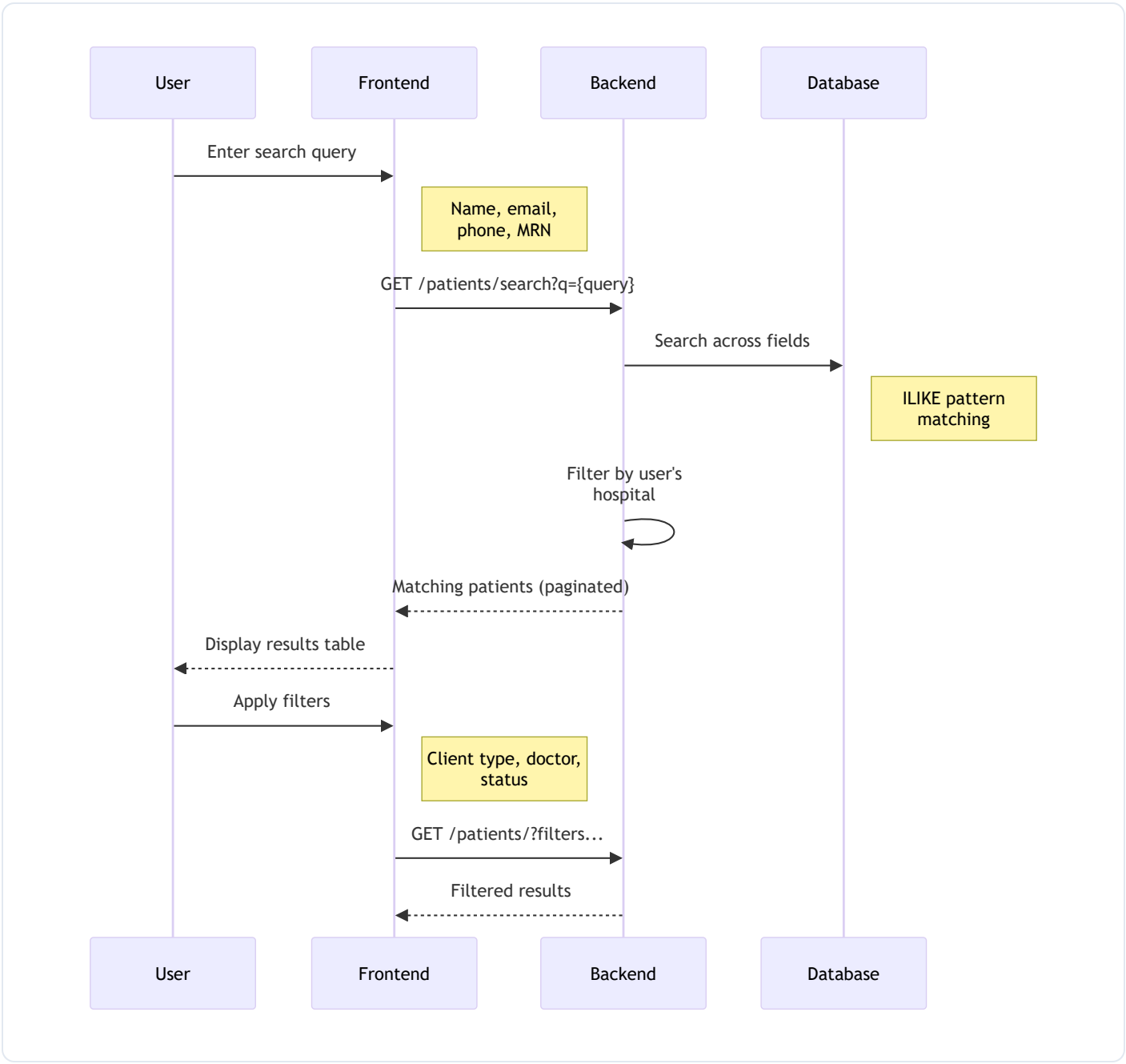

Registration
Staff/Doctor Registration
Patient Profile Created
Login Credentials Generated
Assign Primary Doctor
Pre-Visit
Schedule Appointment
Email Reminders Sent
Visit Day
Patient Checks In
Vitals Recorded
Doctor Examination
Clinical
SOAP Notes
Lab Requests
Prescriptions
Bill Generated
Post-Visit
Patient Summary



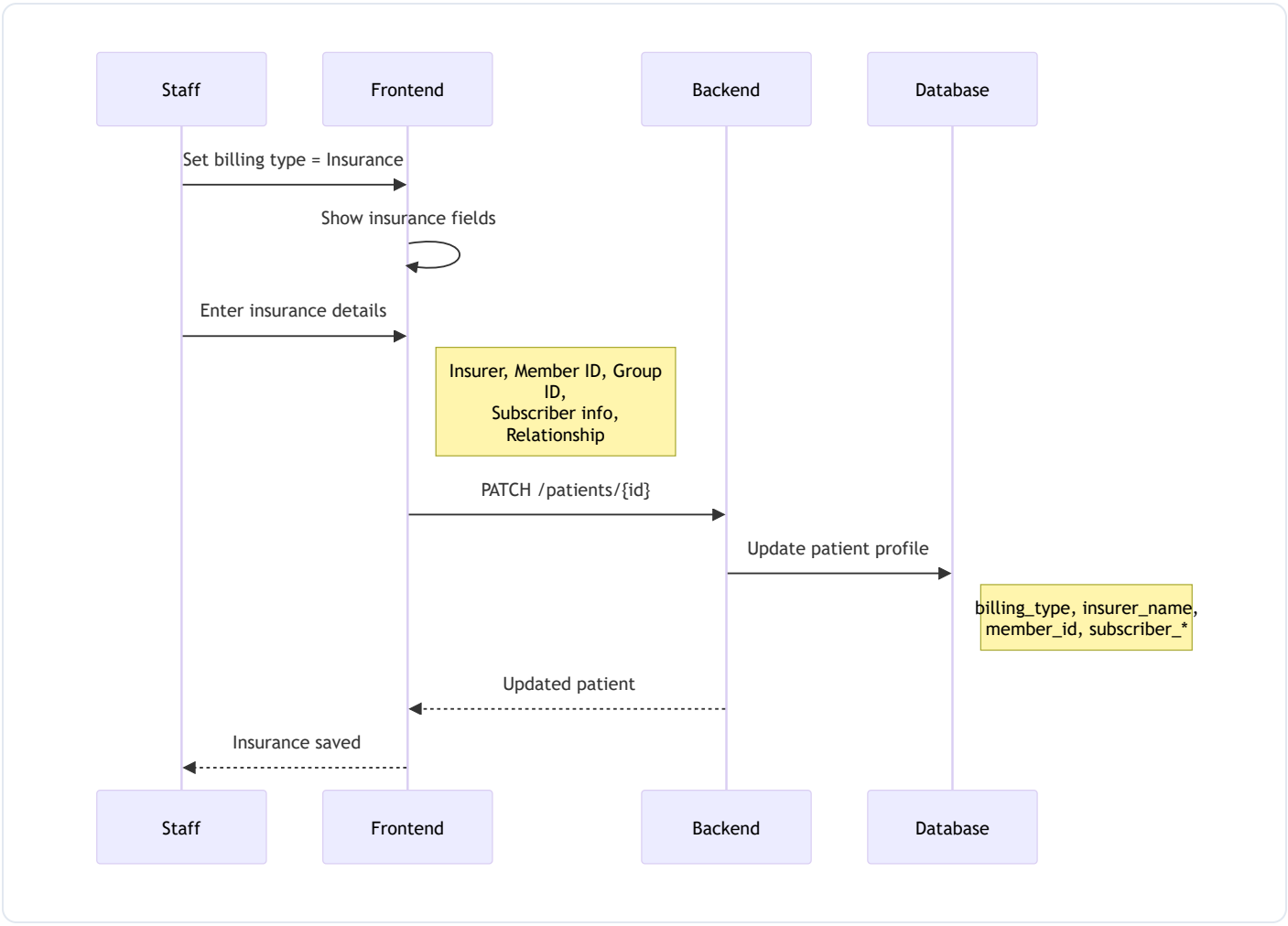
3.3 Vitals Recording Flow



3.4 Patient Search & Filter Flow



3.5 Insurance Assignment Flow



Patient Features:

- **Registration** - Full demographic capture, insurance info, chief complaint
- **Billing Type** - Self-pay or insurance with subscriber details
- **Client Type** - ER, Walk-in, Scheduled, Inpatient
- **Triage Level** - Priority classification
- **Medical History** - Allergies, medications, past history
- **Vitals Tracking** - BP, heart rate, temperature, O2 saturation
- **Doctor Assignment** - Primary care physician assignment

Key Endpoints:

```
POST   /patients/                    # Register patient
GET    /patients/                    # List patients (paginated)
GET    /patients/{id}                # Get patient details
PATCH  /patients/{id}                # Update patient profile
GET    /patients/search              # Search patients
POST   /patients/{id}/vitals         # Record vitals
```

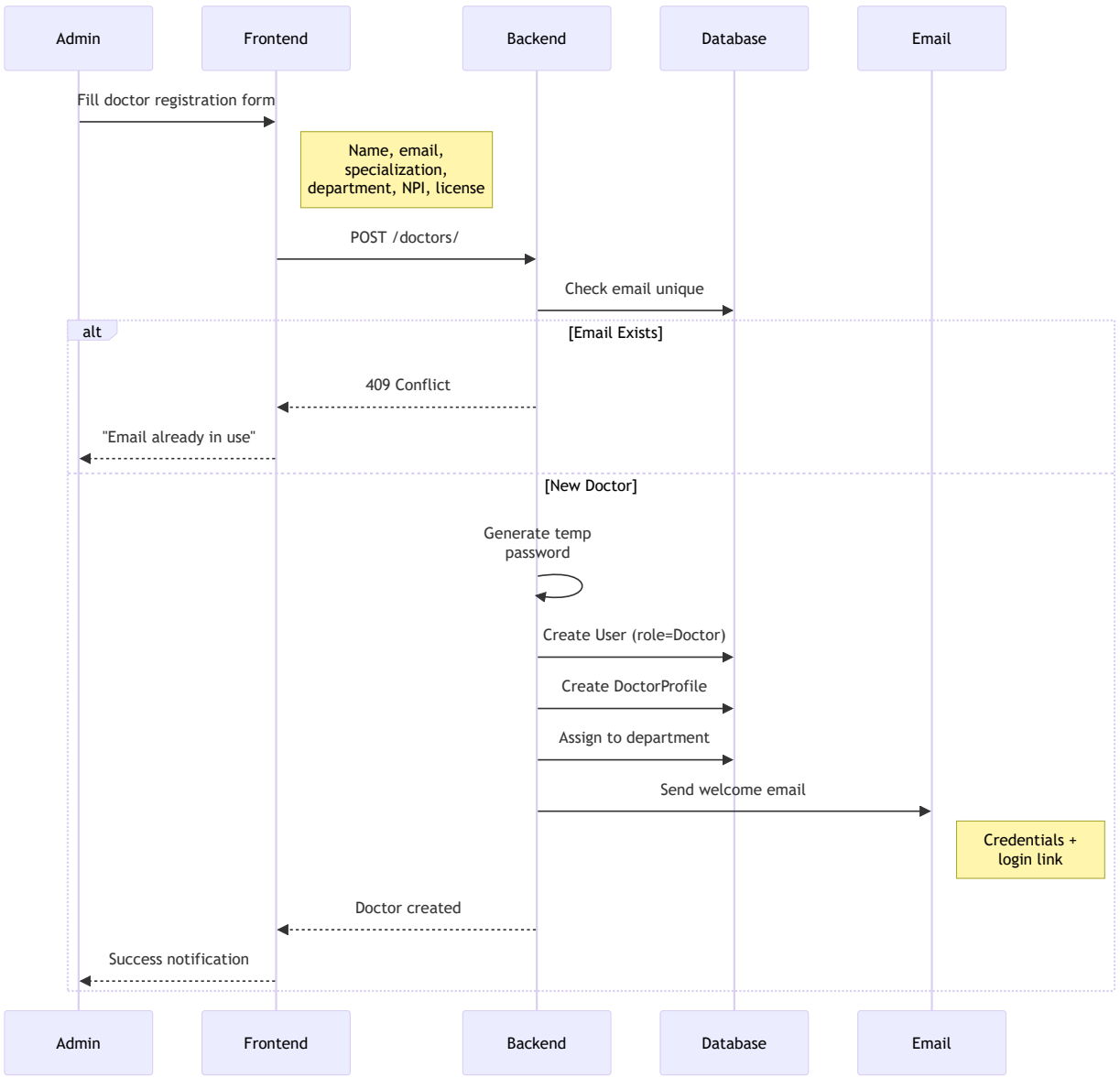
```
GET    /patients/{id}/vitals         # Get vitals history
POST   /patients/{id}/assign-doctor  # Assign primary doctor
```

---

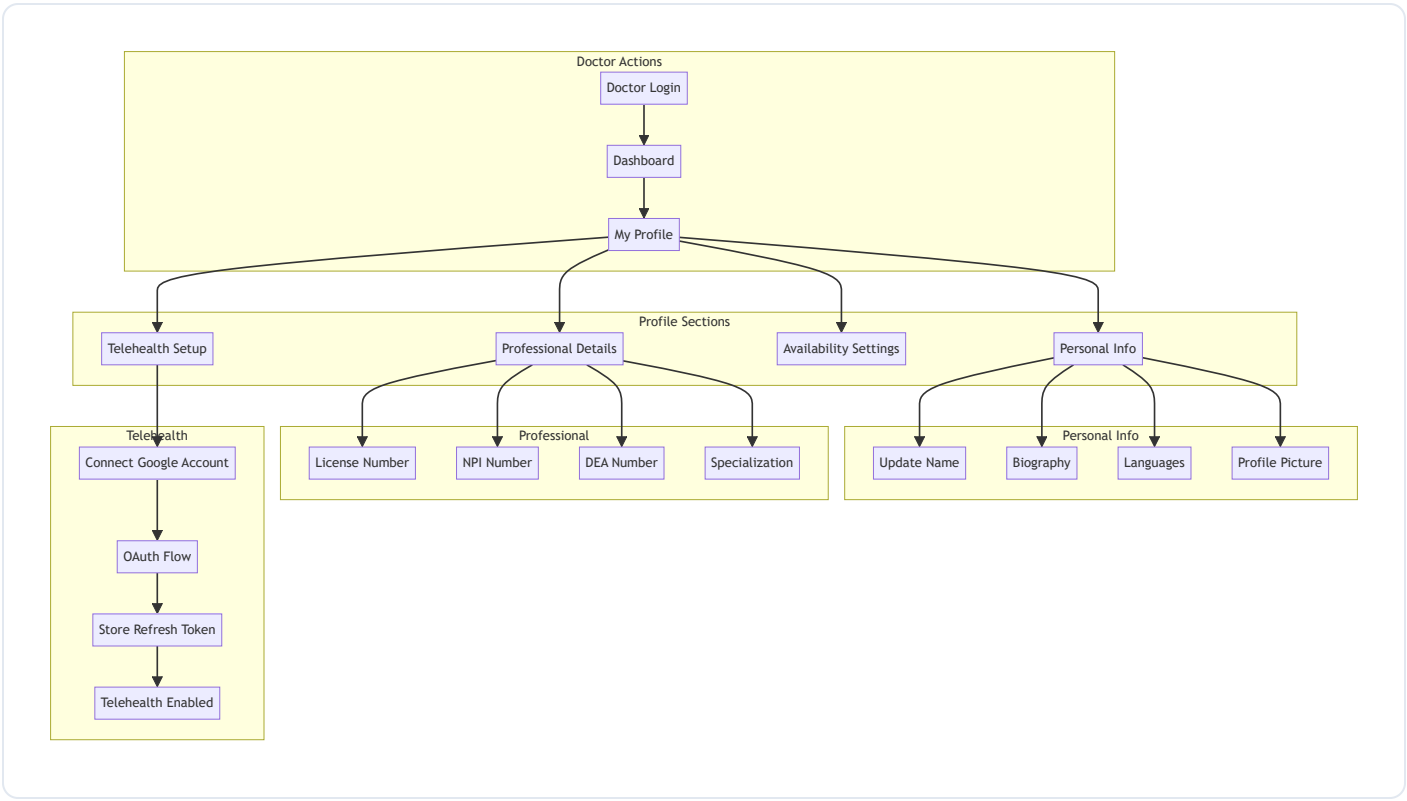
4. Doctor Management

Complete doctor lifecycle from onboarding to profile management.

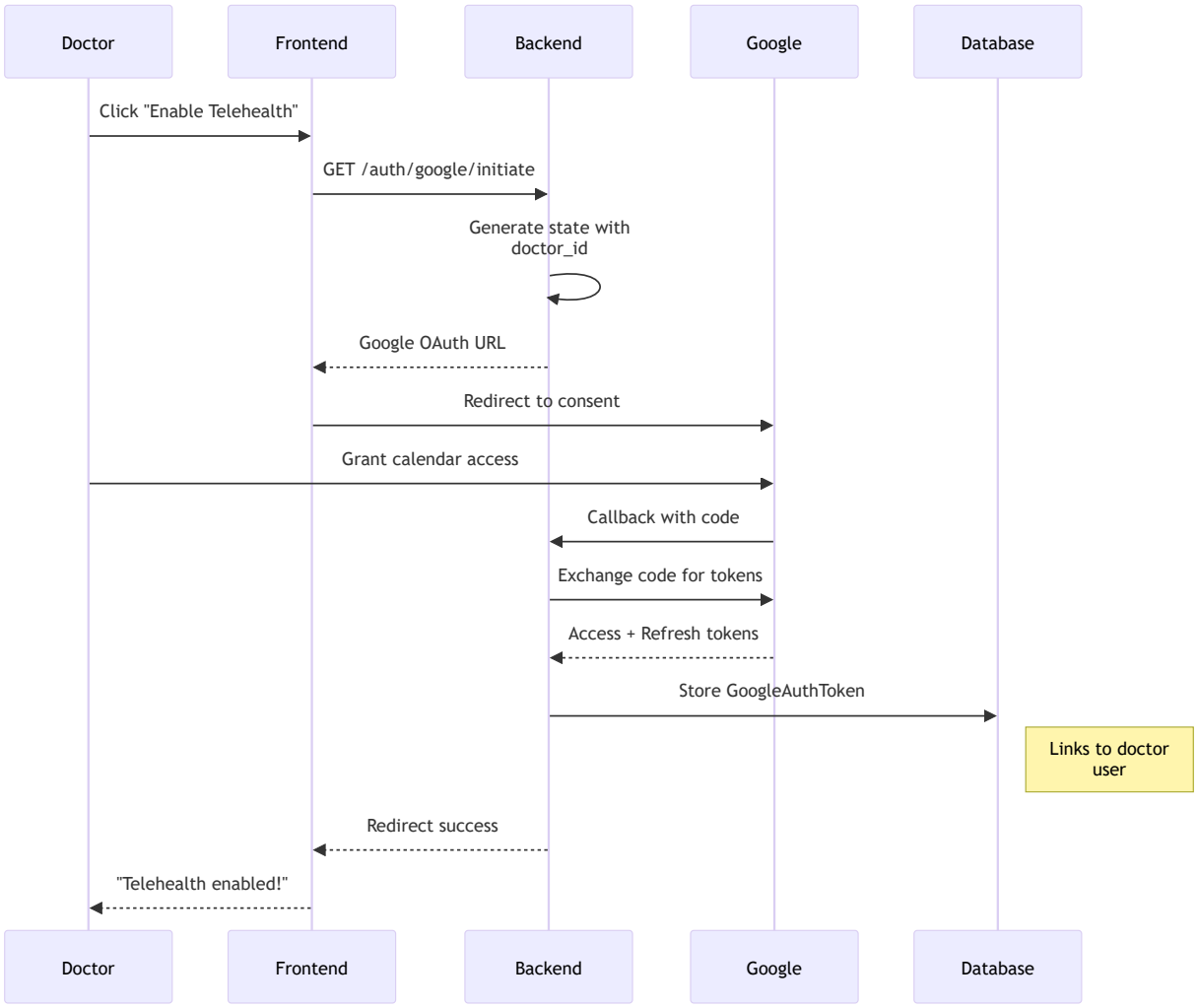
4.1 Doctor Creation Flow



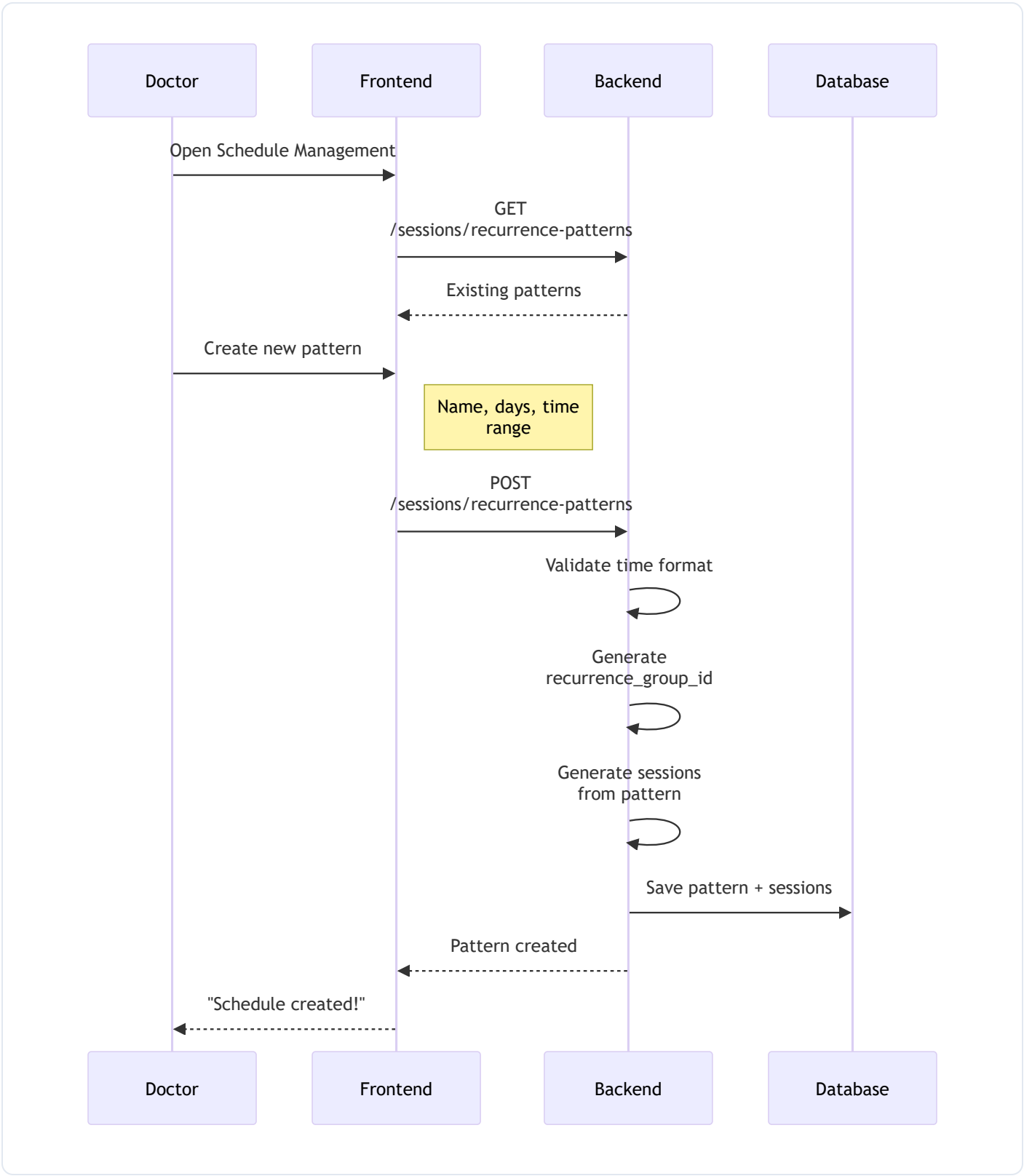
4.2 Doctor Profile Update Flow



4.3 Doctor Telehealth Setup Flow



4.4 Doctor Schedule Setup Flow



Doctor Profile Includes:

- **Personal Info** - Name, email, profile picture
- **Professional** - Medical license, NPI number, DEA number
- **Qualifications** - Specialization, years of experience
- **Department** - Linked to hospital department

- **Telehealth** - Available for virtual visits (requires Google OAuth)
- **Biography** - Doctor's professional summary
- **Languages** - Languages spoken (JSON array)

Key Endpoints:

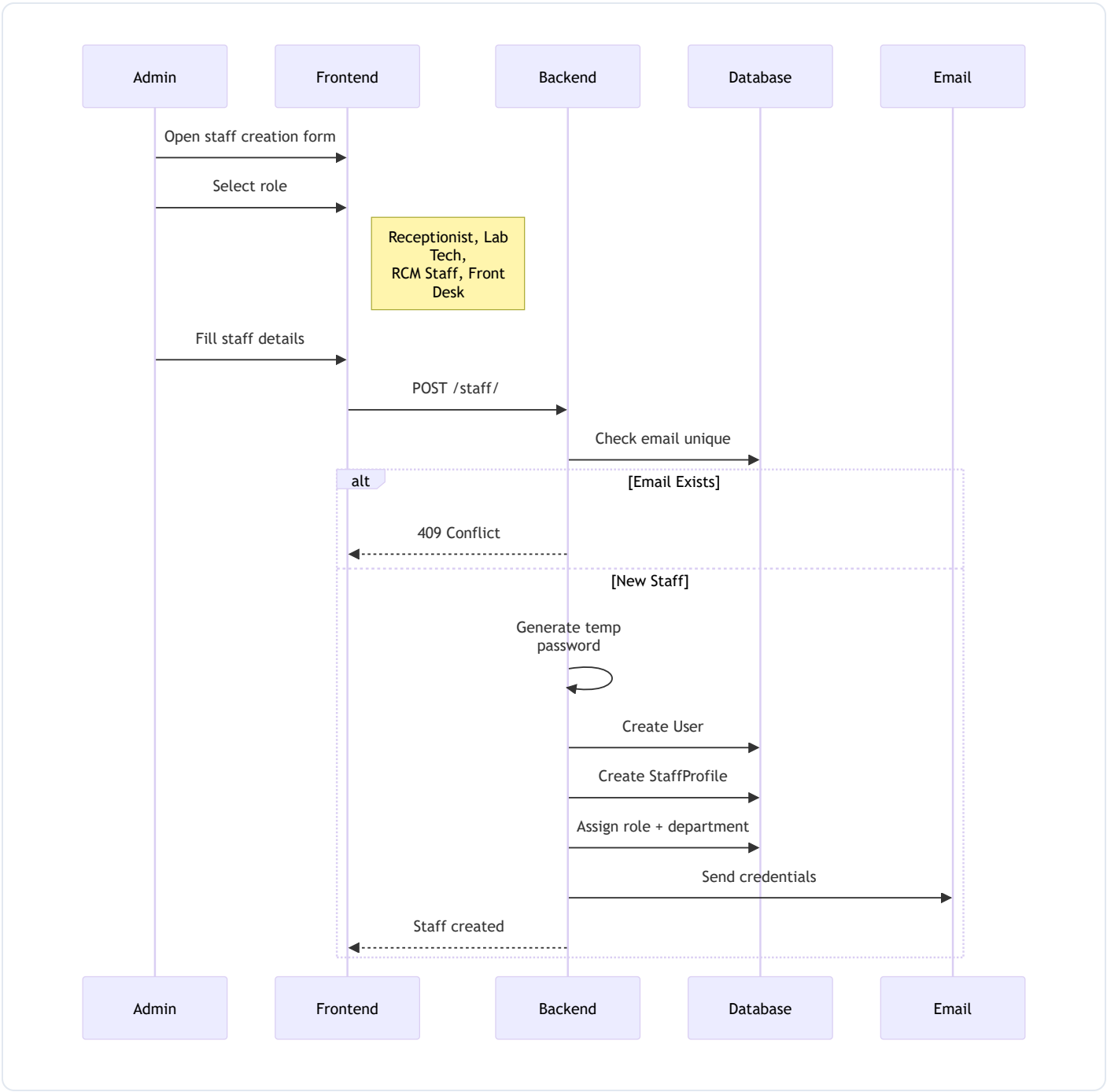
```
POST   /doctors/                    # Create doctor
GET    /doctors/                    # List hospital doctors
GET    /doctors/{id}                # Get doctor details
PATCH  /doctors/{id}                # Update doctor profile
GET    /doctors/me                  # Get current doctor profile
POST   /doctors/me/profile-picture  # Upload profile picture
GET    /auth/google/initiate        # Start telehealth OAuth
```

---

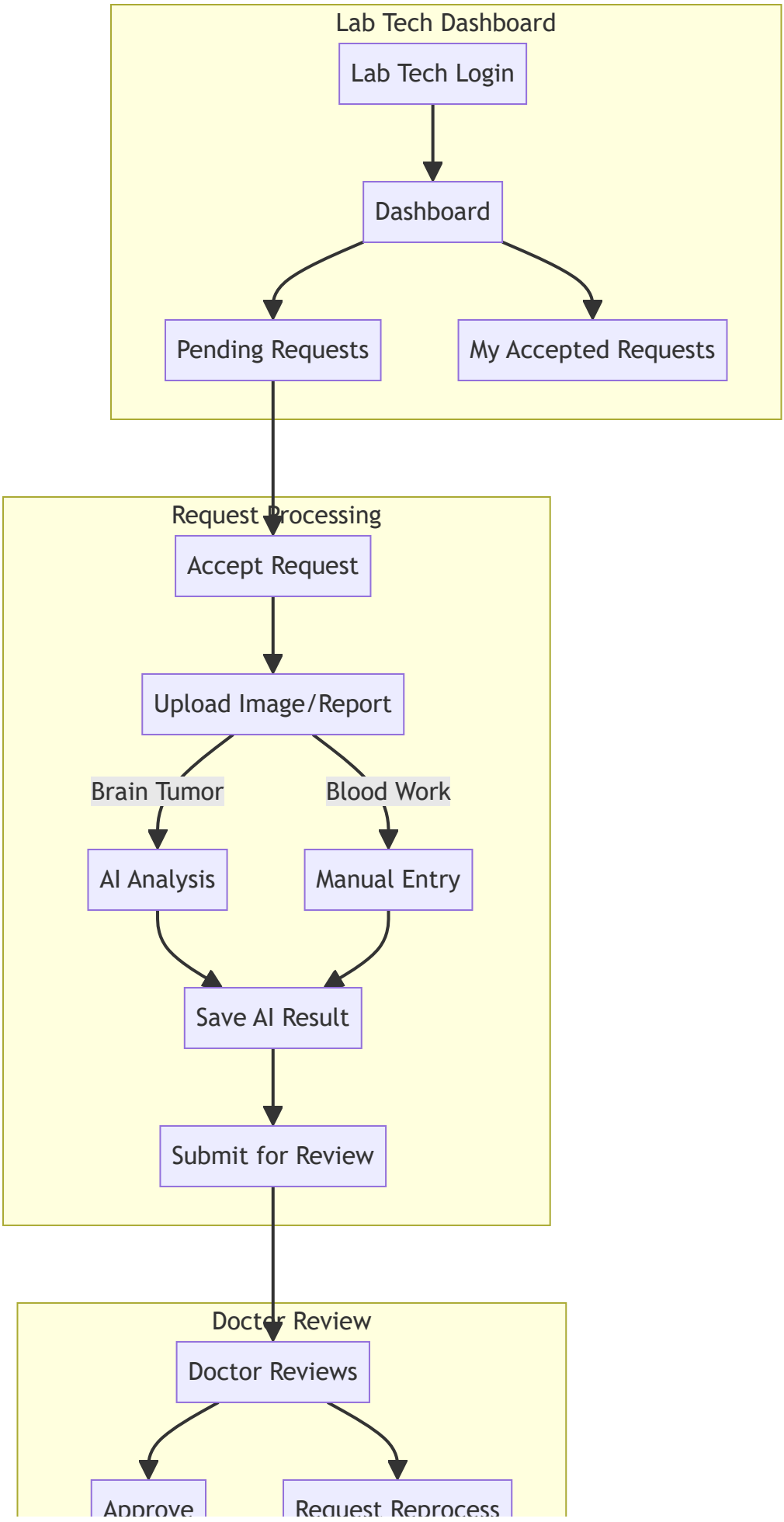
5. Staff Management

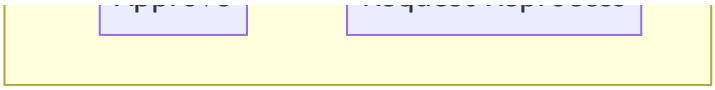
Hospital staff management for non-clinical roles.

5.1 Staff Creation Flow

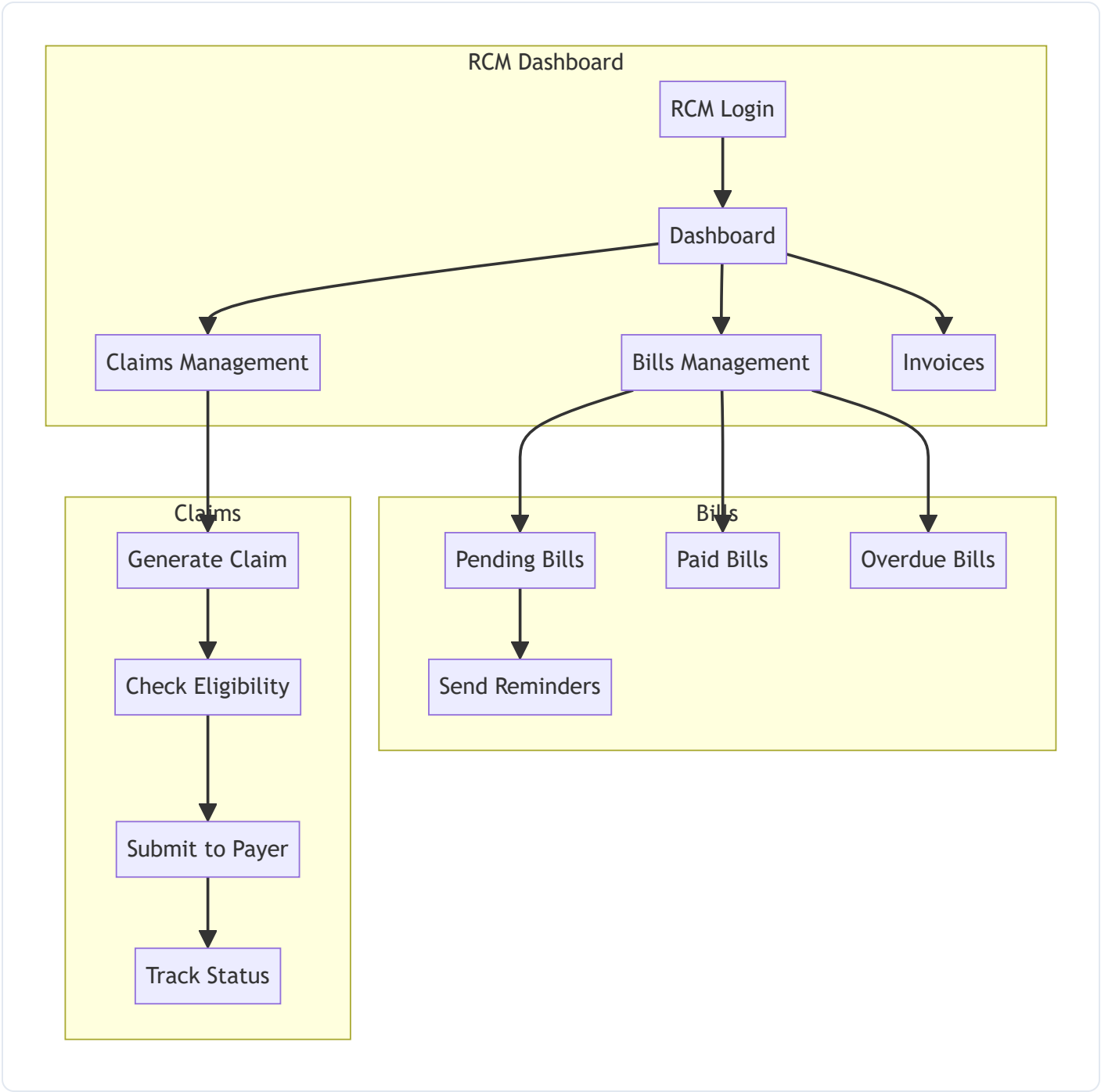


5.2 Lab Technician Workflow


Lab Tech Dashboard
Lab Tech Login
Dashboard
Pending Requests
My Accepted Requests
Request Processing
Accept Request
Upload Image/Report
Brain Tumor
Blood Work
AI Analysis
Manual Entry
Save AI Result
Submit for Review
Doctor Review
Doctor Reviews
Approve
Request Reprocess



5.3 RCM Staff Workflow



Staff Roles:

Role	Description	Key Permissions
Receptionist	Front desk	Patient registration, appointments, waitlist

Role	Description	Key Permissions
Lab Technician	Laboratory	Lab requests, AI analysis, upload results
RCM Staff	Revenue cycle	Billing, claims, invoices, payments
Front Desk	Admin support	Patient check-in, scheduling

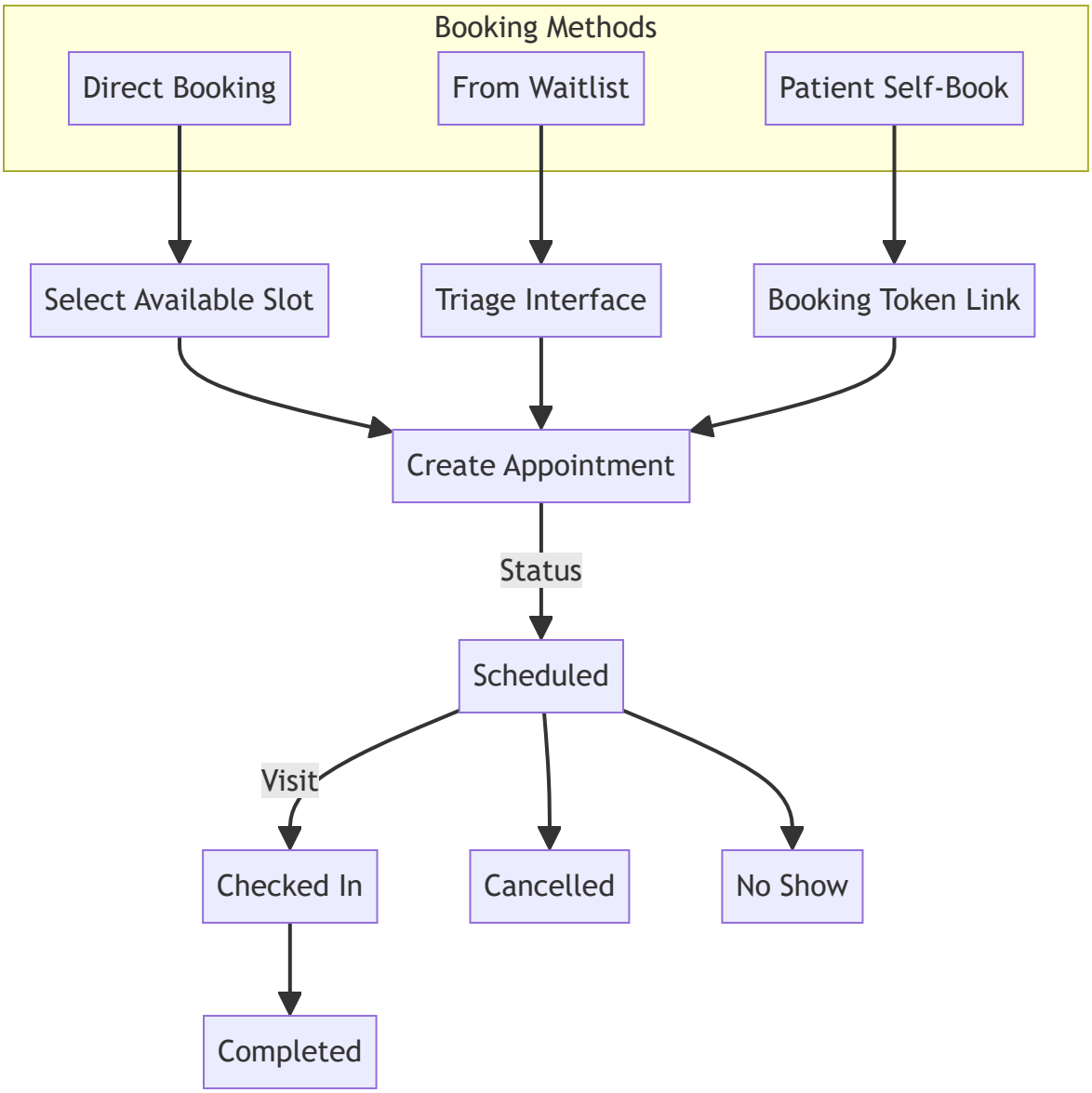
Key Endpoints:

```
POST   /staff/                    # Create staff member
GET    /staff/                    # List hospital staff
GET    /staff/{id}                # Get staff details
PATCH  /staff/{id}                # Update staff profile
DELETE /staff/{id}                # Deactivate staff
```

---

6. Appointment System

Sophisticated appointment scheduling with multiple booking methods and integrated telehealth via Google Meet.



Appointment Features:

Booking Flow:

1. **Select Doctor** - Choose from available doctors
2. **Select Date** - Pick appointment date
3. **View Sessions** - See doctor's available sessions
4. **Select Slot** - Choose specific time slot
5. **Provide Details** - Reason for visit, telehealth preference
6. **Add ICD Codes** - Optional diagnosis codes
7. **Confirmation** - Email sent to patient

Appointment Statuses:

Status	Description
`scheduled`	Initial booking
`checked_in`	Patient arrived
`in_progress`	Currently being seen
`completed`	Visit finished
`cancelled`	Appointment cancelled
`no_show`	Patient didn't appear

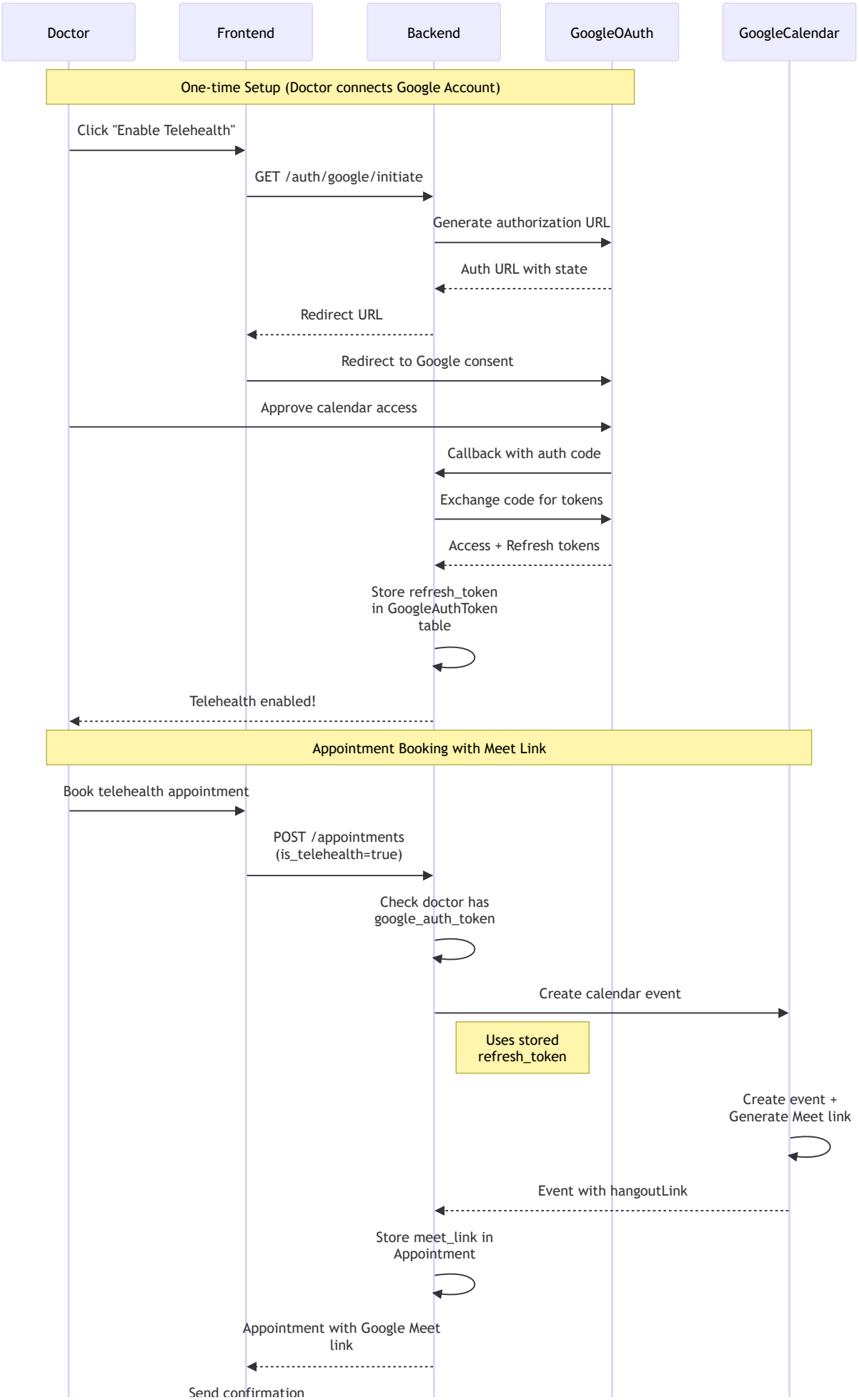
Advanced Features:

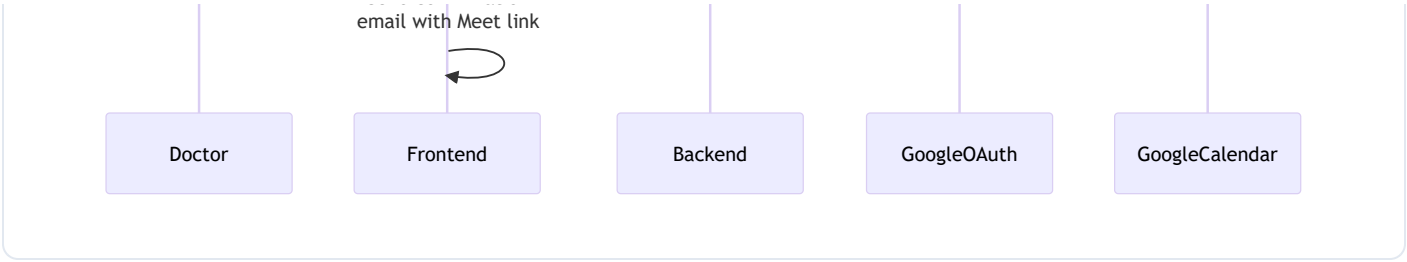
- **Telehealth** - Google Meet integration for virtual visits
- **Rescheduling** - Move to different slot
- **Swapping** - Exchange slots between patients
- **Doctor Reassignment** - Transfer to different doctor
- **ICD Codes** - Multiple diagnosis codes per appointment
- **Email Notifications** - Confirmation, cancellation, reminder emails

---

6.1 Google Meet Integration (Telehealth)

The system provides seamless telehealth functionality through Google Calendar and Google Meet integration.


Doctor
Frontend
Backend
GoogleOAuth
GoogleCalendar
One-time Setup (Doctor connects Google Account)
Click "Enable Telehealth"
GET /auth/google/initiate
Generate authorization URL
Auth URL with state
Redirect URL
Redirect to Google consent
Approve calendar access
Callback with auth code
Exchange code for tokens
Access + Refresh tokens
Store refresh_token in GoogleAuthToken table
Telehealth enabled!
Appointment Booking with Meet Link
Book telehealth appointment
POST /appointments (is_telehealth=true)
Check doctor has google_auth_token
Create calendar event
Uses stored refresh_token
Create event + Generate Meet link
Event with hangoutLink
Store meet_link in Appointment
Appointment with Google Meet link
Send confirmation



How Google Meet Links are Generated:

1. **Doctor OAuth Setup** (One-time):

```
# Doctor must first connect their Google account
# This stores their refresh_token for later use

# Required Google OAuth Scopes:
SCOPES = ['https://www.googleapis.com/auth/calendar.events']
```

2. **During Appointment Booking**:

```
# When is_telehealth=True, the system:

# 1. Validates doctor has connected Google account
if not doctor.google_auth_token or not doctor.google_auth_token.refresh_token:
    raise HTTPException(
        status_code=400,
        detail="This doctor has not enabled telehealth by connecting their Google account."
    )

# 2. Creates Google Calendar event with Meet conference
meet_link = create_calendar_event(
    refresh_token=doctor.google_auth_token.refresh_token,
    summary=f"Appointment with {patient_name}",
    start_time=slot.start_time,
    end_time=slot.end_time,
    attendees=[patient.email, doctor.email]
)

# 3. Stores the Meet link in the appointment
new_appointment = Appointment(
    google_meet_link=meet_link,
    is_telehealth=True,
    ...
)
```

3. **Google Calendar Event Creation**:

```
# Creates event with automatic Meet link generation
event = {
    'summary': summary,
    'start': {'dateTime': start_time.isoformat() + 'Z', 'timeZone': 'UTC'},
    'end': {'dateTime': end_time.isoformat() + 'Z', 'timeZone': 'UTC'},
    'attendees': [{'email': email} for email in attendees],
    'conferenceData': {
        'createRequest': {
            'requestId': f"{start_time.isoformat()}-{summary}",
            'conferenceSolutionKey': {'type': 'hangoutsMeet'}  # This generates the Meet link
        }
    },
    'reminders': {'useDefault': True},
}

# Insert event with conferenceDataVersion=1 to enable Meet
created_event = service.events().insert(
    calendarId='primary',
    body=event,
    conferenceDataVersion=1
).execute()

# Return the Meet link
return created_event.get('hangoutLink')  # e.g., "https://meet.google.com/abc-defg-hij"
```

Meet Link Lifecycle:

Event	Meet Link Action
Appointment Created	New Meet link generated
Appointment Rescheduled	New Meet link generated for new time
Appointment Swapped	New Meet links generated for both appointments
Telehealth Disabled	Meet link set to `null`
Appointment Cancelled	Meet link removed (appointment deleted)

Email Integration:

The Meet link is automatically included in: - Appointment confirmation emails (to patient and doctor) - Reminder emails - Swap notification emails

```
<!-- Email template includes Join button -->
<a href="{meet_link}" class="button" target="_blank">Join Google Meet</a>
```

Required Environment Variables:

```
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback
```

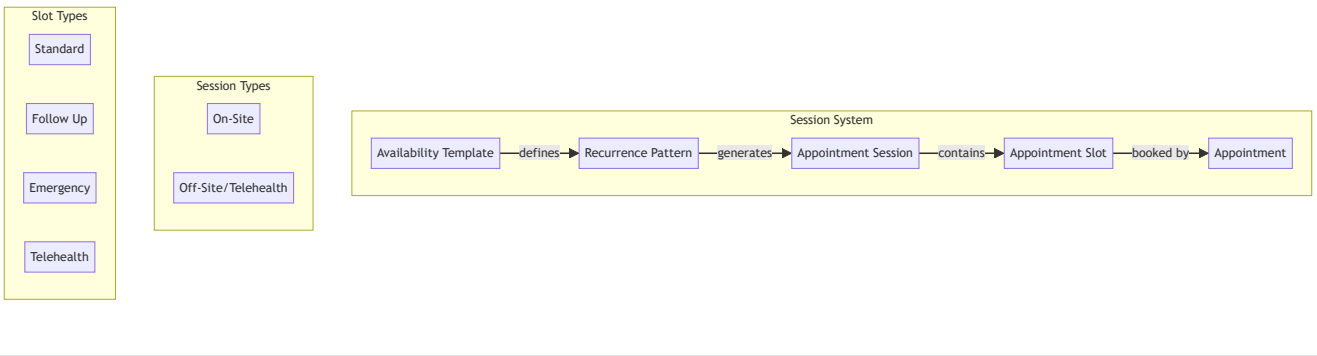
Key Endpoints:

```
POST   /appointments/                # Create appointment
GET    /appointments/                # List appointments (paginated)
GET    /appointments/weekly          # Doctor's weekly view
GET    /appointments/{id}            # Get specific appointment
PATCH  /appointments/{id}/cancel     # Cancel appointment
PATCH  /appointments/{id}/reschedule  # Reschedule appointment
POST   /appointments/swap            # Swap two appointments
PATCH  /appointments/{id}/reassign-doctor  # Reassign to different doctor

# Google OAuth endpoints
GET    /auth/google/initiate         # Start OAuth flow
GET    /auth/google/callback         # OAuth callback
```

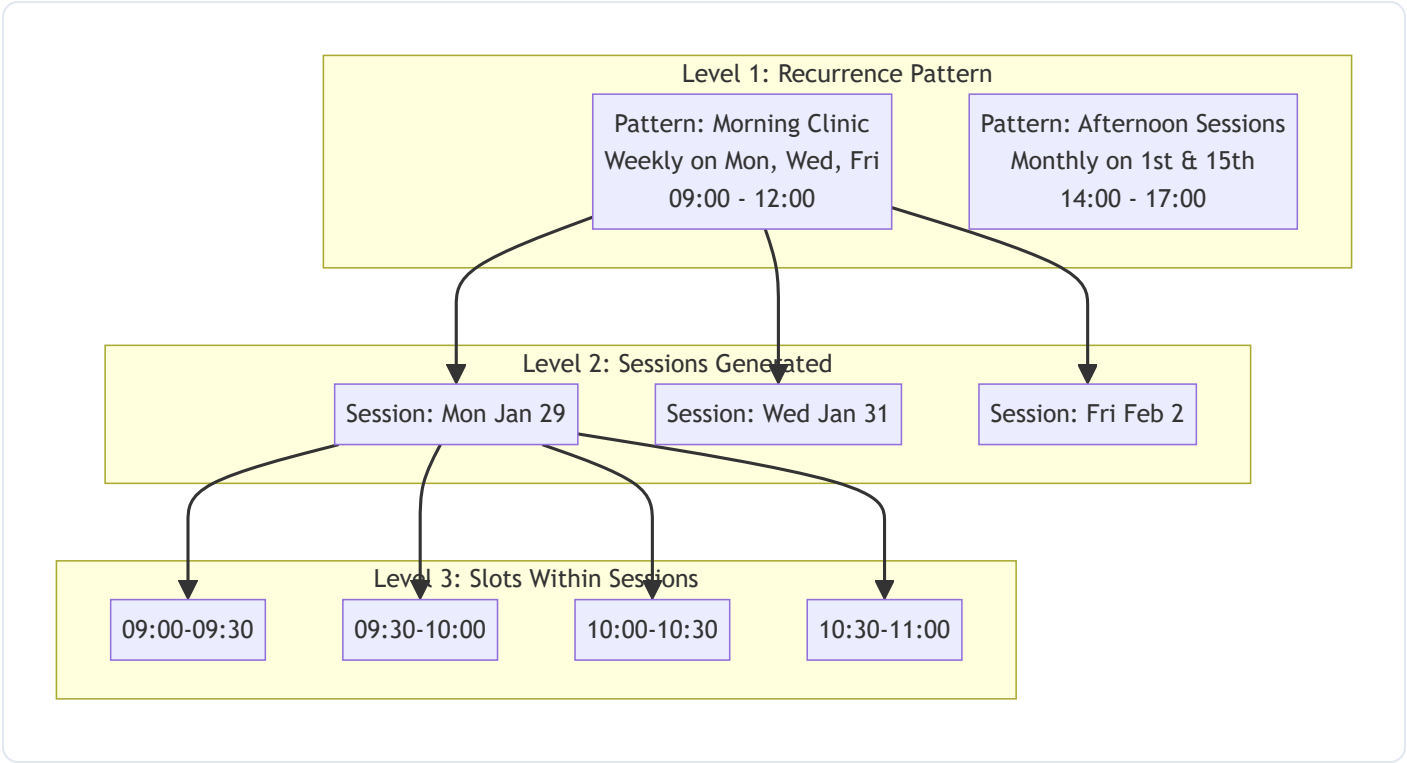
---

7. Session & Slot Management

Hierarchical scheduling system enabling complex availability patterns with support for daily, weekly, and monthly recurrence.



7.1 Hierarchy Overview



1. Recurrence Pattern (Template)

- Defines the recurring schedule rule
- Stored as `SessionRecurrencePattern` in database
- Has unique `recurrence_group_id` for tracking
- Generates sessions automatically

2. Appointment Session

- Actual time block on a specific date
- Created from recurrence patterns
- Status: Available, Booked, Cancelled
- Contains multiple slots

3. Appointment Slot

- Individual bookable time unit
- Customizable duration (15, 30, 45, 60 minutes)
- Can be independently blocked
- Links to appointment when booked

7.2 Recurrence Pattern Types

The system supports three types of recurrence patterns:

Daily Recurrence

Sessions repeat every day within a date range.

```
{
    "name": "Daily Morning Clinic",
    "session_type": "on_site",
    "start_time_of_day": "09:00",
    "end_time_of_day": "12:00",
    "recurrence_config": {
        "duration": "daily",
        "start_date": "2025-02-01",
        "end_date": "2025-02-28",        # OR
        "repeat_count": 30,                # Number of occurrences
        "selected_option": "on_day"
    }
}
```

Weekly Recurrence

Sessions repeat on specific days of the week.

```
{
    "name": "Weekly Consultations",
    "session_type": "on_site",
    "start_time_of_day": "09:00",
    "end_time_of_day": "13:00",
    "recurrence_config": {
        "duration": "weekly",
        "start_date": "2025-02-01",
        "end_date": "2025-12-31",
        "selected_option": "on_day",
        "selected_days": ["Mon", "Wed", "Fri"]  # Days of week
    }
}
```

Day Mapping:

Short	Full	Weekday
Mon	Monday	0
Tue	Tuesday	1

Short	Full	Weekday
Wed	Wednesday	2
Thu	Thursday	3
Fri	Friday	4
Sat	Saturday	5
Sun	Sunday	6

Monthly Recurrence - On Specific Dates

Sessions repeat on specific dates of each month (e.g., 1st, 15th, 28th).

```
{
    "name": "Monthly Check-ups",
    "session_type": "on_site",
    "start_time_of_day": "14:00",
    "end_time_of_day": "17:00",
    "recurrence_config": {
        "duration": "monthly",
        "start_date": "2025-01-01",
        "end_date": "2025-12-31",
        "selected_option": "on_date",       # Key difference!
        "month_days": [1, 15]               # 1st and 15th of each month
    }
}
```

Monthly Recurrence - On Specific Day

Sessions repeat on a specific week and day of each month (e.g., "First Monday", "Last Friday").

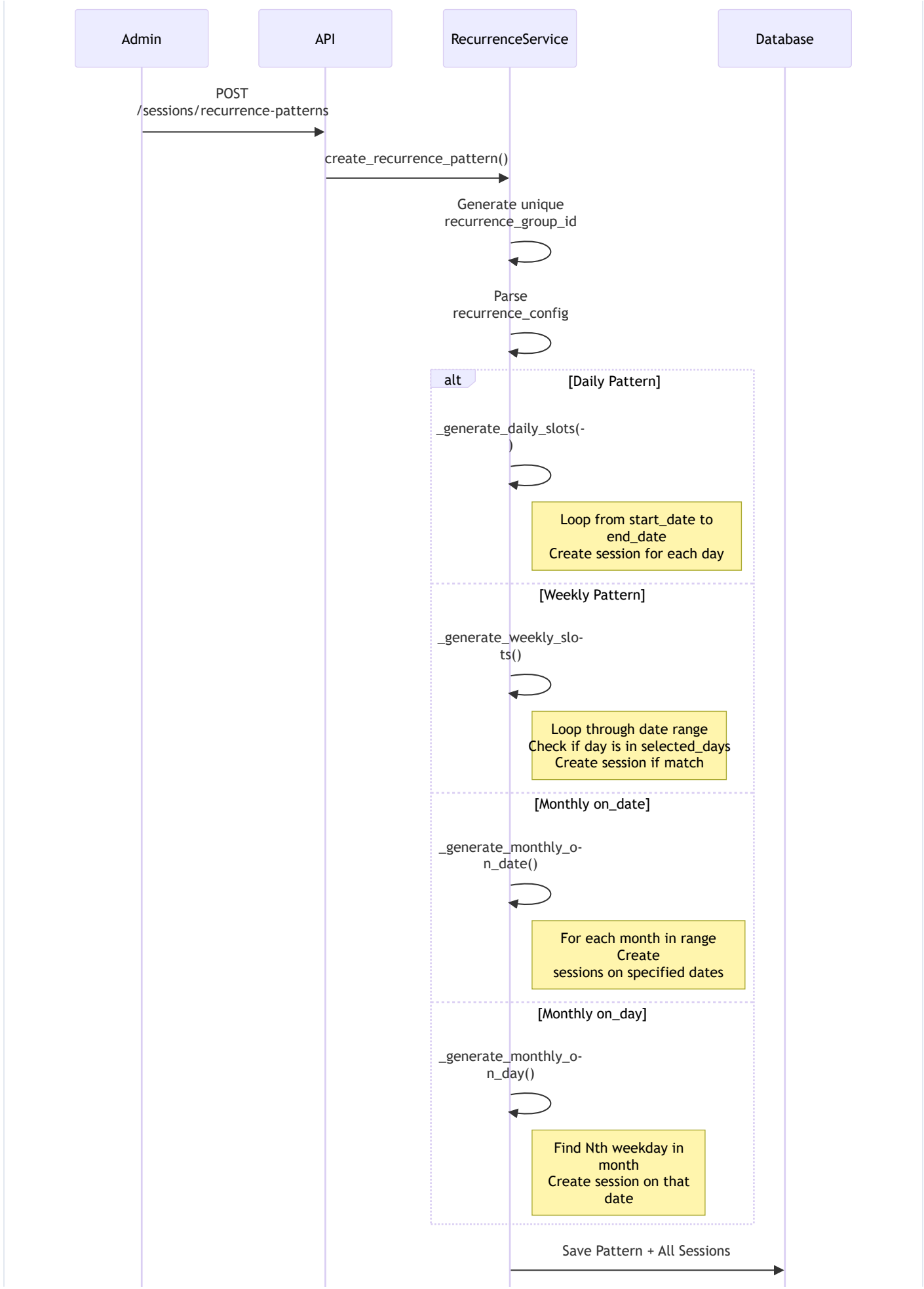
```
{
    "name": "First Monday Clinic",
    "session_type": "on_site",
    "start_time_of_day": "10:00",
    "end_time_of_day": "14:00",
    "recurrence_config": {
        "duration": "monthly",
        "start_date": "2025-01-01",
        "end_date": "2025-12-31",
        "selected_option": "on_day",        # Key difference!
        "week": "first",                    # first, second, third, fourth, last
        "week_day": "Monday"                # Day of week
```

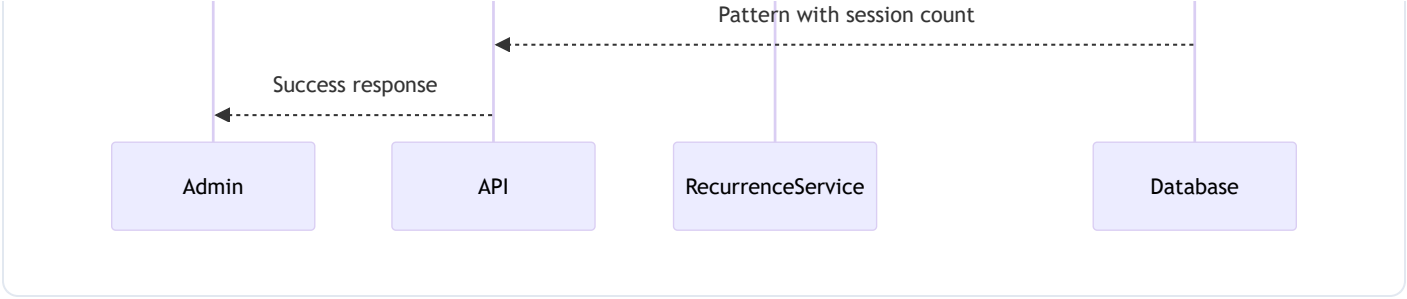
```
    }
}
```

Week Mapping:

Value	Meaning
first	1st occurrence
second	2nd occurrence
third	3rd occurrence
fourth	4th occurrence
last	Last occurrence

7.3 Session Generation Logic


Admin
API
RecurrenceService
Database
POST /sessions/recurrence-patterns
create_recurrence_pattern()
Generate unique recurrence_group_id
Parse recurrence_config
alt
[Daily Pattern]
_generate_daily_slots()
Loop from start_date to end_date
Create session for each day
[Weekly Pattern]
_generate_weekly_slots()
Loop through date range
Check if day is in selected_days
Create session if match
[Monthly on_date]
_generate_monthly_on_date()
For each month in range
Create
sessions on specified dates
[Monthly on_day]
_generate_monthly_on_day()
Find Nth weekday in month
Create session on that date
Save Pattern + All Sessions



Code Logic for Weekly Generation:

```
def _generate_weekly_slots(pattern, start_date, end_date, start_time, end_time, config):
    slots = []
    selected_days = config.get('selected_days', [])  # e.g., ["Mon", "Wed", "Fri"]

    # Convert day names to weekday numbers
    target_weekdays = [DAY_MAPPING[day] for day in selected_days]  # [0, 2, 4]

    current_date = start_date

    while current_date <= end_date:
        # Check if current day is in selected days
        if current_date.weekday() in target_weekdays:
            slot_start = datetime.combine(current_date, start_time)
            slot_end = datetime.combine(current_date, end_time)

            slot = AppointmentSession(
                doctor_user_id=pattern.doctor_user_id,
                name=pattern.name,
                session_type=pattern.session_type,
                start_time=slot_start,
                end_time=slot_end,
                status=SessionStatus.AVAILABLE,
                is_recurring=True,
                recurrence_group_id=pattern.recurrence_group_id
            )
            slots.append(slot)

        current_date += timedelta(days=1)

    return slots
```

Code Logic for Monthly "on_day" (e.g., "First Monday"):

```
def _find_weekday_in_month(year, month, weekday, week_index):
    """
    Find the Nth occurrence of a weekday in a month.
    week_index: 0=first, 1=second, 2=third, 3=fourth, -1=last
    """
```

```
    # Find all occurrences of the target weekday in the month
    occurrences = []
    for day in range(1, days_in_month + 1):
        date = datetime(year, month, day).date()
        if date.weekday() == weekday:
            occurrences.append(date)

    # Return the requested occurrence
    if week_index == -1:
        return occurrences[-1]  # Last occurrence
    elif 0 <= week_index < len(occurrences):
        return occurrences[week_index]

    return None
```

7.4 Slot Management Features

Feature	Description
Auto-Generation	Slots created from session template based on default duration
Manual Creation	Add custom slots within a session
Blocking	Temporarily block slots (lunch breaks, personal time)
Unblocking	Re-enable blocked slots
Duration	Configurable: 15, 30, 45, 60 minutes
Overlap Prevention	System prevents overlapping slot times
Waitlist Integration	Slots show badge with matching waitlist count

7.5 Key Endpoints

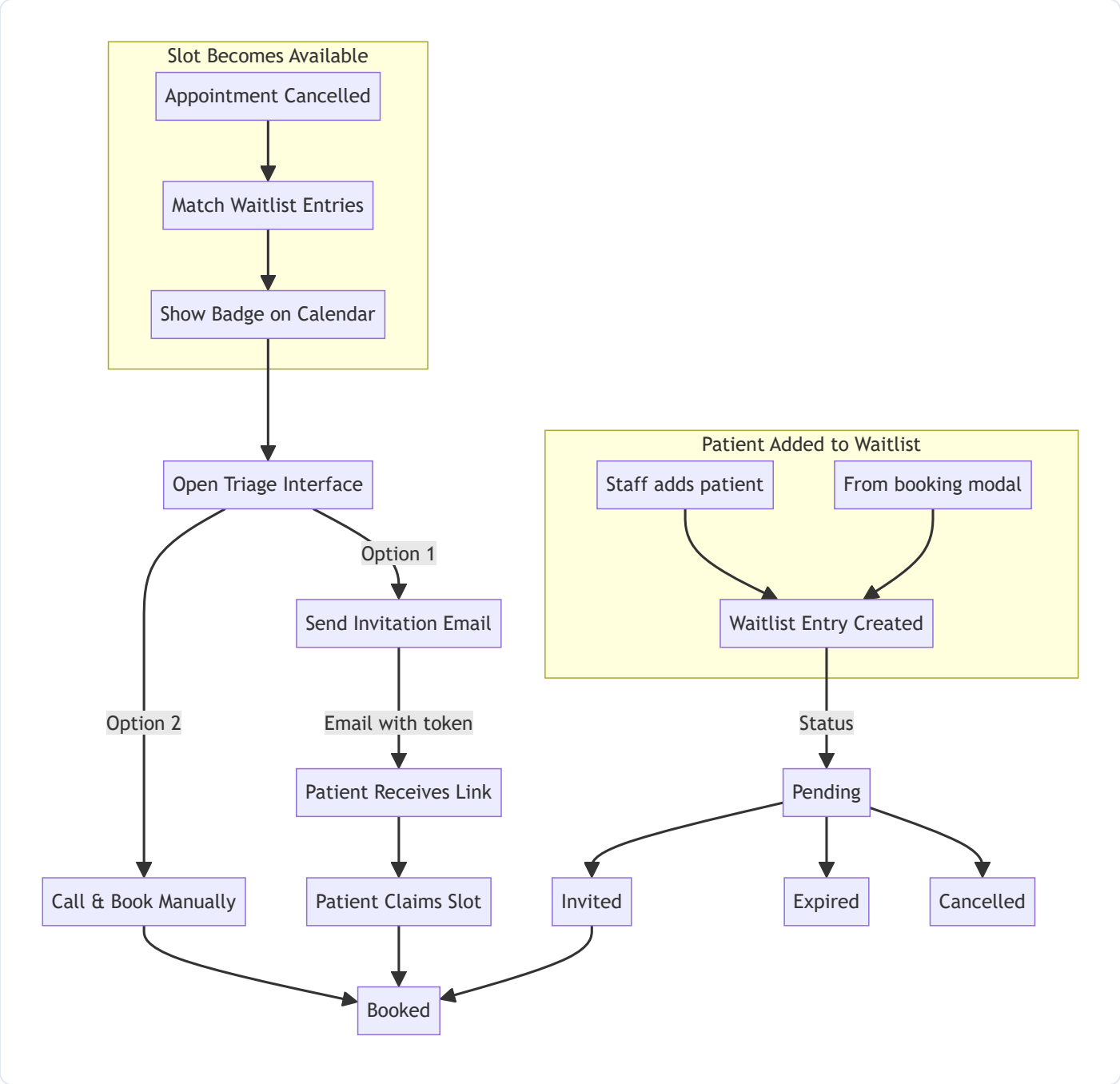
```
# Recurrence Patterns
POST   /sessions/recurrence-patterns                  # Create pattern + generate sessions
GET    /sessions/recurrence-patterns                  # List doctor's patterns
DELETE /sessions/recurrence-patterns/{group_id}       # Delete pattern + sessions

# Sessions
GET    /sessions/doctor/{doctor_id}                   # Get doctor's sessions
GET    /sessions/{session_id}                         # Get specific session
```

```
DELETE /sessions/{session_id}                    # Delete single session

# Slots
GET    /slots/doctor/{doctor_id}                 # Get doctor's available slots
GET    /slots/{slot_id}                          # Get slot details
POST   /slots                                    # Create custom slot
PATCH  /slots/{slot_id}/block                    # Block slot
PATCH  /slots/{slot_id}/unblock                  # Unblock slot
```

---

8. Waitlist System

Comprehensive waitlist management for handling appointment demand with intelligent matching and automated notifications.



8.1 Waitlist Entry Model

Each waitlist entry contains:

```
class WaitlistEntry:
    id: int
    patient_profile_id: int          # Patient being waitlisted
    doctor_user_id: int              # Doctor they want to see
    priority: WaitlistPriority       # NORMAL or HIGH
    preferred_days: List[str]        # ["Mon", "Wed", "Fri"] or ["Anytime"]
    notes: Optional[str]             # Staff notes
```

```
    status: WaitlistStatus          # pending, invited, booked, expired, cancelled
    expiry_date: date               # Auto-expire date (default: +30 days)
    created_by_user_id: int         # Staff who created entry
    updated_by_user_id: Optional[int]
    created_at: datetime
    updated_at: datetime
```

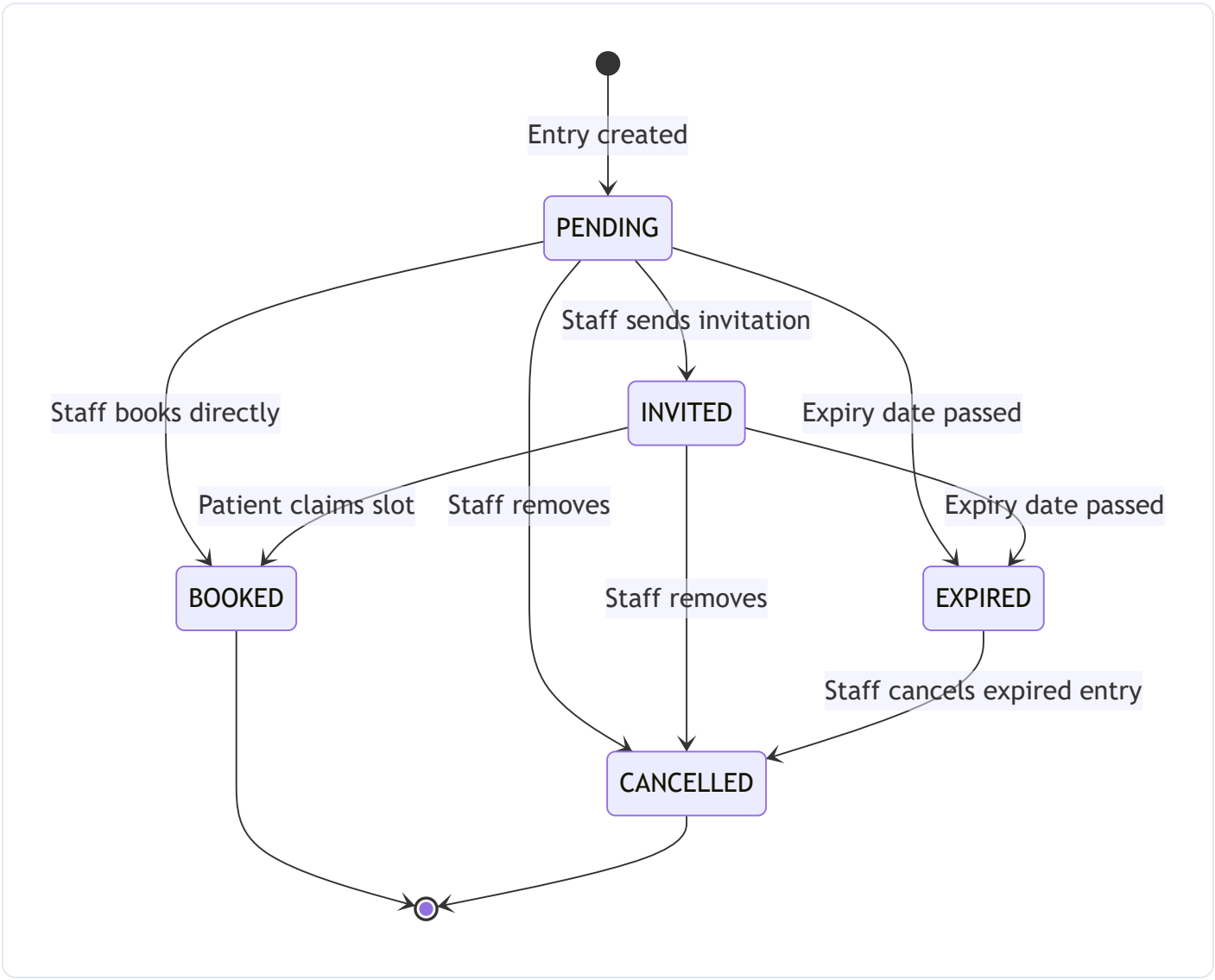
Priority Levels:

Priority	Description	Display
`NORMAL`	Standard priority	Regular badge
`HIGH`	Urgent need	Red/highlighted badge

Preferred Days Options:

```
# Specific days
preferred_days = ["Mon", "Wed", "Fri"]

# Any day
preferred_days = ["Anytime"]

# Weekdays only
preferred_days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
```

8.2 Waitlist Statuses & Transitions



Status Descriptions:

Status	Description	Terminal?
`pending`	Waiting for slot match	No
`invited`	Invitation email sent	No
`booked`	Appointment confirmed	Yes
`expired`	Entry expired (past expiry_date)	Yes*
`cancelled`	Manually removed	Yes

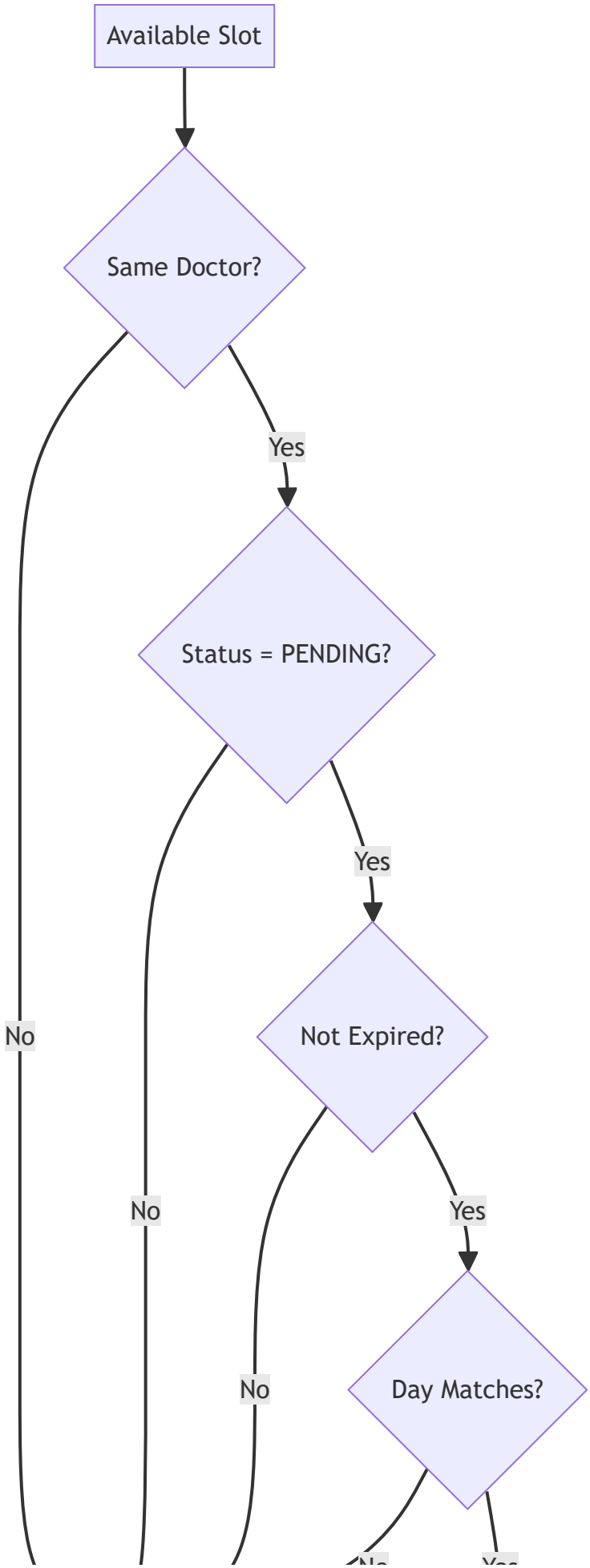
*Expired entries can be transitioned to cancelled.

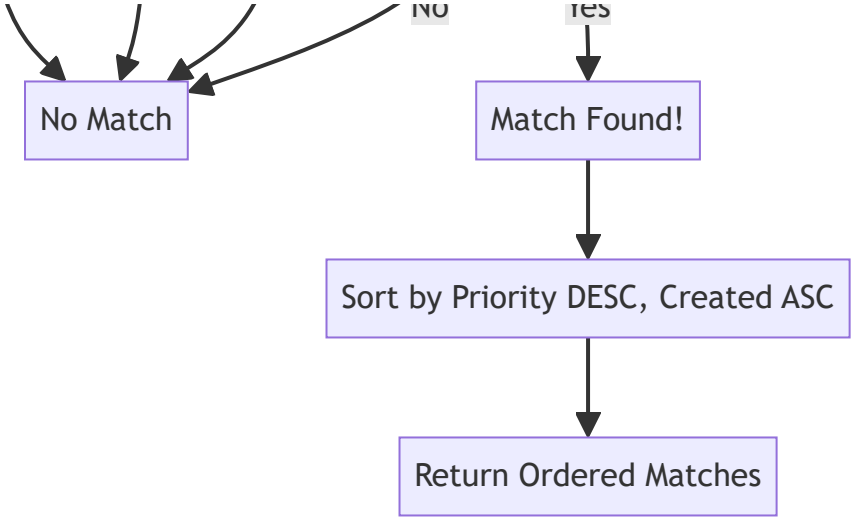
Valid Status Transitions:

```
valid_transitions = {
    WaitlistStatus.PENDING: [INVITED, BOOKED, CANCELLED, EXPIRED],
    WaitlistStatus.INVITED: [BOOKED, CANCELLED, EXPIRED],
    WaitlistStatus.BOOKED: [],           # Terminal state
    WaitlistStatus.CANCELLED: [],         # Terminal state
    WaitlistStatus.EXPIRED: [CANCELLED]   # Can cancel an expired entry
}
```

8.3 Matching Algorithm

The system automatically matches waitlist entries to available slots based on these criteria:


Available Slot
Same Doctor?
Yes
No
Status = PENDING?
Yes
No
Not Expired?
Yes
No
Day Matches?



Matching Criteria:

1. **Same Doctor** - Entry's `doctor_user_id` matches slot's session doctor
2. **Pending Status** - Entry status is `PENDING`
3. **Not Expired** - Entry's `expiry_date >= today`
4. **Day Preference Match** - One of:
   - Entry's `preferred_days` contains slot's day-of-week (e.g., "Mon")
   - Entry's `preferred_days` contains `"Anytime"`

Matching Code:

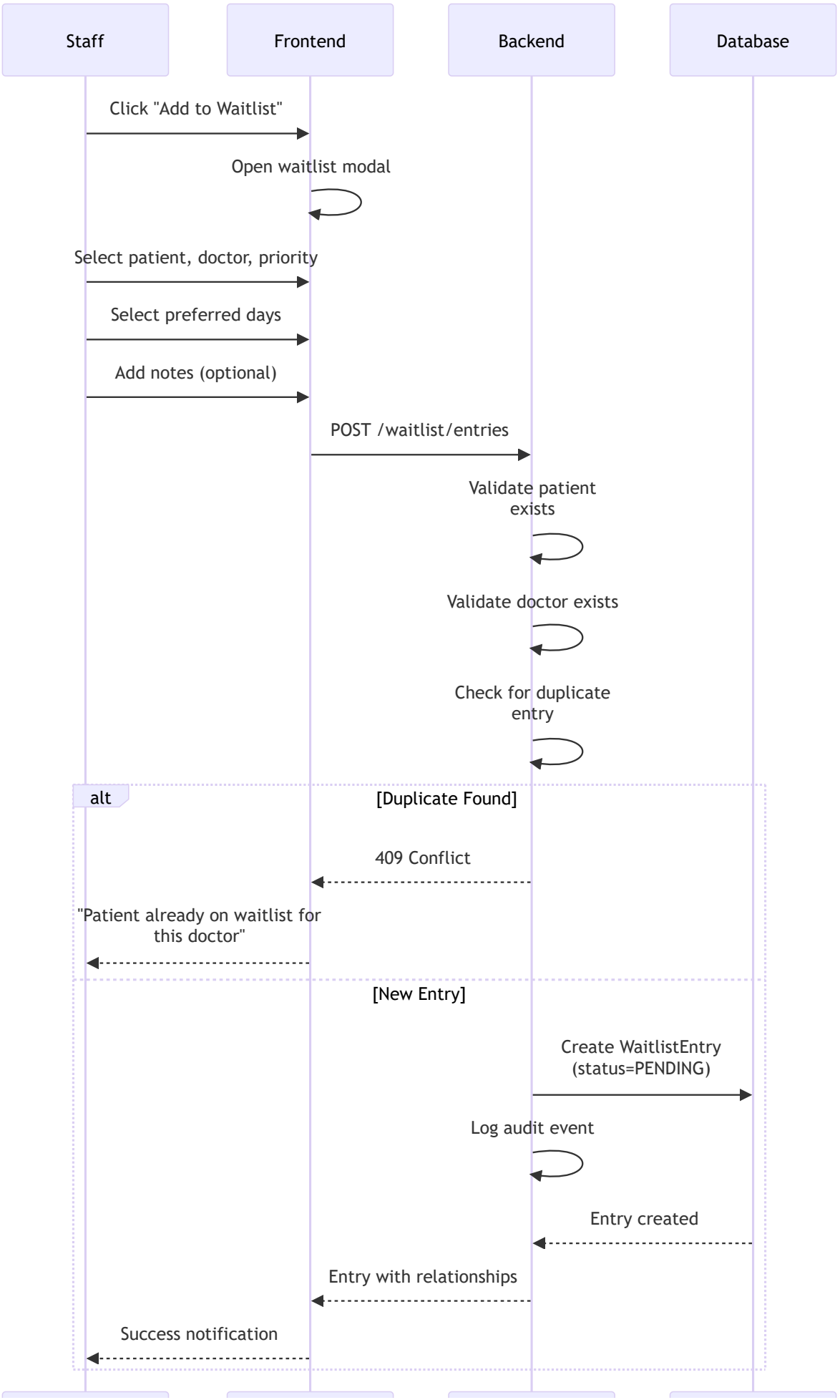
```
def find_matches_for_slot(slot: AppointmentSlot, db: Session) -> List[WaitlistEntry]:
    # Extract day-of-week from slot start_time
    slot_day = slot.start_time.strftime("%a")  # "Mon", "Tue", etc.

    # Get the doctor_user_id from the slot's session
    doctor_user_id = slot.session.doctor_user_id

    today = date.today()

    # Query entries matching criteria
    query = db.query(WaitlistEntry).filter(
        and_(
            WaitlistEntry.doctor_user_id == doctor_user_id,
            WaitlistEntry.status == WaitlistStatus.PENDING,
            WaitlistEntry.expiry_date >= today,
            or_(
                # Check if preferred_days JSON contains the slot's day
                cast(WaitlistEntry.preferred_days, Text).contains(f'"{slot_day}"'),
                # Or if it contains "Anytime"
                cast(WaitlistEntry.preferred_days, Text).contains('"Anytime"')
            )
```

```
        )
    )

    # Order by priority DESC (HIGH first), then created_at ASC (oldest first - FIFO)
    query = query.order_by(
        WaitlistEntry.priority.desc(),
        WaitlistEntry.created_at.asc()
    )

    return query.all()
```

Prioritization:

Matches are sorted by: 1. **Priority** (descending) - HIGH priority entries first 2. **Created At** (ascending) - Oldest entries first (FIFO within same priority)

8.4 Workflow: Staff Adding Patient to Waitlist


Staff
Frontend
Backend
Database
Click "Add to Waitlist"
Open waitlist modal
Select patient, doctor, priority
Select preferred days
Add notes (optional)
POST /waitlist/entries
Validate patient exists
Validate doctor exists
Check for duplicate entry
alt
[Duplicate Found]
409 Conflict
"Patient already on waitlist for this doctor"
[New Entry]
Create WaitlistEntry (status=PENDING)
Log audit event
Entry created
Entry with relationships
Success notification

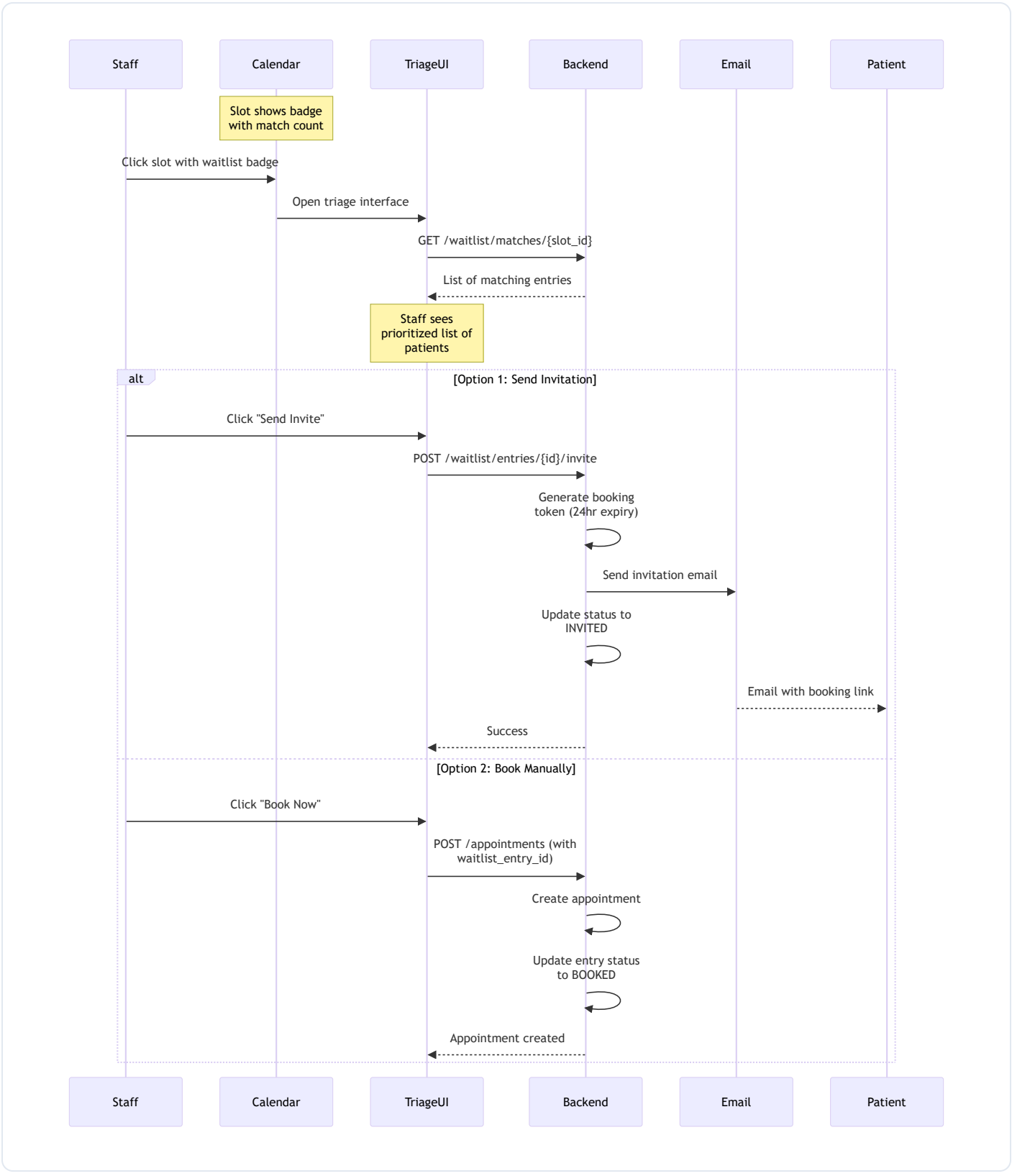
Staff | Frontend | Backend | Database

Create Entry Request:

```
{
    "patient_profile_id": 123,
    "doctor_user_id": 456,
    "priority": "HIGH",           # or "NORMAL"
    "preferred_days": ["Mon", "Wed", "Fri"],
    "notes": "Needs urgent follow-up",
    "expiry_date": "2025-02-28"   # Optional, defaults to +30 days
}
```

8.5 Workflow: Staff Triage Interface

When a slot becomes available (e.g., appointment cancelled), staff can manage waitlist matches:



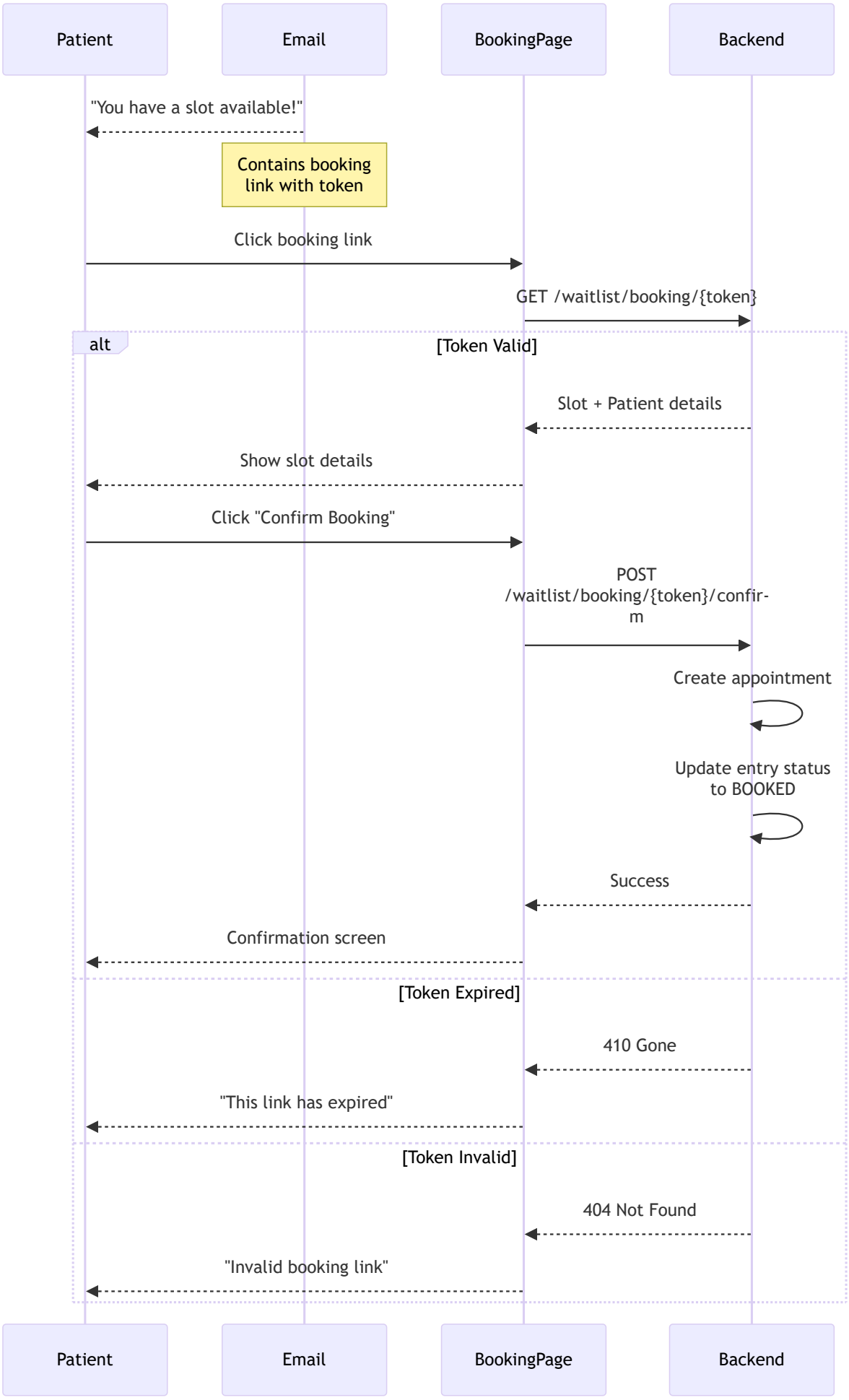
Triage Response Data:

```
{
    "slot_id": 789,
    "slot_start_time": "2025-02-15T09:00:00",
    "slot_end_time": "2025-02-15T09:30:00",
```

```
    "doctor_name": "Dr. John Smith",
    "doctor_user_id": 456,
    "match_count": 3,
    "matches": [
        {
            "entry_id": 101,
            "patient_name": "Jane Doe",
            "patient_phone": null,
            "priority": "HIGH",
            "notes": "Urgent follow-up needed",
            "created_at": "2025-02-01T10:30:00",
            "days_waiting": 14,
            "preferred_days": ["Mon", "Wed", "Fri"]
        },
        {
            "entry_id": 102,
            "patient_name": "Bob Wilson",
            "priority": "NORMAL",
            "days_waiting": 7,
            "preferred_days": ["Anytime"]
        }
    ]
}
```

8.6 Workflow: Patient Self-Service Booking

When a patient receives an invitation email:


Patient
Email
BookingPage
Backend
"You have a slot available!"
Contains booking link with token
Click booking link
GET /waitlist/booking/{token}
alt
[Token Valid]
Slot + Patient details
Show slot details
Click "Confirm Booking"
POST /waitlist/booking/{token}/confir-m
Create appointment
Update entry status to BOOKED
Success
Confirmation screen
[Token Expired]
410 Gone
"This link has expired"
[Token Invalid]
404 Not Found
"Invalid booking link"
Patient
Email
BookingPage
Backend

Booking Token:

- Generated when staff sends invitation
- Valid for 24 hours
- Single-use (consumed on booking)
- Contains: `waitlist_entry_id`, `slot_id`, `expiry_timestamp`

8.7 Automatic Expiry

A background job runs to expire old waitlist entries:

```
def expire_old_entries(db: Session) -> int:
    """
    Queries entries where expiry_date < today AND status = PENDING,
    then updates their status to EXPIRED.
    """
    today = date.today()

    expired_entries = db.query(WaitlistEntry).filter(
        and_(
            WaitlistEntry.expiry_date < today,
            WaitlistEntry.status == WaitlistStatus.PENDING
        )
    ).all()

    for entry in expired_entries:
        entry.status = WaitlistStatus.EXPIRED
        entry.updated_at = datetime.utcnow()

        # Log audit event (system action)
        log_waitlist_audit_event(
            action="expire_entry",
            user_id=None,  # System action
            entry_id=entry.id,
            ...
        )

    db.commit()
    return len(expired_entries)
```

8.8 Calendar Badge Display

Available slots show waitlist match information:

```
def get_slot_waitlist_info(slot: AppointmentSlot, db: Session) -> Dict:
    """Returns waitlist information for calendar display."""

    slot_day = slot.start_time.strftime("%a")
    doctor_user_id = slot.session.doctor_user_id
    today = date.today()

    # Count total matches
    total_count = db.query(func.count(WaitlistEntry.id)).filter(
        # ... matching criteria ...
    ).scalar() or 0

    # Count high priority matches
    high_priority_count = db.query(func.count(WaitlistEntry.id)).filter(
        # ... matching criteria + priority = HIGH ...
    ).scalar() or 0

    return {
        "waitlist_match_count": total_count,
        "has_high_priority_matches": high_priority_count > 0
    }
```

Badge Appearance:

Condition	Badge Display
No matches	No badge
Normal matches only	Blue badge with count
Has HIGH priority	Red/highlighted badge

8.9 Key Endpoints

```
# Waitlist Entries CRUD
POST   /waitlist/entries                  # Create entry
GET    /waitlist/entries                  # List entries (filter by doctor/status)
GET    /waitlist/entries/{id}             # Get specific entry
PATCH  /waitlist/entries/{id}             # Update entry (priority, days, notes)
DELETE /waitlist/entries/{id}             # Cancel entry

# Waitlist Operations
```

```
POST   /waitlist/entries/{id}/invite        # Send invitation email
POST   /waitlist/entries/{id}/book          # Book directly for patient

# Slot Matching
GET    /waitlist/matches/{slot_id}          # Get matches for a slot
GET    /waitlist/slots/{doctor_id}/summary  # Doctor's waitlist summary

# Patient Self-Service
GET    /waitlist/booking/{token}            # Validate booking token
POST   /waitlist/booking/{token}/confirm    # Confirm booking

# Admin
POST   /waitlist/expire-old                 # Trigger expiry job
```

8.10 Audit Trail

All waitlist actions are logged for compliance:

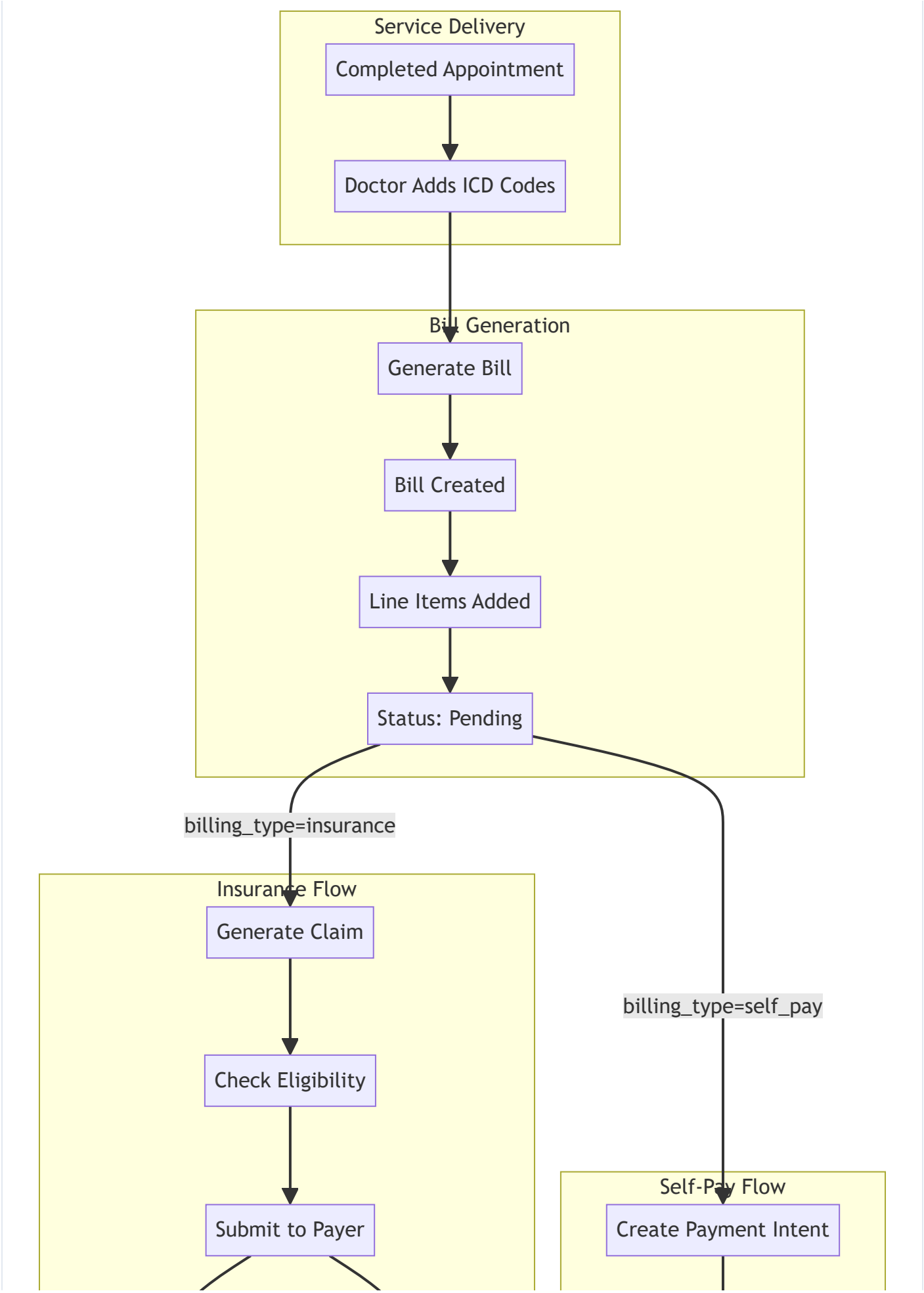
```
log_waitlist_audit_event(
    action="create_entry",      # or update_entry, delete_entry, update_status, expire_entry
    user_id=current_user.id,    # Who performed the action
    entry_id=entry.id,
    patient_profile_id=patient_id,
    doctor_user_id=doctor_id,
    old_values={...},           # Previous state
    new_values={...},           # New state
    additional_data={...}       # Extra context
)
```

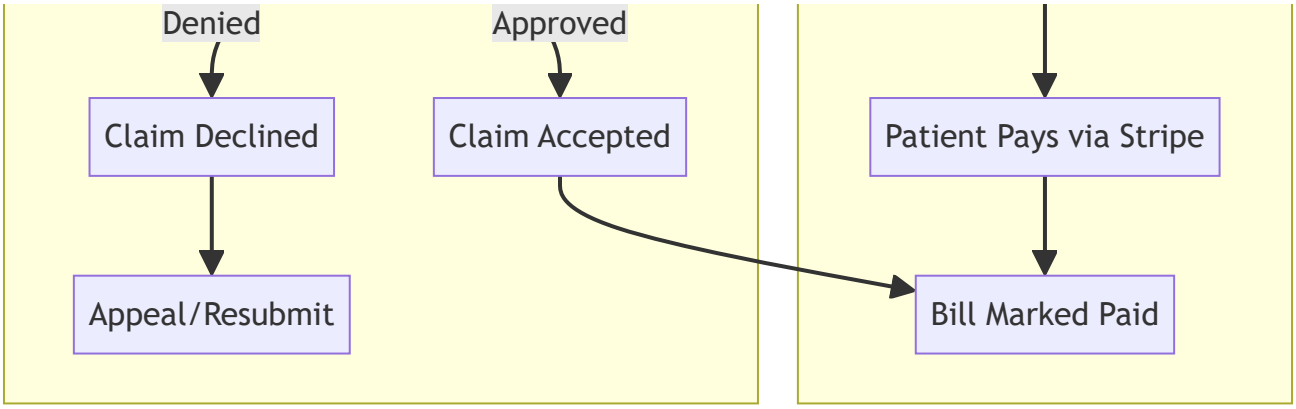
---

9. Billing & Revenue Cycle Management (RCM)

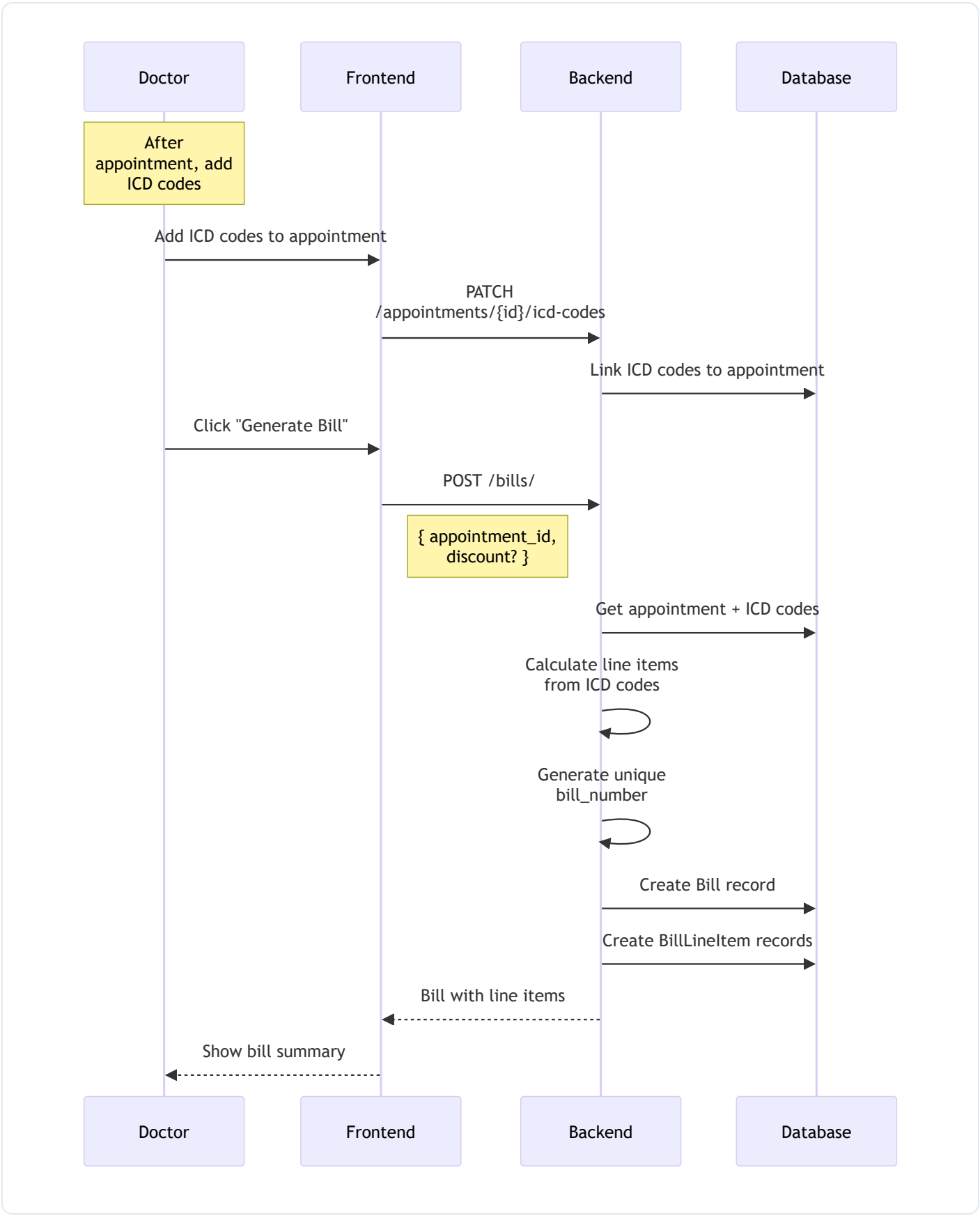
Complete billing workflow from service to payment.

9.1 Complete Billing Lifecycle

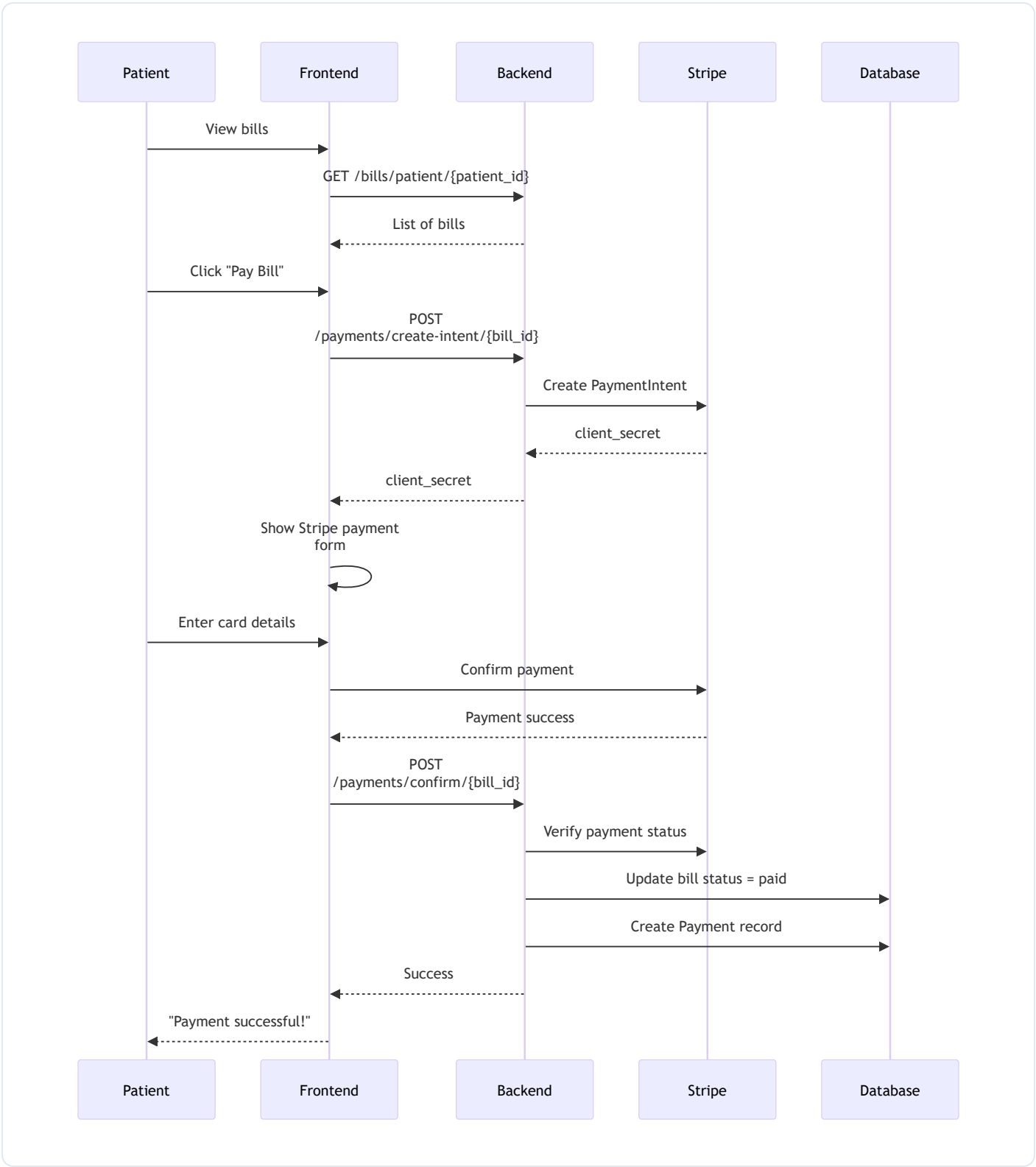

Service Delivery
Completed Appointment
Doctor Adds ICD Codes
Bill Generation
Generate Bill
Bill Created
Line Items Added
Status: Pending
billing_type=insurance
billing_type=self_pay
Insurance Flow
Generate Claim
Check Eligibility
Submit to Payer
Self-Pay Flow
Create Payment Intent



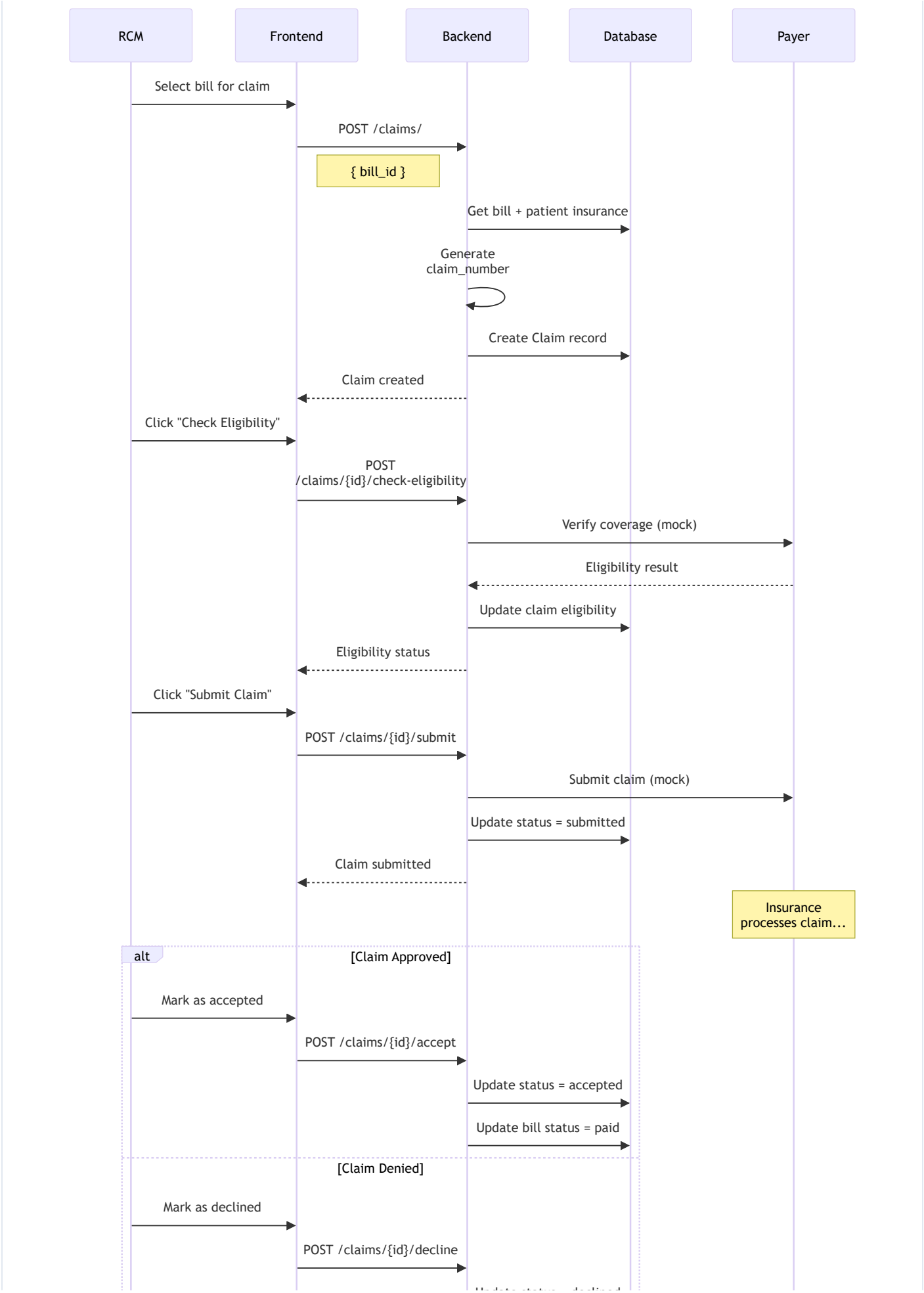
9.2 Bill Generation Flow

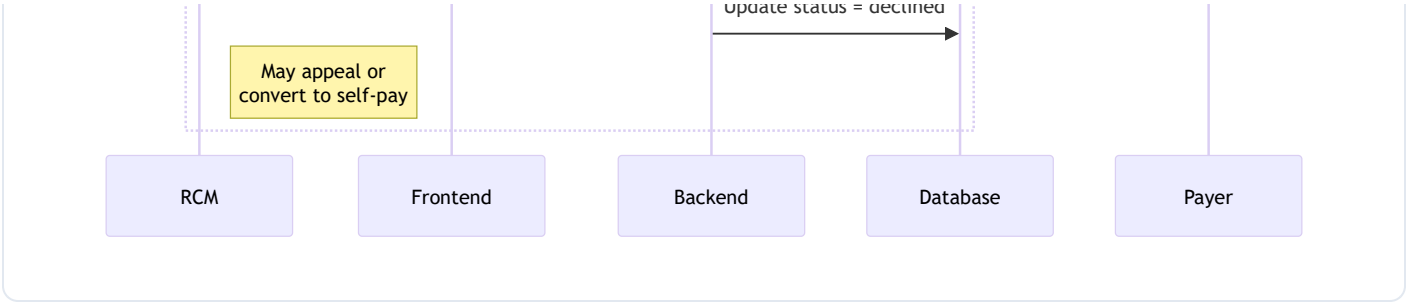


9.3 Self-Pay Payment Flow

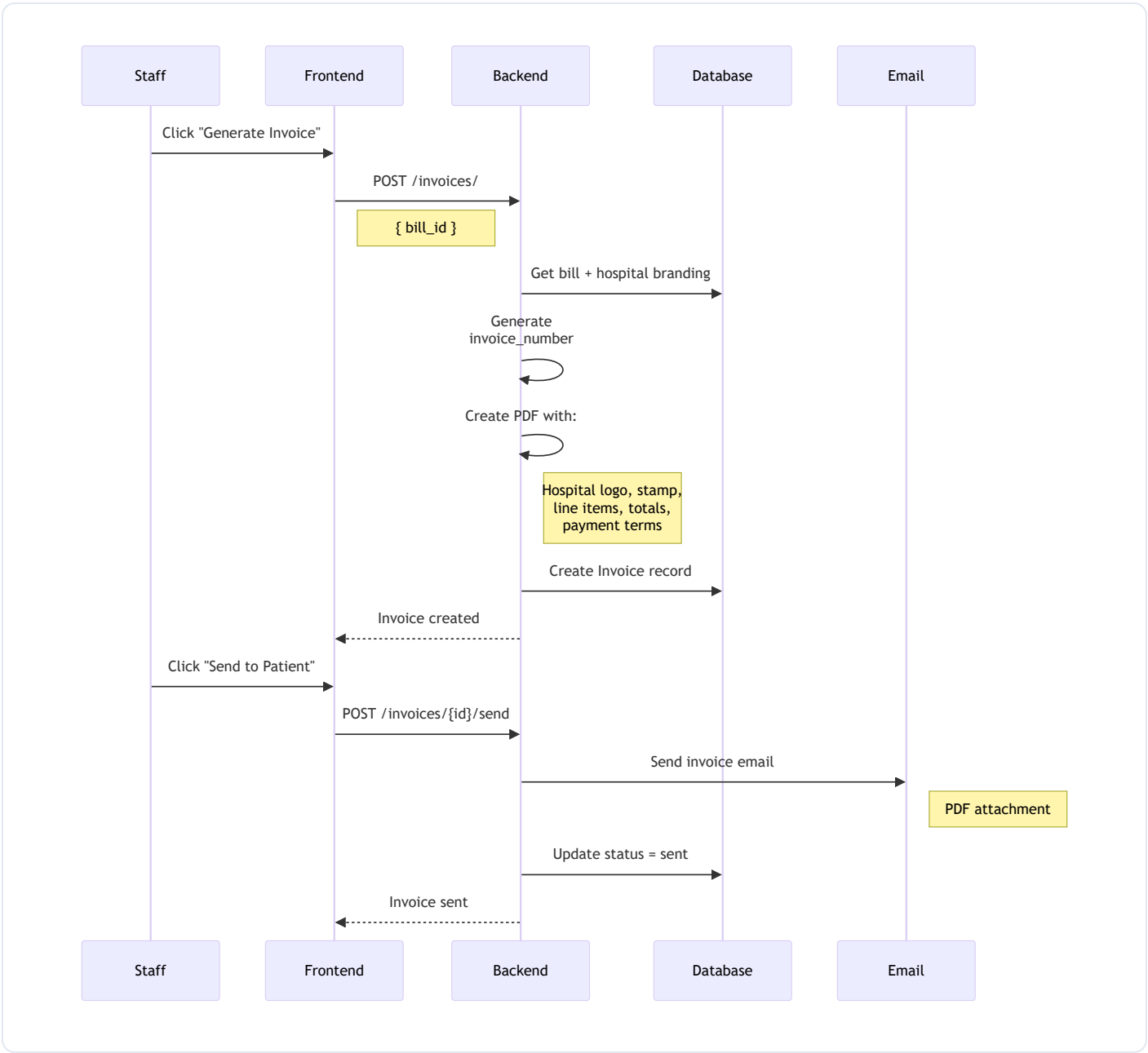


9.4 Insurance Claim Flow

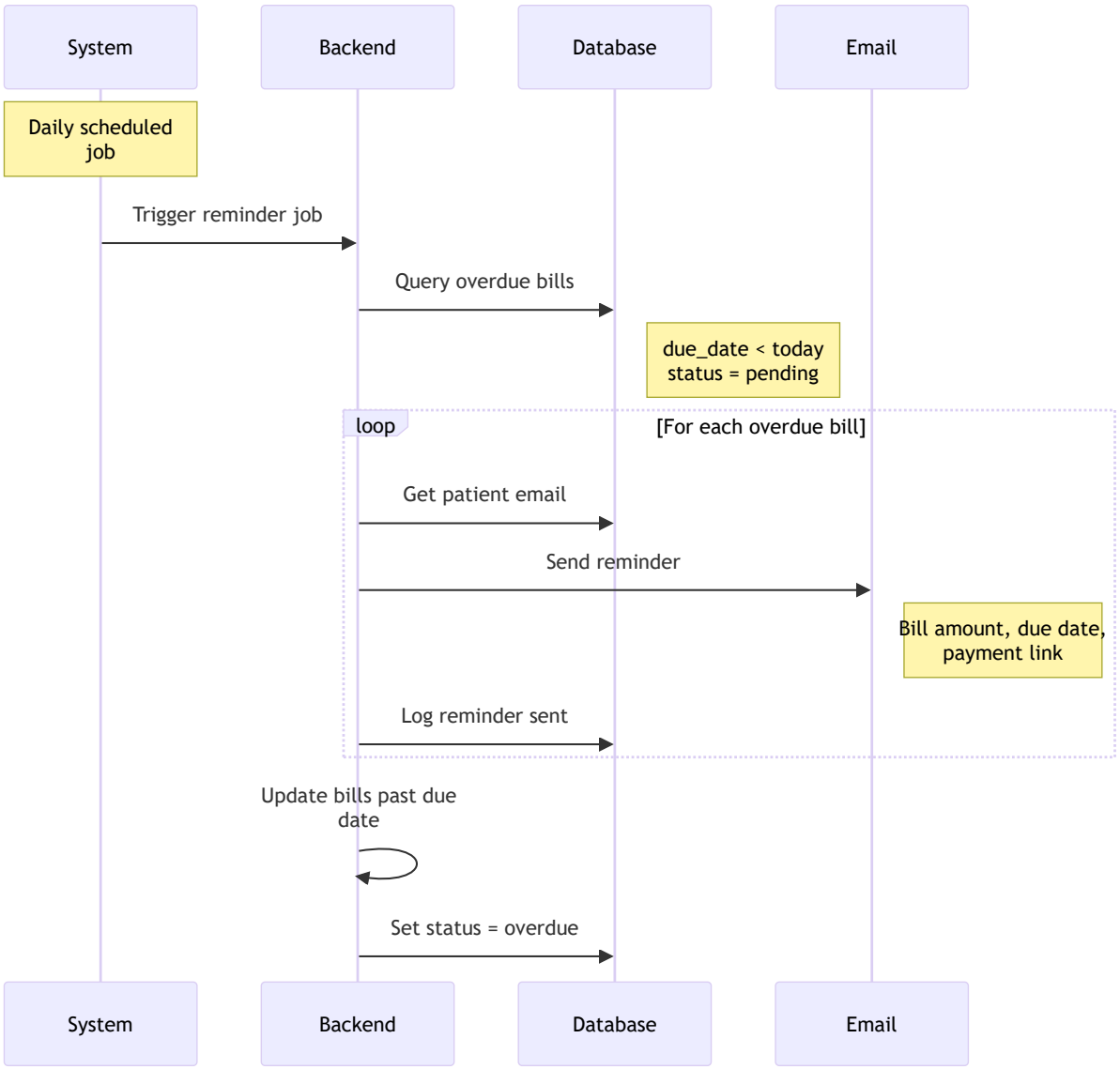

RCM
Frontend
Backend
Database
Payer
Select bill for claim
POST /claims/
{ bill_id }
Get bill + patient insurance
Generate claim_number
Create Claim record
Claim created
Click "Check Eligibility"
POST /claims/{id}/check-eligibility
Verify coverage (mock)
Eligibility result
Update claim eligibility
Eligibility status
Click "Submit Claim"
POST /claims/{id}/submit
Submit claim (mock)
Update status = submitted
Claim submitted
Insurance processes claim...
alt
[Claim Approved]
Mark as accepted
POST /claims/{id}/accept
Update status = accepted
Update bill status = paid
[Claim Denied]
Mark as declined
POST /claims/{id}/decline



9.5 Invoice Generation Flow



9.6 Bill Reminder Flow



Billing Features:

Bill Generation:

- **Auto-Generation** - From completed appointments with ICD codes
- **Bill Number** - Unique identifier (e.g., BILL-20250129-A1B2)
- **Line Items** - Based on ICD codes (mock pricing)
- **Discounts** - Configurable discount percentage
- **Due Date** - Auto-calculated or manual

Bill Statuses:

Status	Description
`pending`	Awaiting payment
`paid`	Full payment received
`partially_paid`	Partial payment made
`overdue`	Past due date
`cancelled`	Bill cancelled

Payment Processing:

- **Stripe Integration** - Secure payment processing
- **Payment Intent** - Create payment intent for frontend
- **Payment Confirmation** - Verify and update bill status
- **Billing Reminders** - Email reminders for outstanding bills

Claims Management:

- **Claim Generation** - Create insurance claims from bills
- **Eligibility Check** - Verify patient coverage (mock)
- **Payer Submission** - Submit to insurance (mock)
- **Accept/Decline** - Process payer responses

Key Endpoints:

```
# Bills
POST   /bills/                      # Create bill from appointment
GET    /bills/                      # List bills (filtered)
GET    /bills/{id}                  # Get bill details
GET    /bills/patient/{patient_id}  # Patient's bills

# Payments
POST   /payments/create-intent/{id} # Create Stripe intent
POST   /payments/confirm/{id}       # Confirm payment

# Claims
POST   /claims/                     # Create claim
POST   /claims/{id}/check-eligibility # Check insurance
POST   /claims/{id}/submit          # Submit to payer
POST   /claims/{id}/accept          # Mark accepted
POST   /claims/{id}/decline         # Mark declined
```

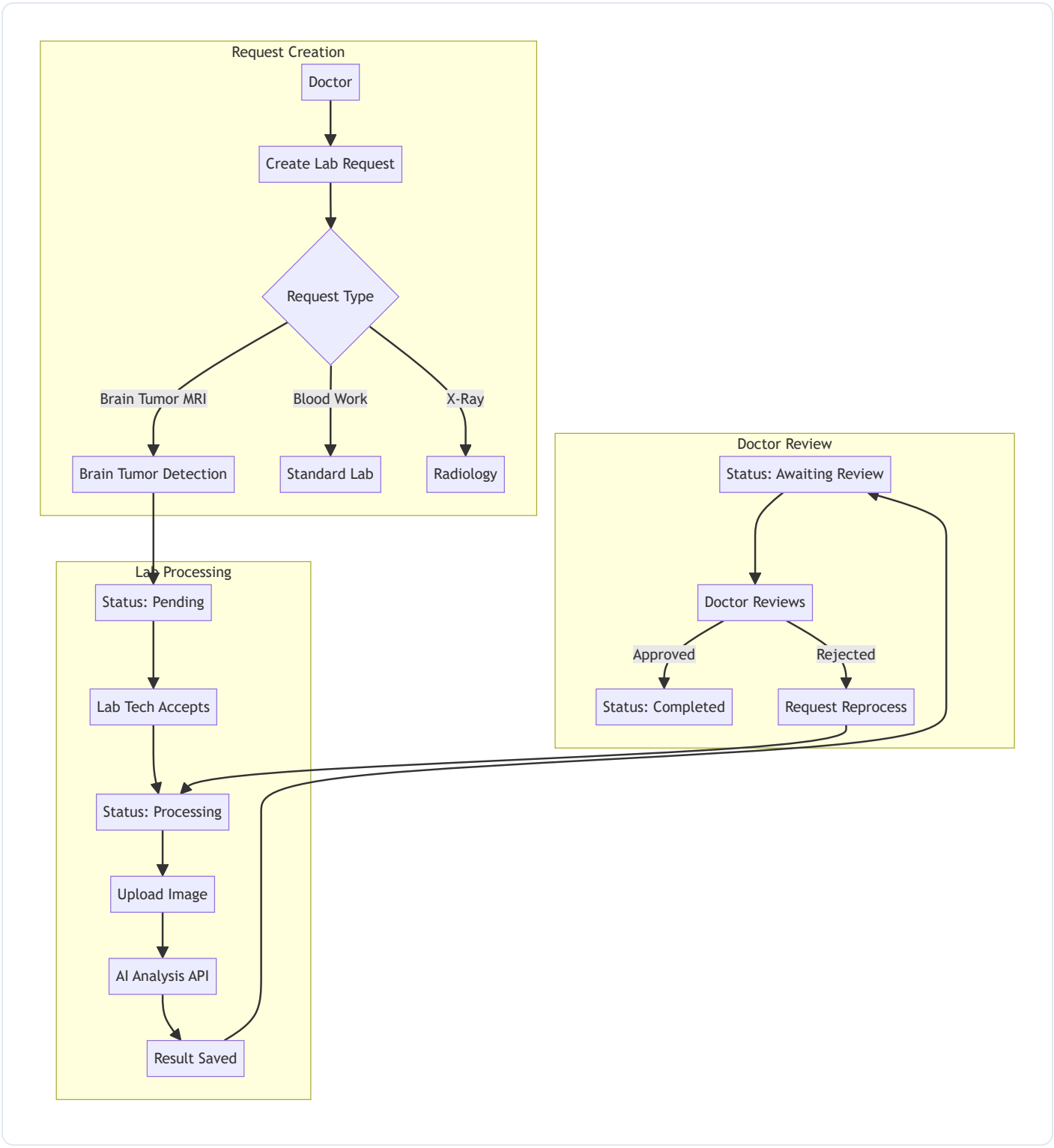
```
# Invoices
POST   /invoices/                    # Generate invoice
GET    /invoices/{id}/pdf            # Download PDF
POST   /invoices/{id}/send           # Email to patient
```

---

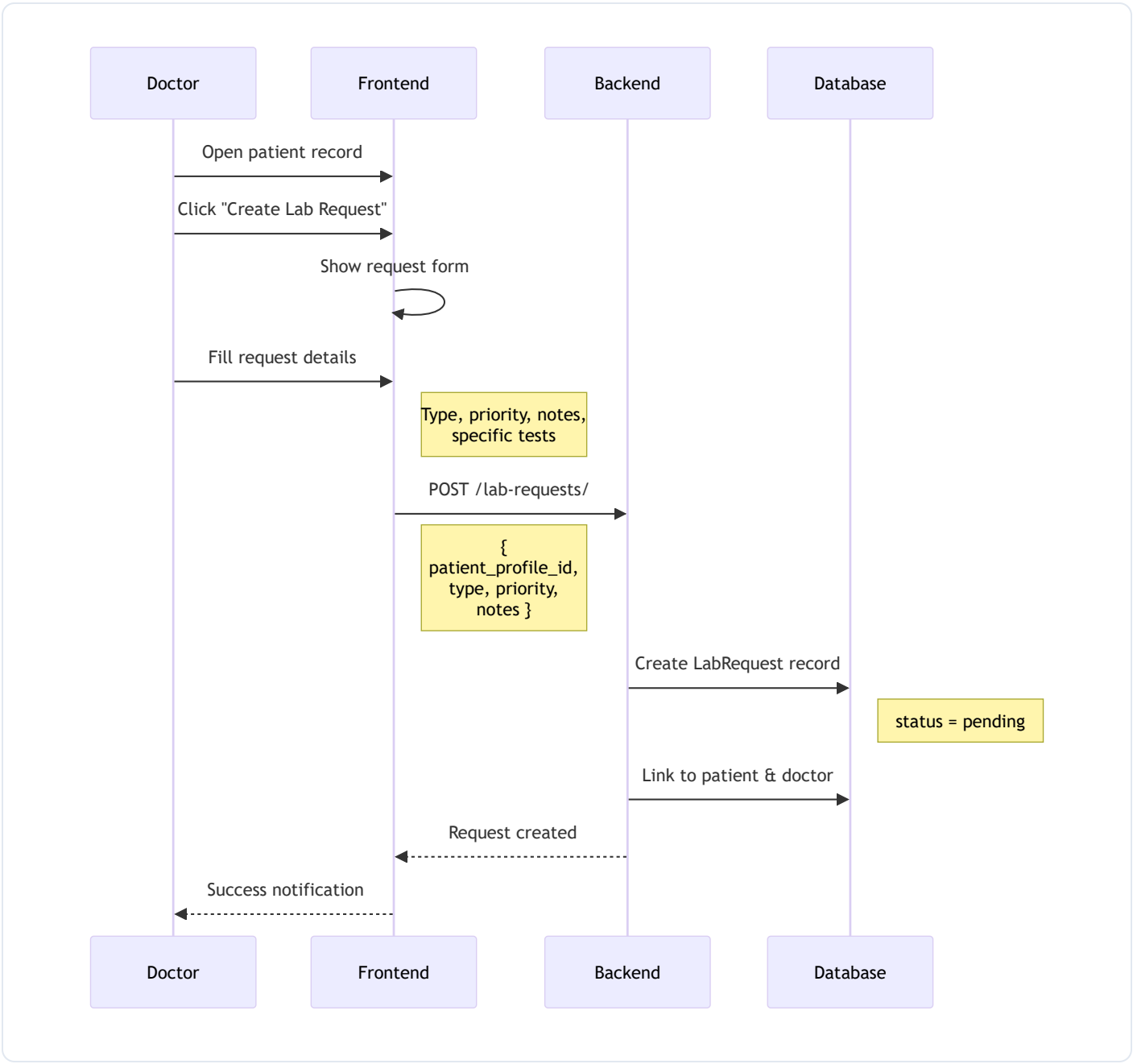
10. Lab Requests & AI Integration

Laboratory workflow with AI-powered image analysis.

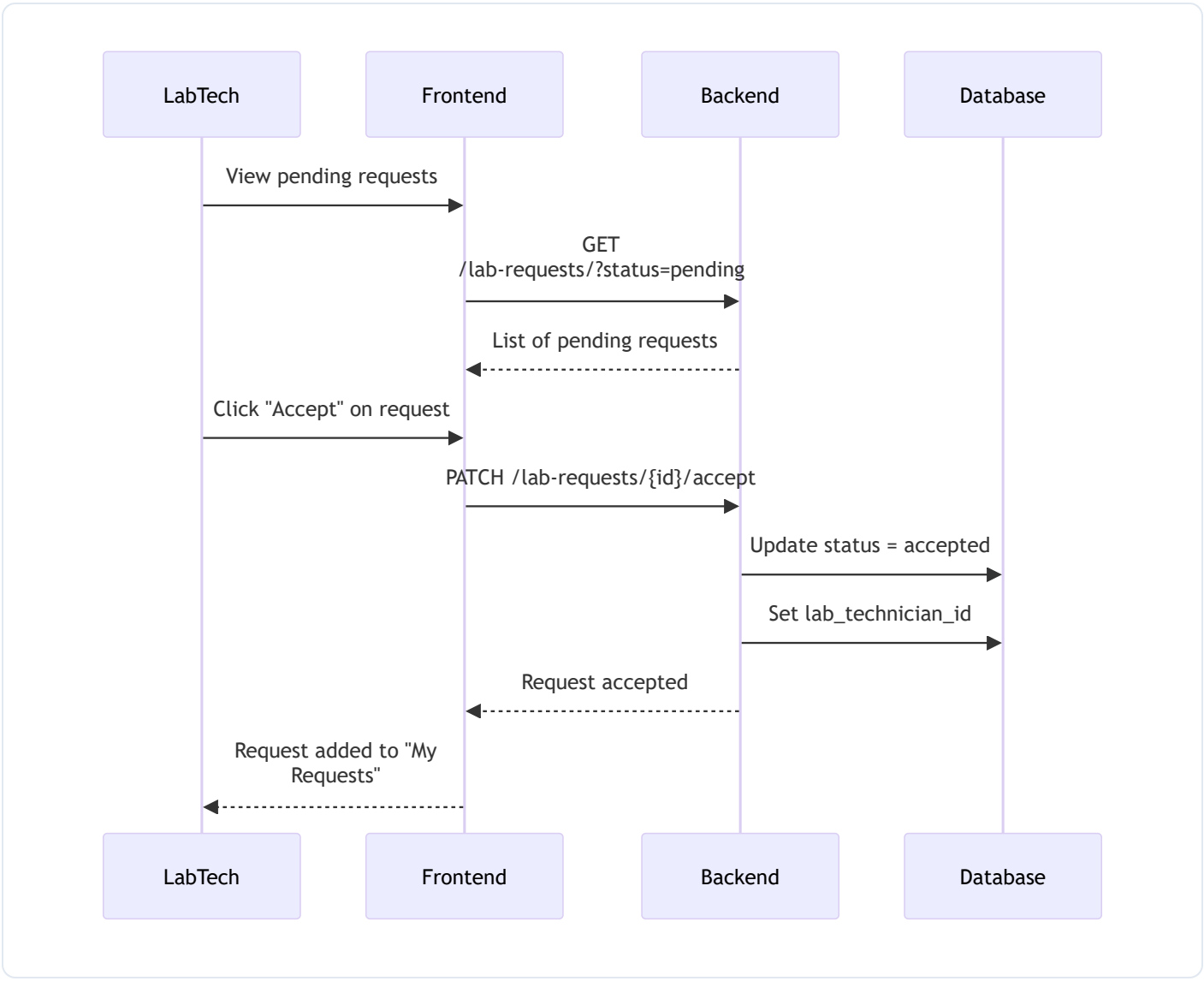
10.1 Complete Lab Request Lifecycle



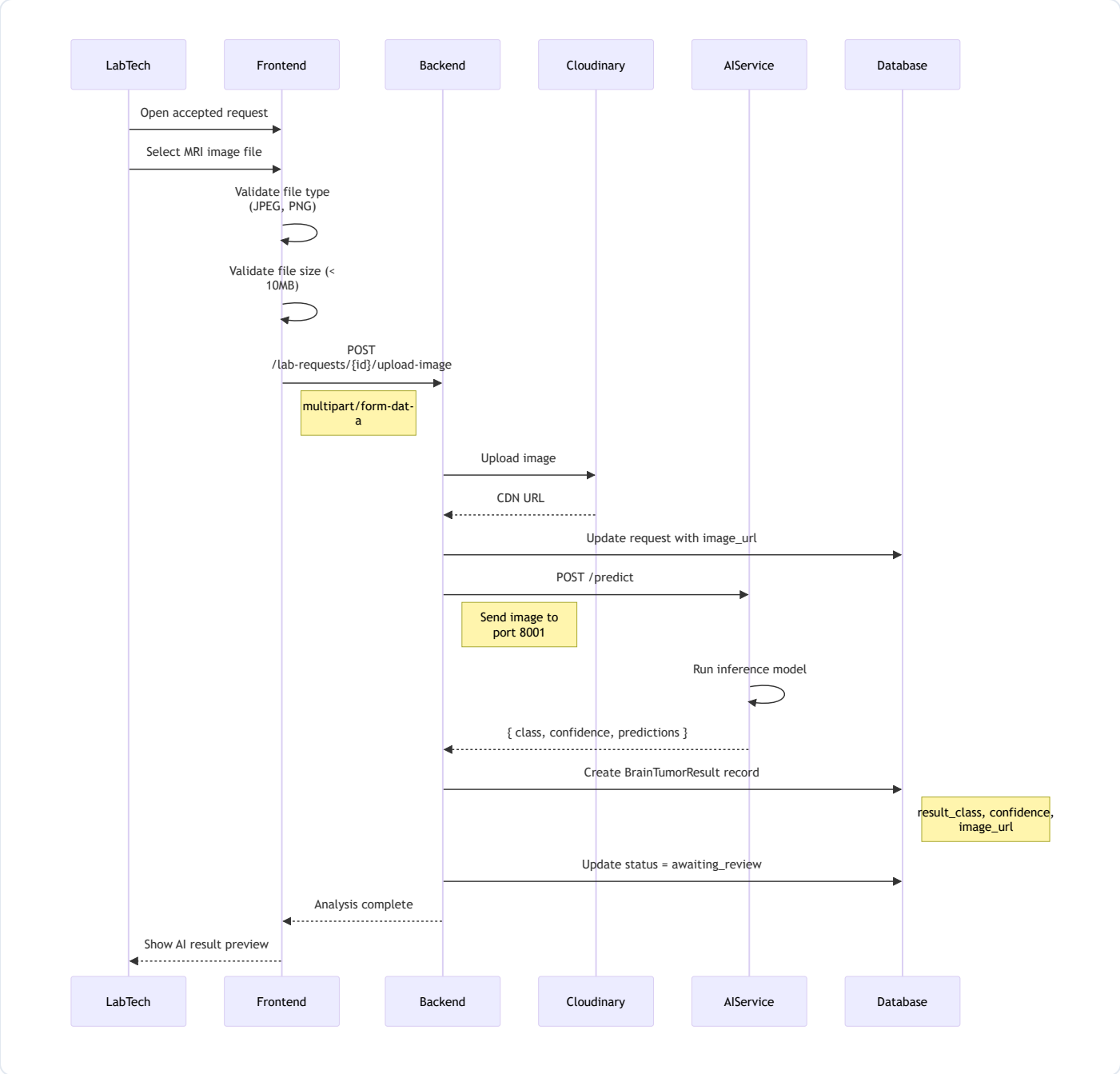
10.2 Doctor Creates Lab Request Flow



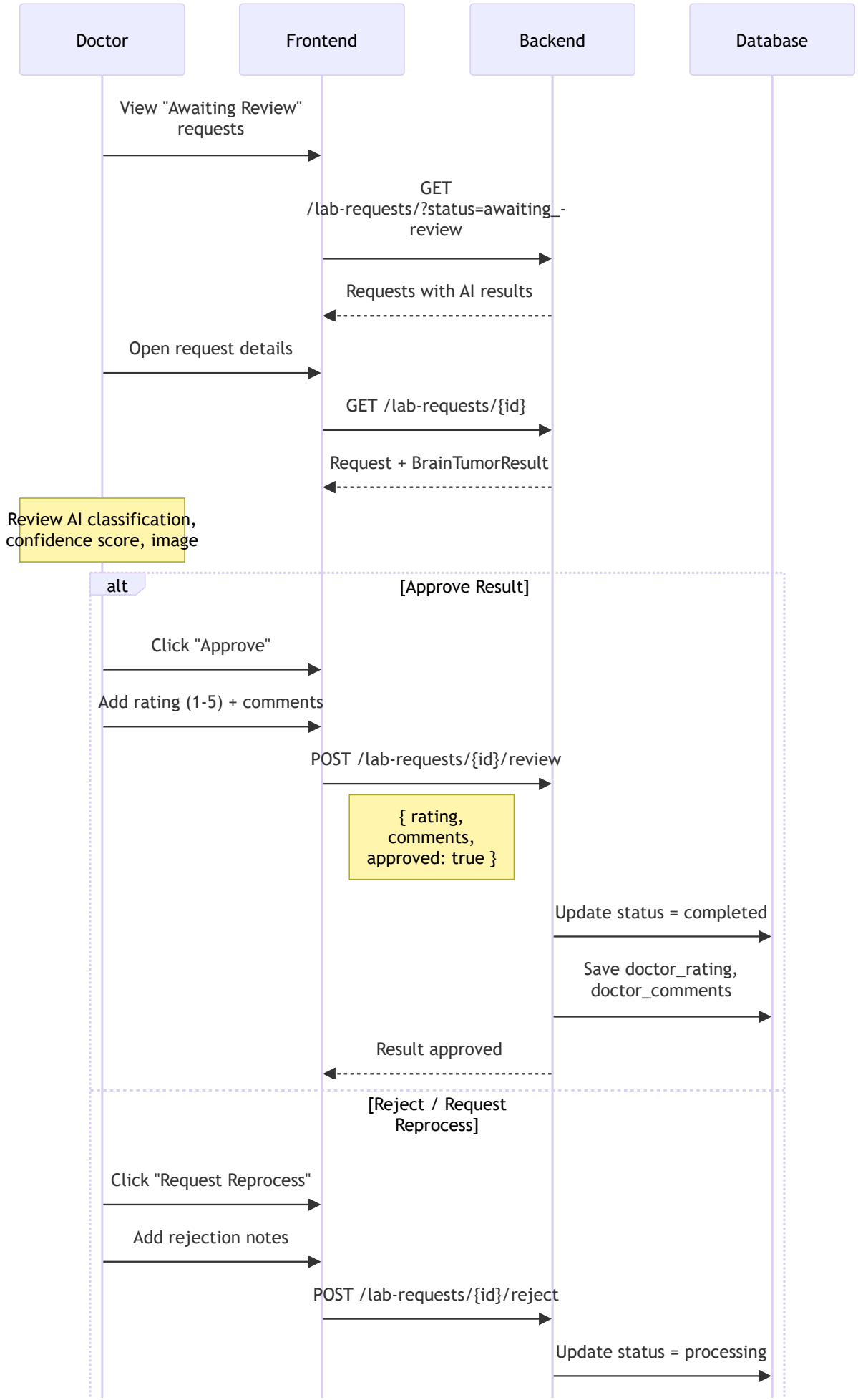
10.3 Lab Technician Accepts Request Flow

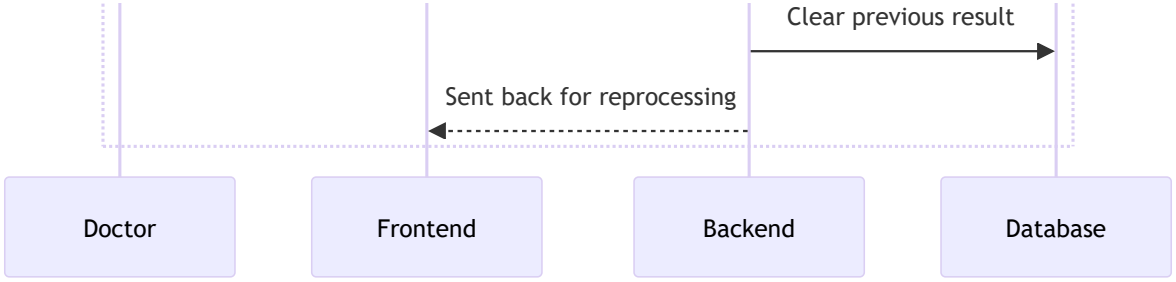


10.4 Brain Tumor AI Analysis Flow



10.5 Doctor Reviews AI Result Flow


Doctor
Frontend
Backend
Database
View "Awaiting Review" requests
GET /lab-requests/?status=awaiting_review
Requests with AI results
Open request details
GET /lab-requests/{id}
Request + BrainTumorResult
Review AI classification, confidence score, image
alt
[Approve Result]
Click "Approve"
Add rating (1-5) + comments
POST /lab-requests/{id}/review
{ rating, comments, approved: true }
Update status = completed
Save doctor_rating, doctor_comments
Result approved
[Reject / Request Reprocess]
Click "Request Reprocess"
Add rejection notes
POST /lab-requests/{id}/reject
Update status = processing



Lab Request Features:

Request Types:

- `brain_tumor_detection` - MRI image analysis
- `blood_work` - Standard laboratory tests
- `x_ray` - Radiological imaging
- `urinalysis` - Urine tests
- `other` - Custom tests

Request Statuses:

Status	Description
`pending`	Awaiting lab tech acceptance
`accepted`	Lab tech assigned
`rejected`	Lab tech declined
`processing`	Analysis in progress
`awaiting_review`	Results ready for doctor
`completed`	Doctor approved

AI Integration (Brain Tumor Detection):

1. **Image Upload** - Lab tech uploads MRI scan
2. **Cloudinary Storage** - Image stored in CDN
3. **AI API Call** - Send to inference service (port 8001)
4. **Result Storage** - Class, confidence, image URL
5. **Doctor Review** - Rating, comments, approval

Brain Tumor Result Model:

```
class BrainTumorResult:
    id: int
    request_id: int
    image_url: str         # Cloudinary URL
    result_class: str      # e.g., "glioma", "meningioma", "no_tumor"
    confidence: float      # 0.0 - 1.0
    doctor_rating: int     # 1-5 stars
    doctor_comments: str
    created_at: datetime
```

Key Endpoints:

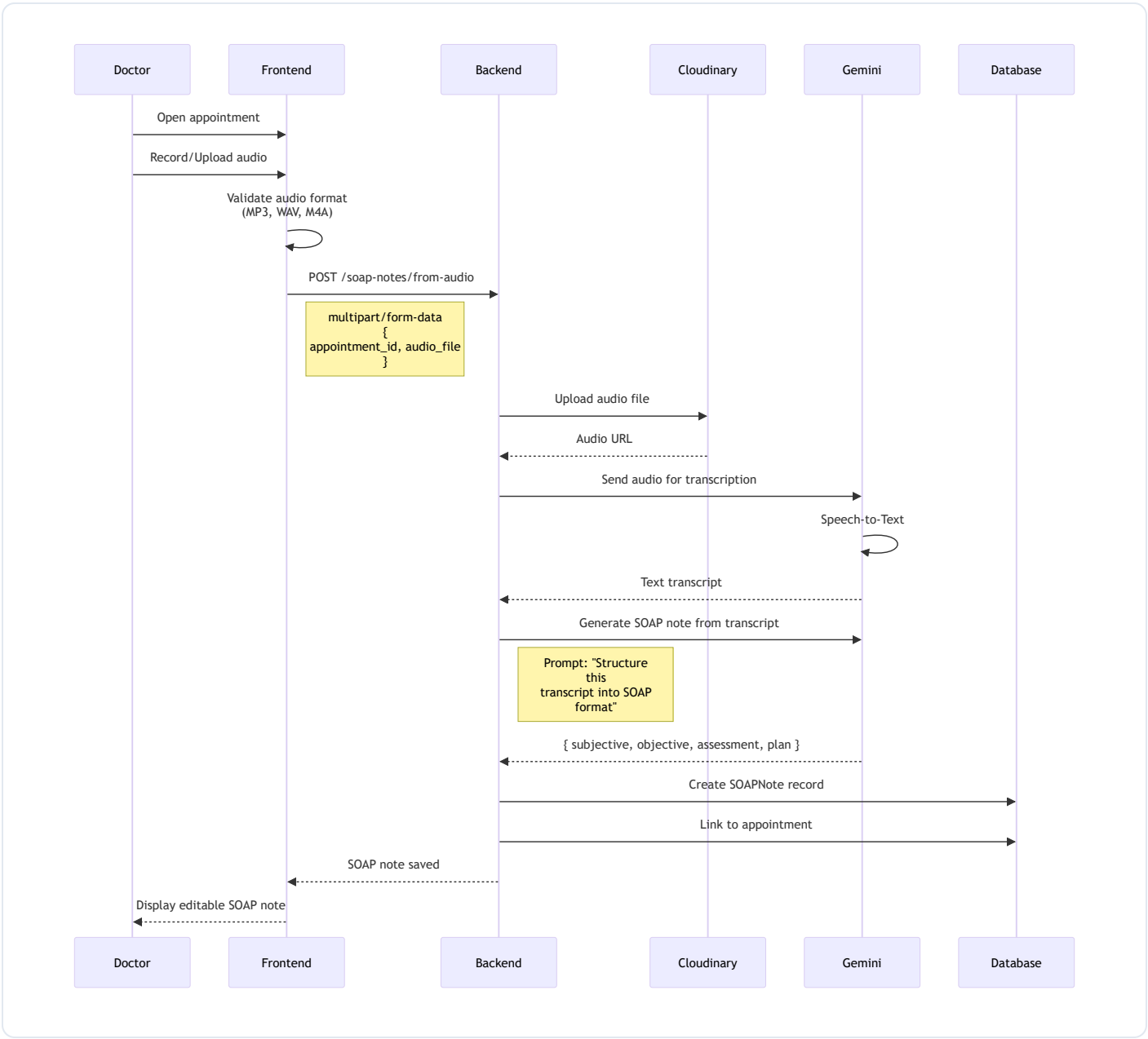
```
# Lab Requests
POST   /lab-requests/                    # Create request
GET    /lab-requests/                    # List requests (filtered)
GET    /lab-requests/{id}                # Get request details
PATCH  /lab-requests/{id}/accept         # Lab tech accepts
PATCH  /lab-requests/{id}/reject         # Lab tech rejects
POST   /lab-requests/{id}/upload-image   # Upload for AI analysis
POST   /lab-requests/{id}/review         # Doctor approves
POST   /lab-requests/{id}/request-reprocess # Doctor requests redo
```

---

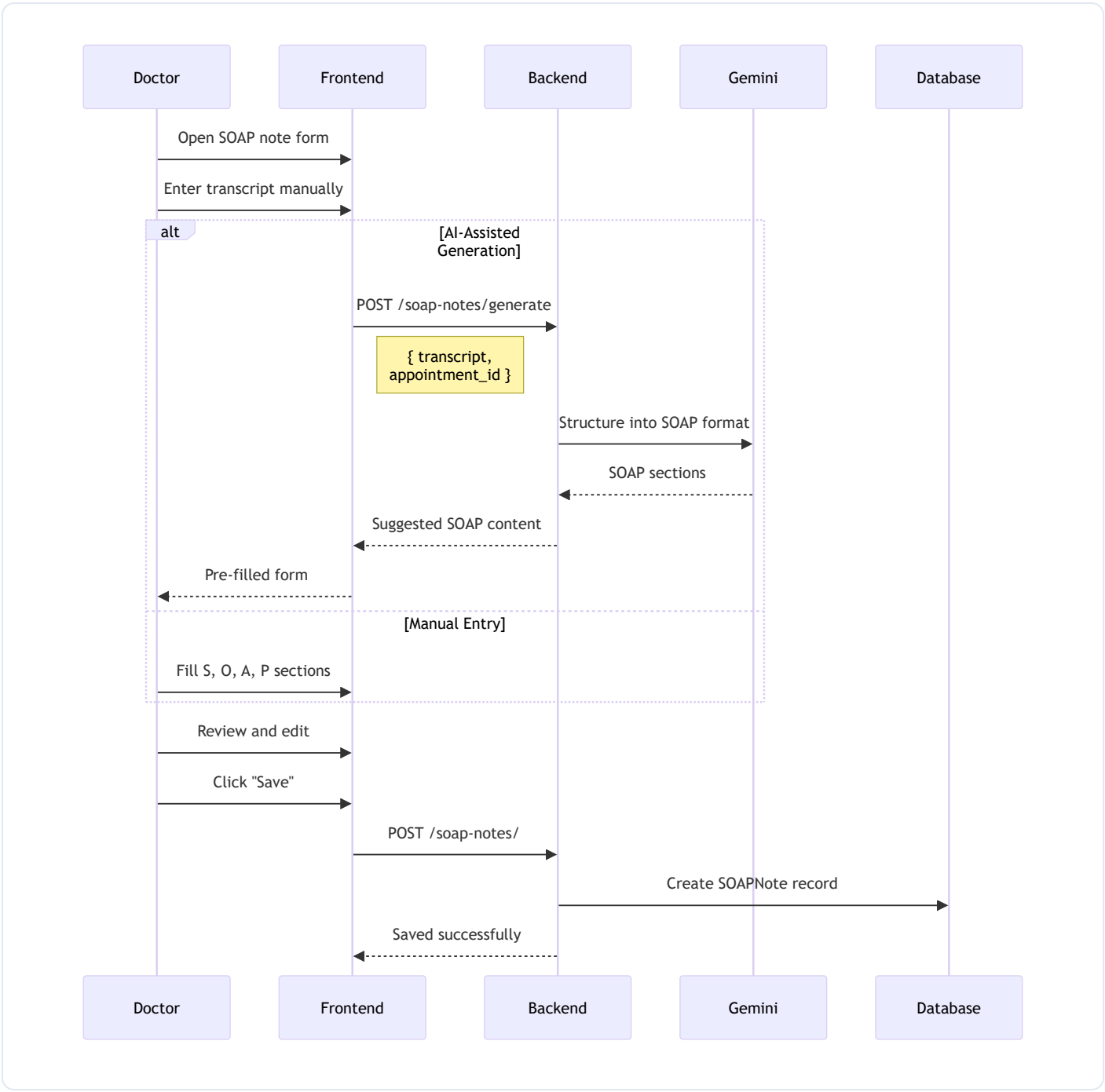
11. Clinical Documentation

AI-powered clinical documentation for comprehensive patient records.

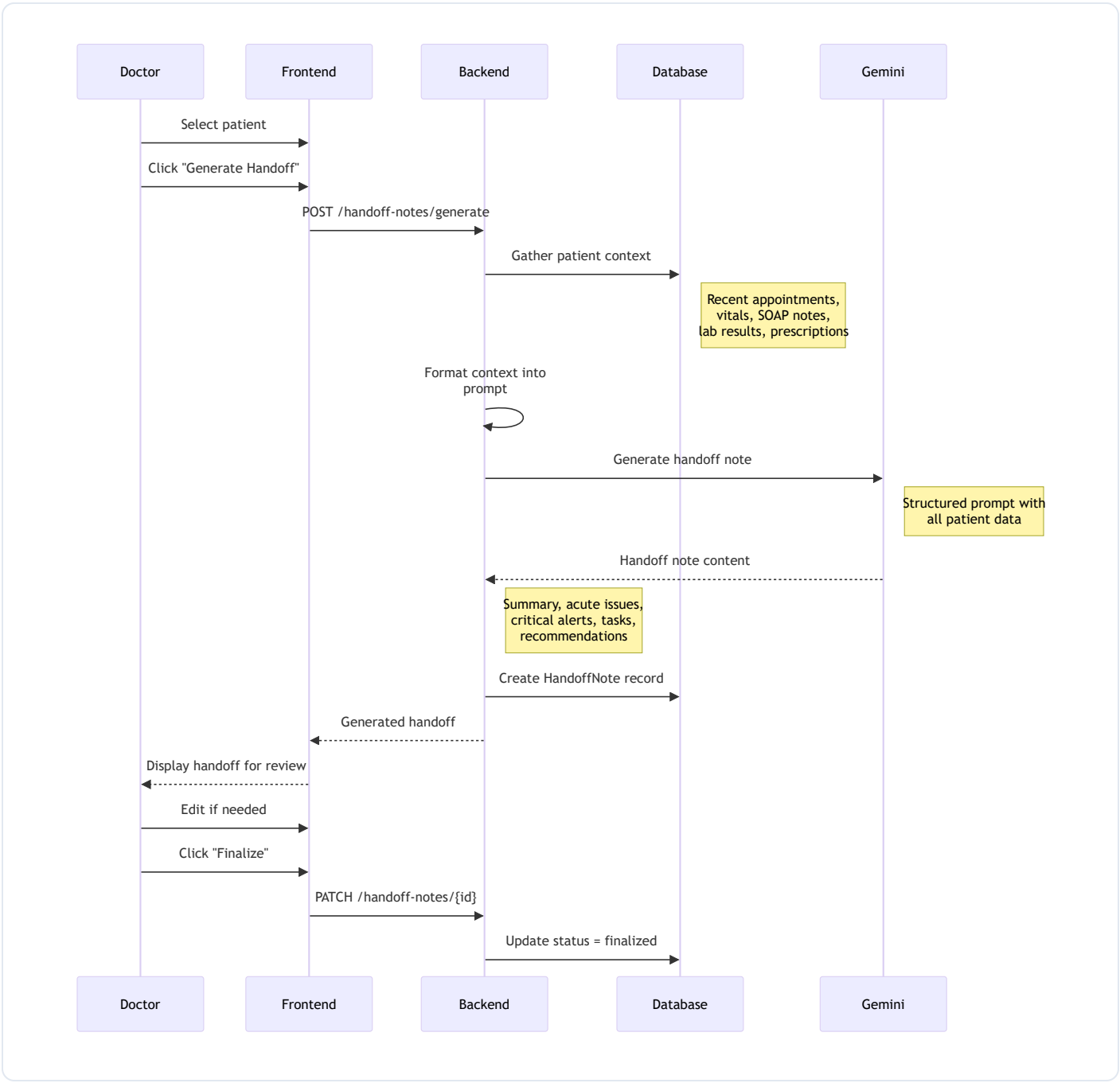
11.1 SOAP Notes from Audio Flow



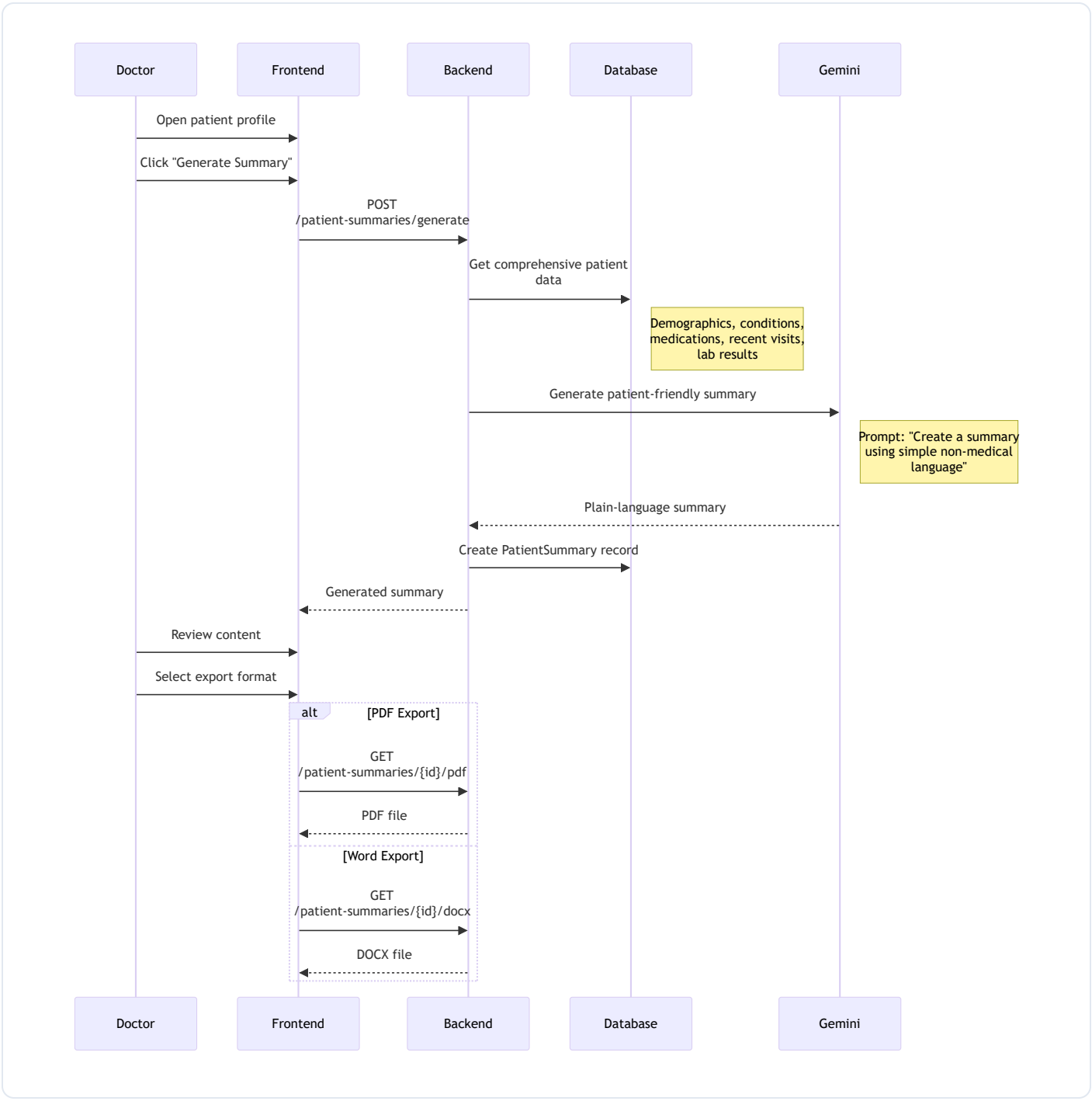
11.2 Manual SOAP Note Entry Flow



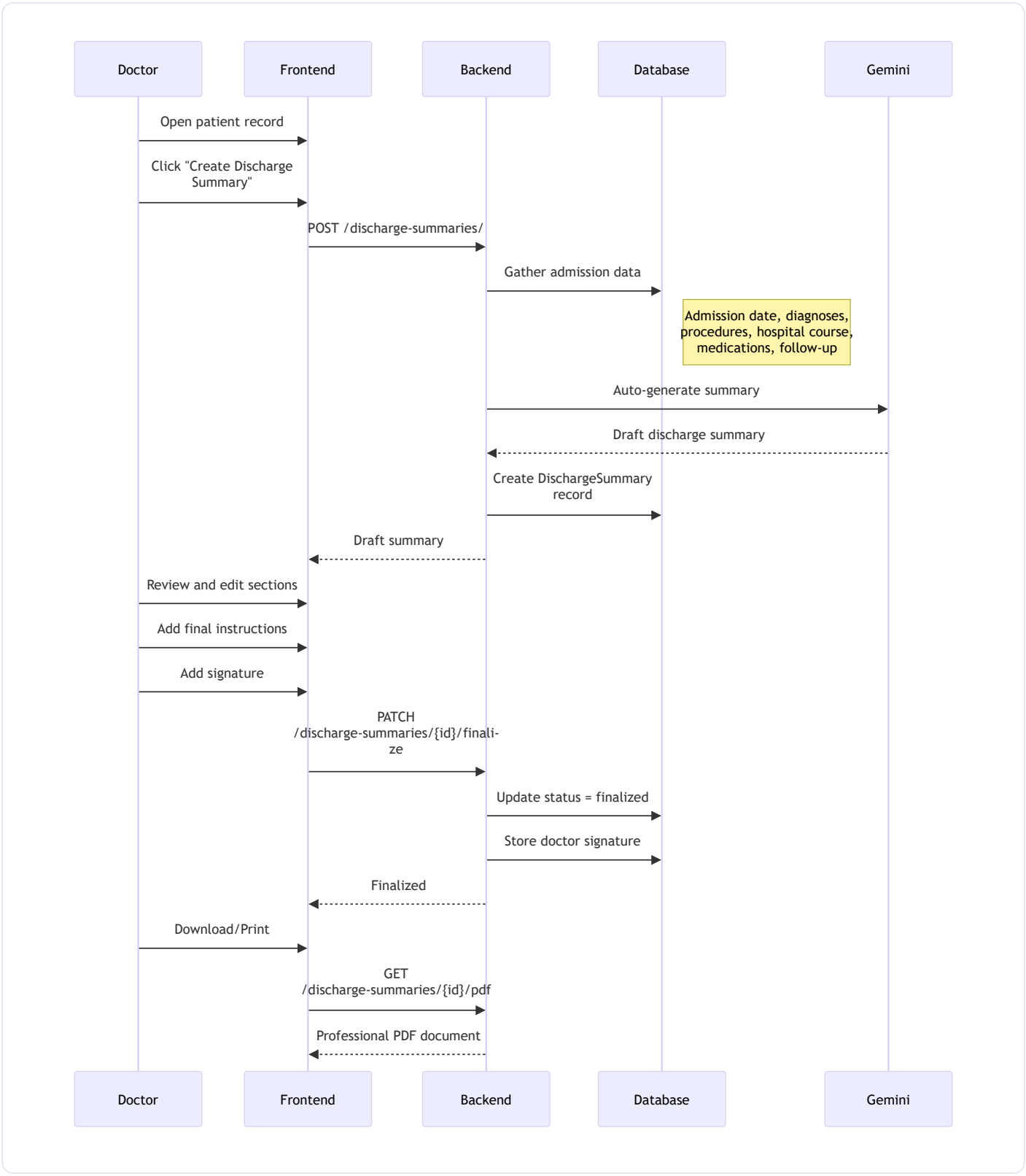
11.3 Handoff Note Generation Flow



11.4 Patient Summary Generation Flow



11.5 Discharge Summary Flow



SOAP Note Features:

- **Audio Upload** - Record or upload audio file
- **AI Transcription** - Gemini converts speech to text
- **SOAP Generation** - AI structures into SOAP format
- **Text Input** - Direct transcript entry

- **Appointment Link** - Associated with specific appointment
- **Historical View** - All SOAP notes for patient

Handoff Note Features:

- **Auto-Generation** - AI creates from patient data
- **Comprehensive Context** - Pulls all relevant patient info
- **Structured Sections** - Summary, acute issues, alerts, tasks, recommendations
- **PDF Export** - Professional printable document
- **Acknowledgment** - Receiving provider can acknowledge

Patient Summary Features:

- **Simple Language** - Non-medical terminology
- **Personalized** - Specific to patient's conditions
- **Actionable** - Clear next steps
- **Multiple Formats** - PDF and Word export

Discharge Summary Sections:

- Patient demographics
- Admission/discharge dates
- Primary diagnosis
- Hospital course
- Procedures performed
- Medications at discharge
- Follow-up instructions
- Attending physician signature

Key Endpoints:

```
# SOAP Notes
POST   /soap-notes/                    # Create SOAP note
POST   /soap-notes/from-audio          # Generate from audio
POST   /soap-notes/generate            # Generate from transcript
GET    /soap-notes/appointment/{id}    # Get notes for appointment

# Handoff Notes
POST   /handoff-notes/generate         # Generate handoff
GET    /handoff-notes/{id}             # Get handoff details
```

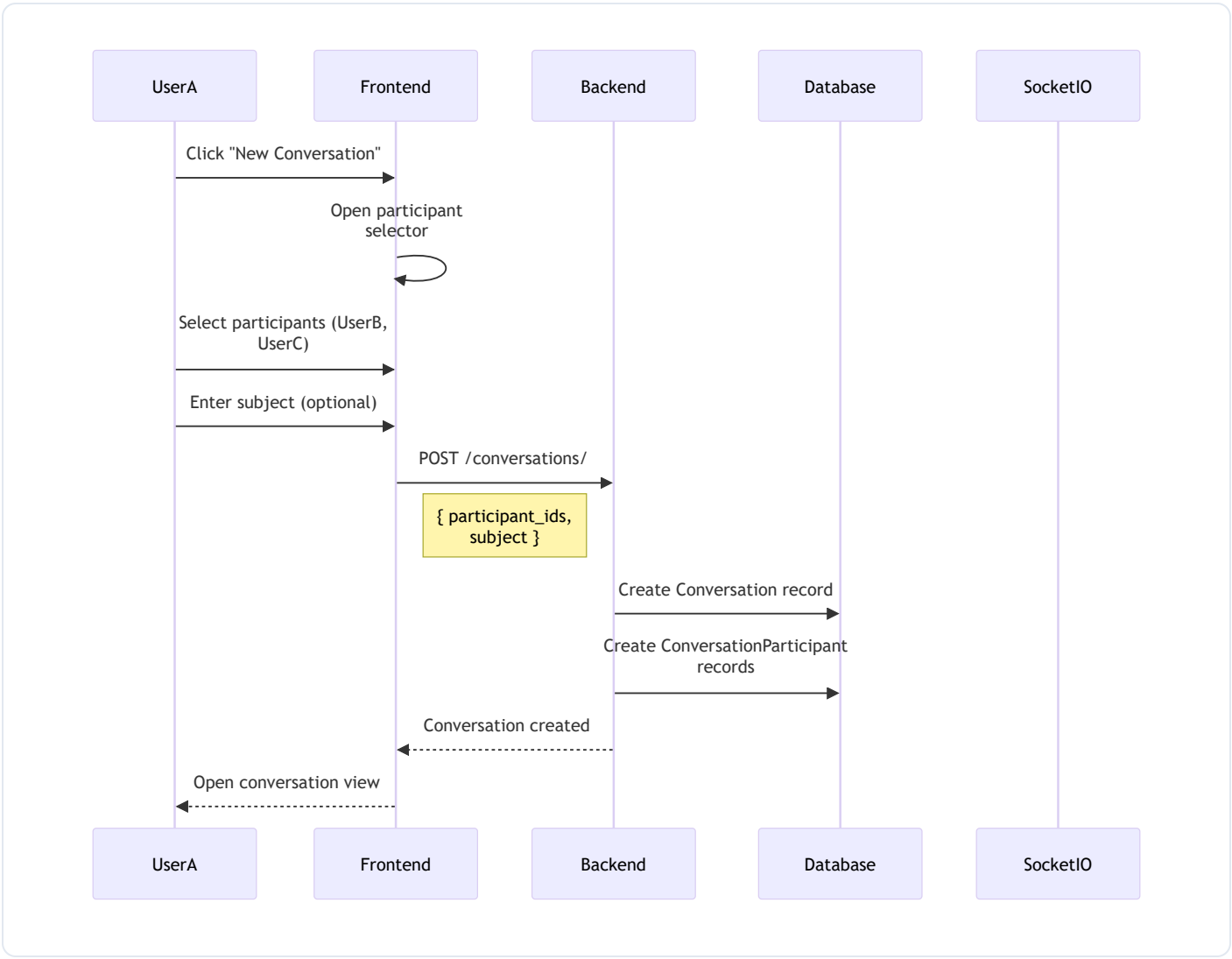
```
PATCH  /handoff-notes/{id}              # Update/finalize
GET    /handoff-notes/{id}/pdf          # Download PDF

# Patient Summaries
POST   /patient-summaries/generate      # Generate summary
GET    /patient-summaries/{id}/pdf      # Download PDF
GET    /patient-summaries/{id}/docx     # Download Word

# Discharge Summaries
POST   /discharge-summaries/            # Create discharge
PATCH  /discharge-summaries/{id}/finalize # Finalize
GET    /discharge-summaries/{id}/pdf    # Download PDF
```

---

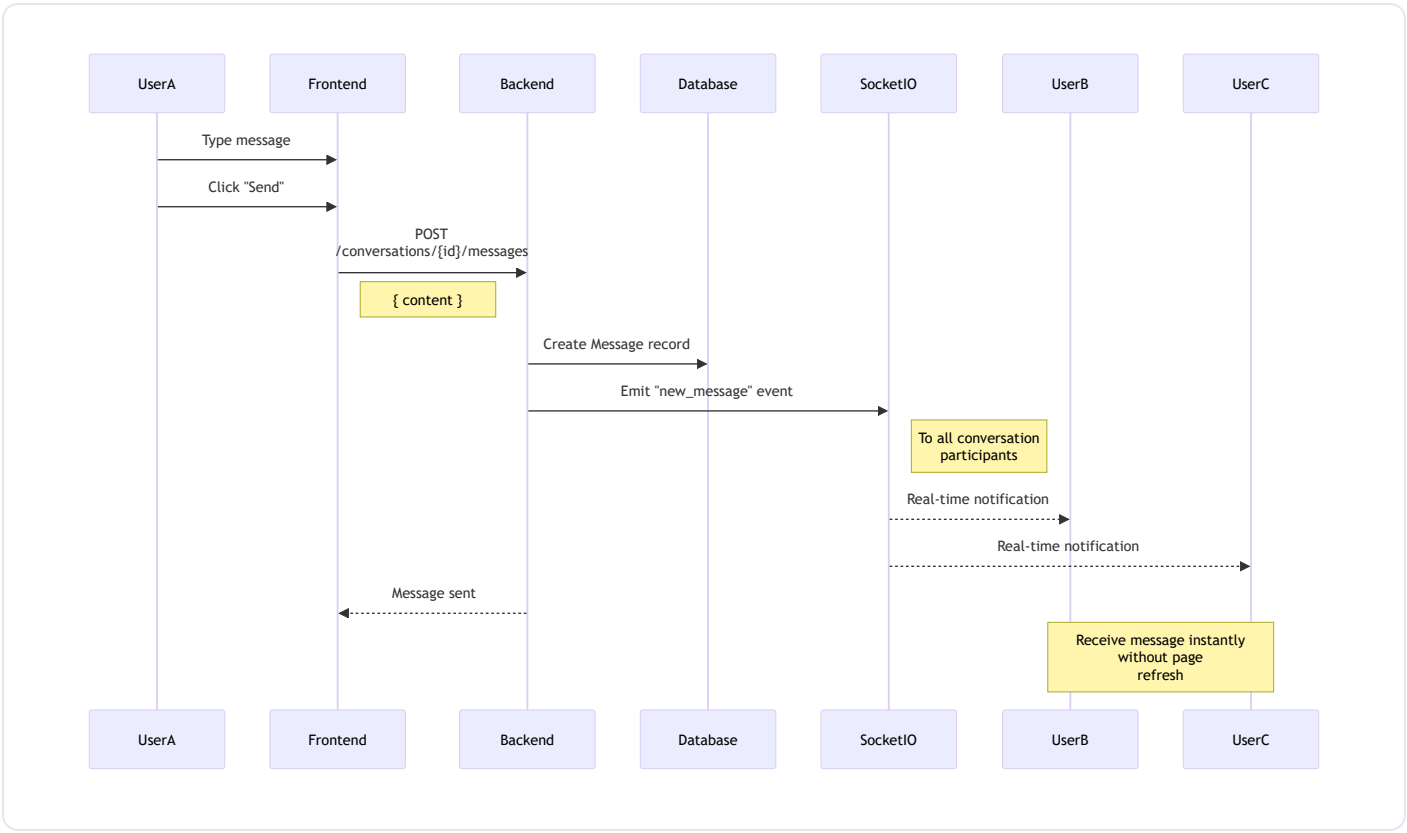
12. Messaging System

Real-time internal messaging between healthcare providers.

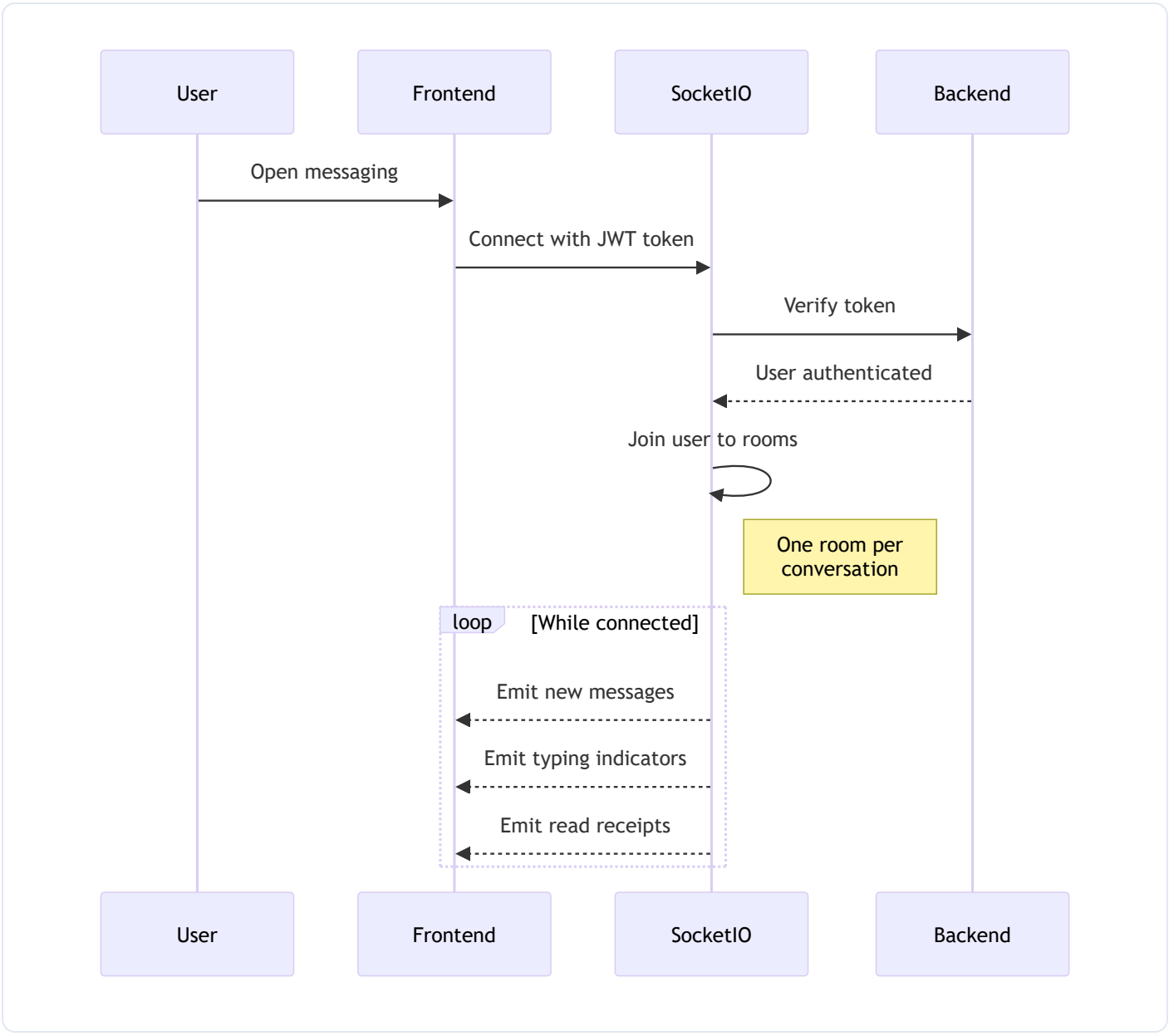
12.1 Create Conversation Flow



12.2 Send Message Flow



12.3 Real-Time Connection Flow



Messaging Features:

- **Create Conversations** - One-on-one or group
- **Subject Line** - Optional topic
- **Real-time** - Socket.IO for instant delivery
- **Hospital Scoped** - Only within same hospital
- **Inbox View** - List of conversations
- **Message History** - Full conversation thread
- **Profile Pictures** - Sender identification

Key Endpoints:

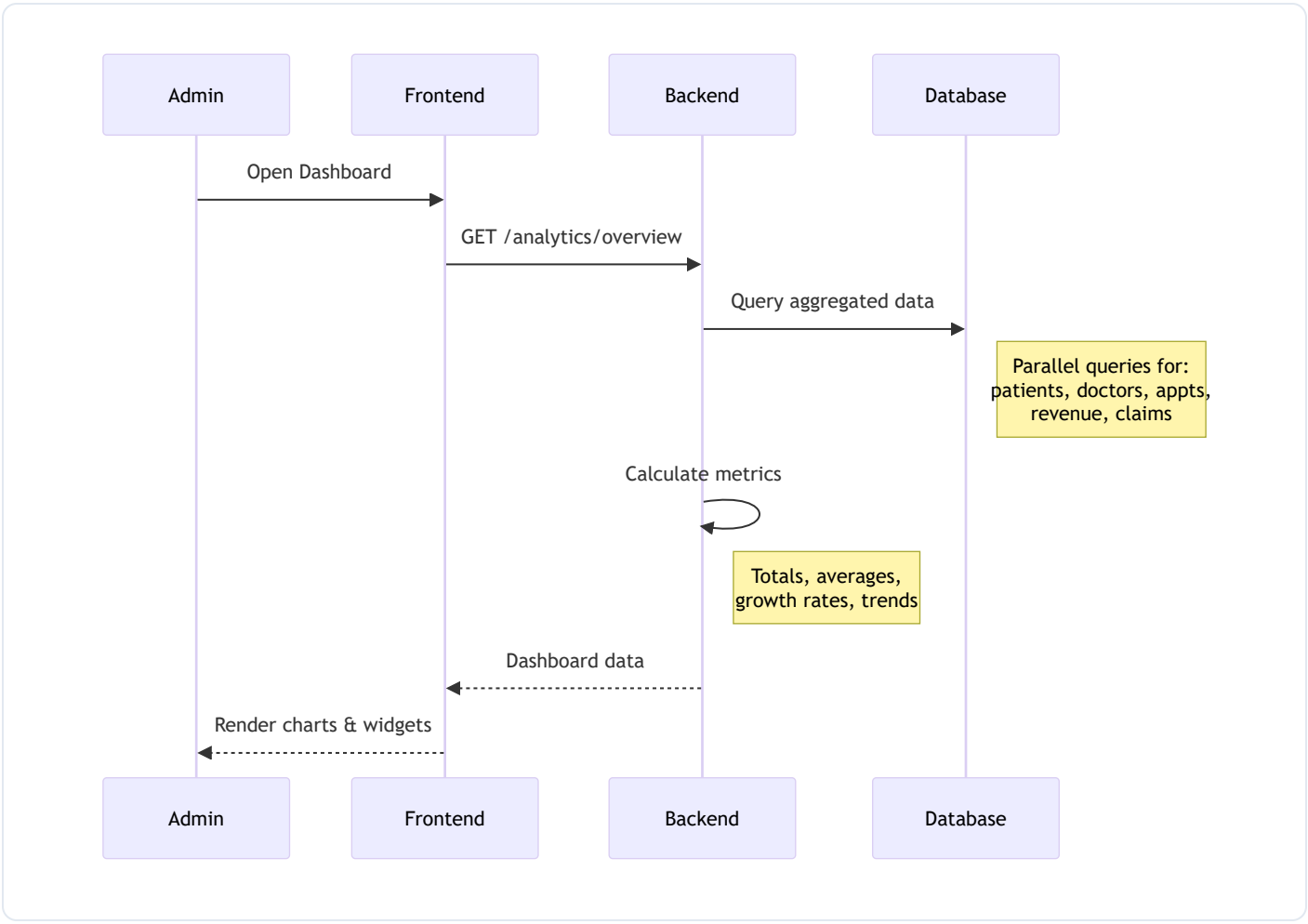
```
POST   /conversations/                # Create conversation
GET    /conversations/                # List user's conversations
GET    /conversations/{id}/messages   # Get messages
POST   /conversations/{id}/messages   # Send message
```

---

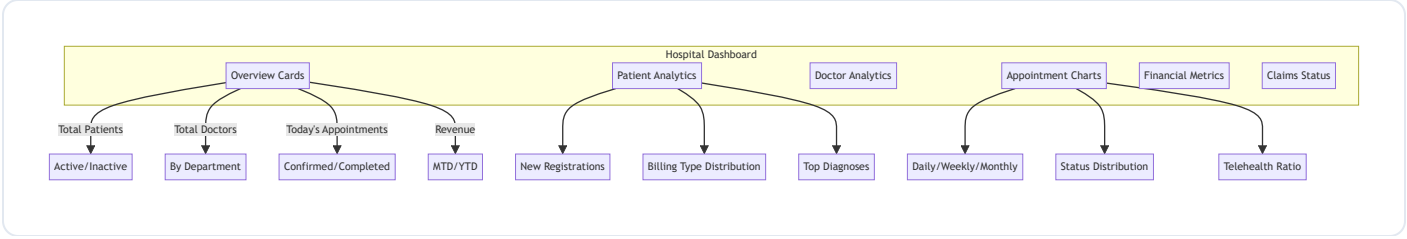
13. Analytics & Dashboard

Comprehensive analytics for hospital administrators.

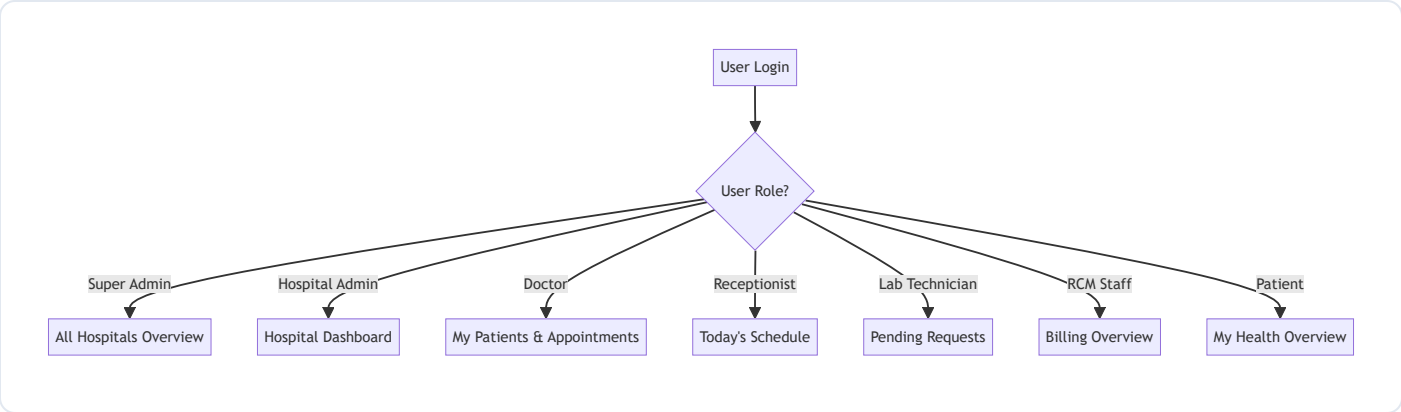
13.1 Dashboard Data Flow



13.2 Dashboard Sections



13.3 Role-Specific Dashboards



Analytics Available:

Hospital Overview:

- Total patients (active/inactive)
- Total doctors by department
- Appointment counts by status
- Revenue metrics
- Trends and growth

Patient Analytics:

- New registrations over time
- Billing type distribution
- Client type breakdown
- Top diagnoses

Doctor Analytics:

- Appointments per doctor
- Patient load distribution
- Revenue by doctor
- Department performance

Appointment Analytics:

- Daily/weekly/monthly trends
- Status distribution
- No-show rates

- Telehealth vs in-person

Financial Analytics:

- Revenue by period
- Outstanding payments
- Payment collection rate
- Average bill amount

Key Endpoints:

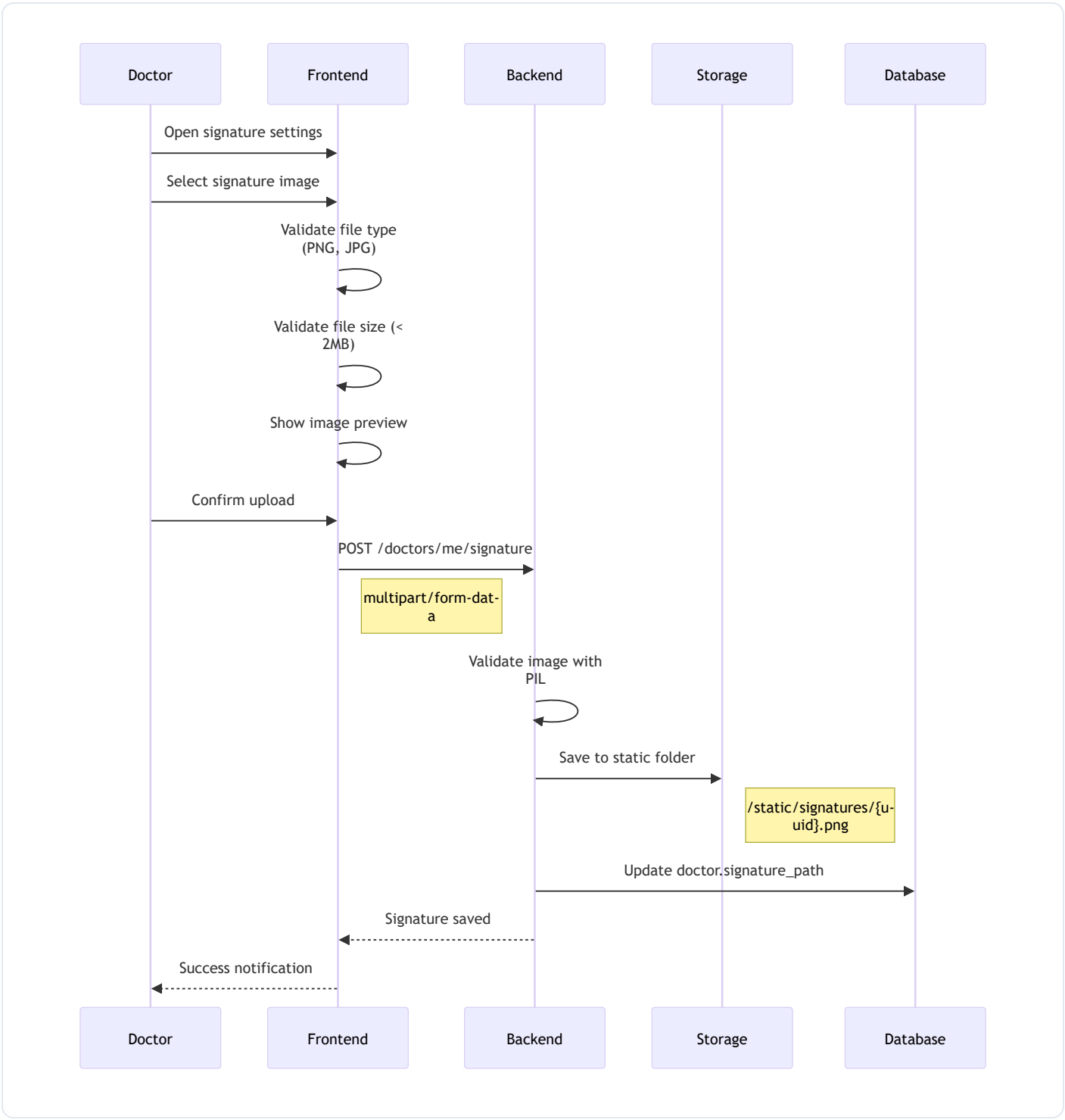
```
GET    /analytics/overview              # Dashboard overview
GET    /analytics/patients              # Patient metrics
GET    /analytics/doctors               # Doctor metrics
GET    /analytics/appointments          # Appointment trends
GET    /analytics/revenue               # Financial metrics
GET    /analytics/claims                # Claims status
```

---

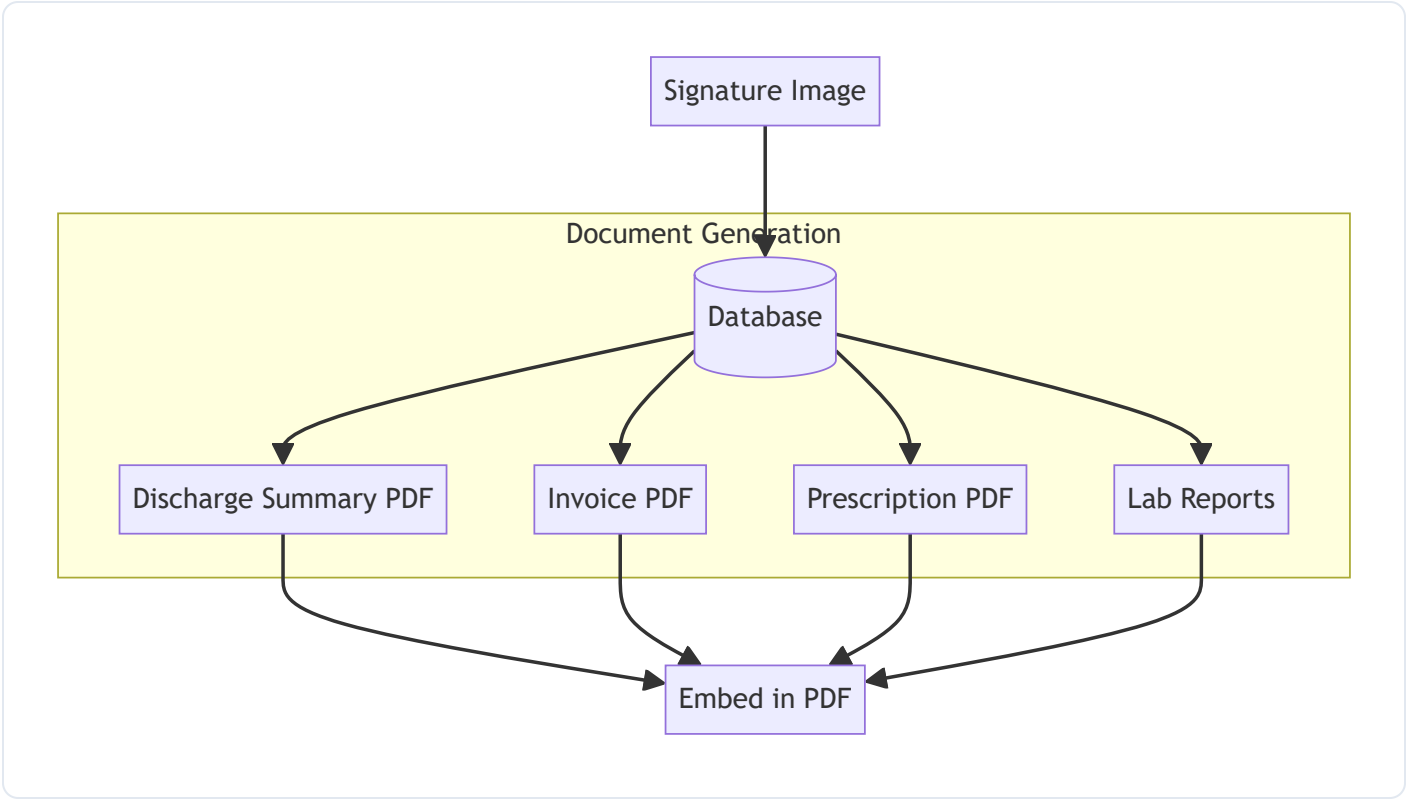
14. Signature Management

Electronic signature system for document signing.

14.1 Signature Upload Flow



14.2 Signature Usage in Documents



Signature Features:

- **Image Upload** - PNG, JPG, JPEG formats
- **Size Limit** - 2MB maximum
- **Image Validation** - PIL verification
- **Update/Replace** - Change signature anytime
- **Document Integration** - Embed in PDFs

Key Endpoints:

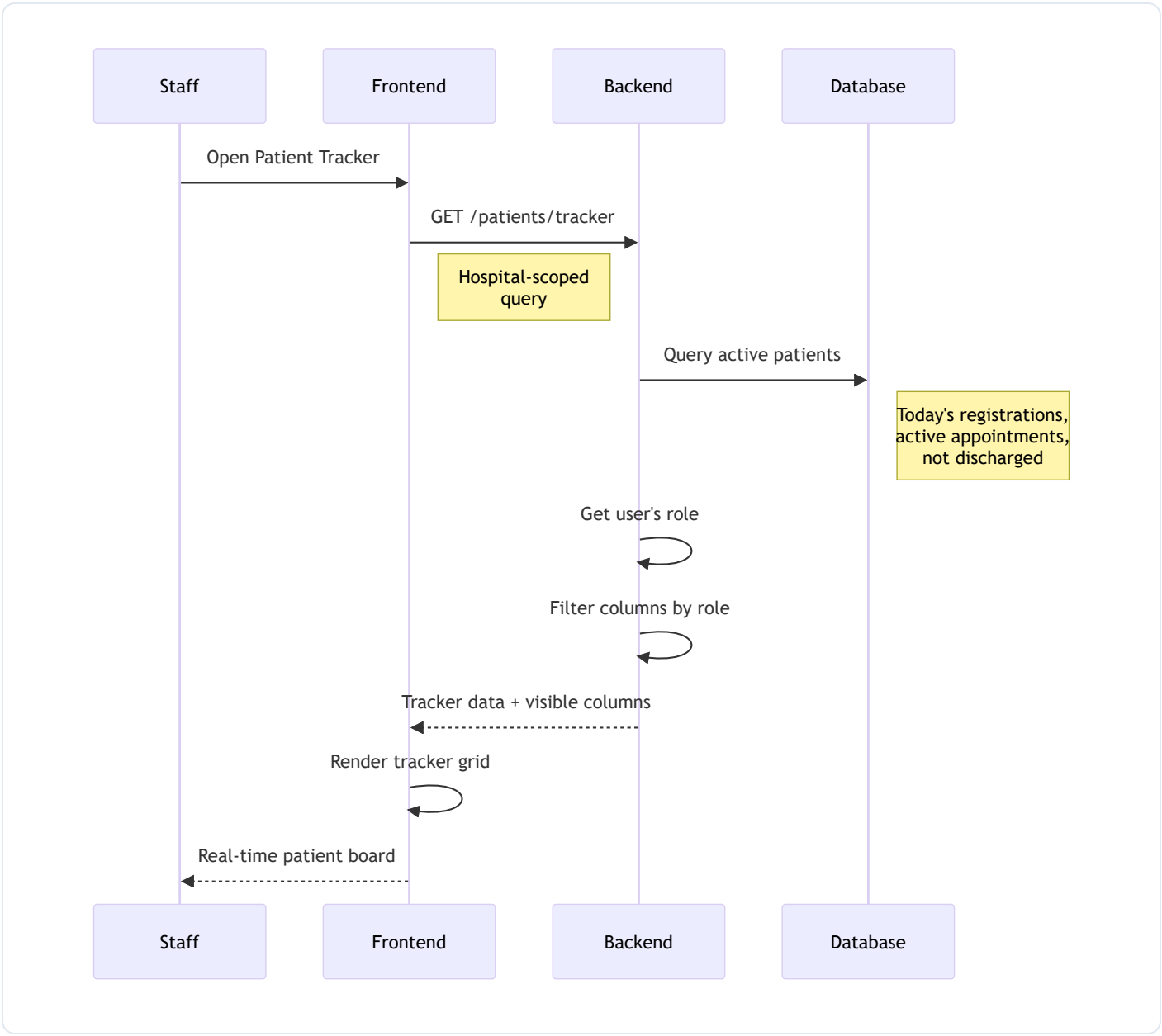
```
POST   /doctors/me/signature          # Upload signature
GET    /doctors/me/signature          # Get signature URL
DELETE /doctors/me/signature          # Remove signature
```

---

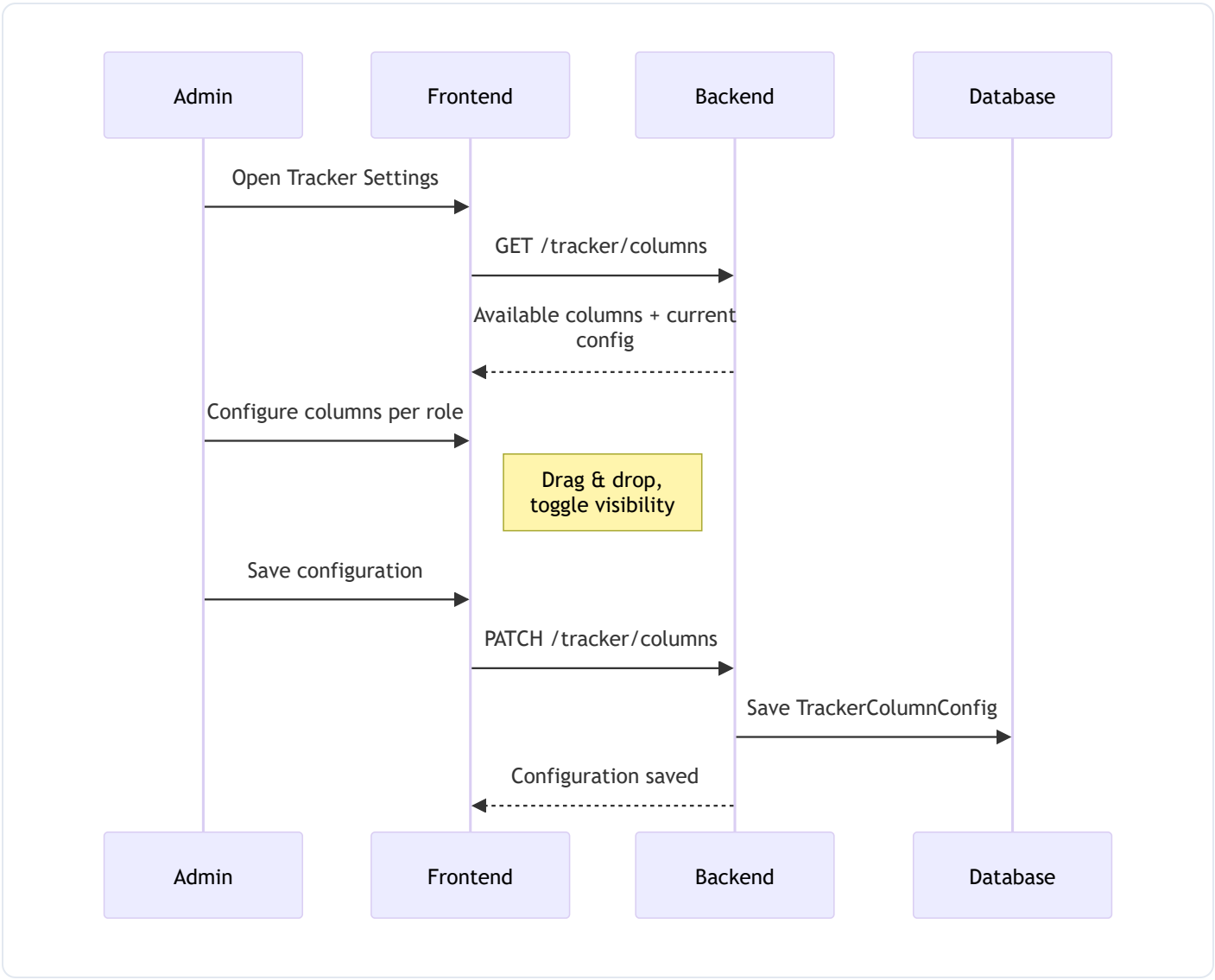
15. Patient Tracker Board

Real-time patient tracking dashboard for staff.

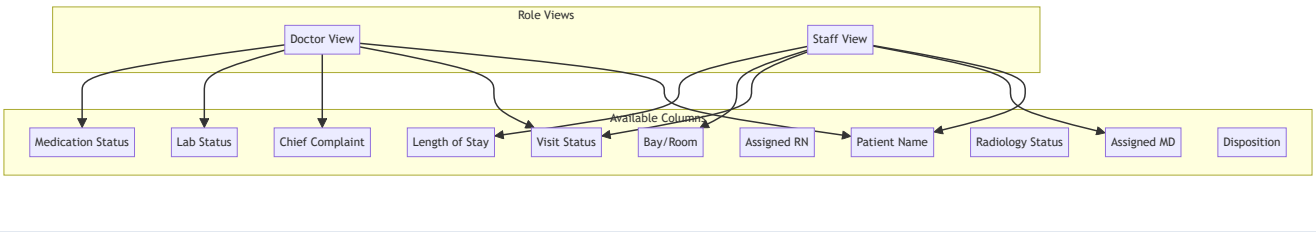
15.1 Tracker Board Flow



15.2 Column Configuration Flow



15.3 Tracker Column Configuration



Tracker Features:

- **Customizable Columns** - Admin configures per role
- **Role-Based Views** - Different columns visible per role
- **Real-time Updates** - Patient status changes
- **Hospital Scoped** - Only current hospital patients
- **Sorting** - By registration date (newest first)

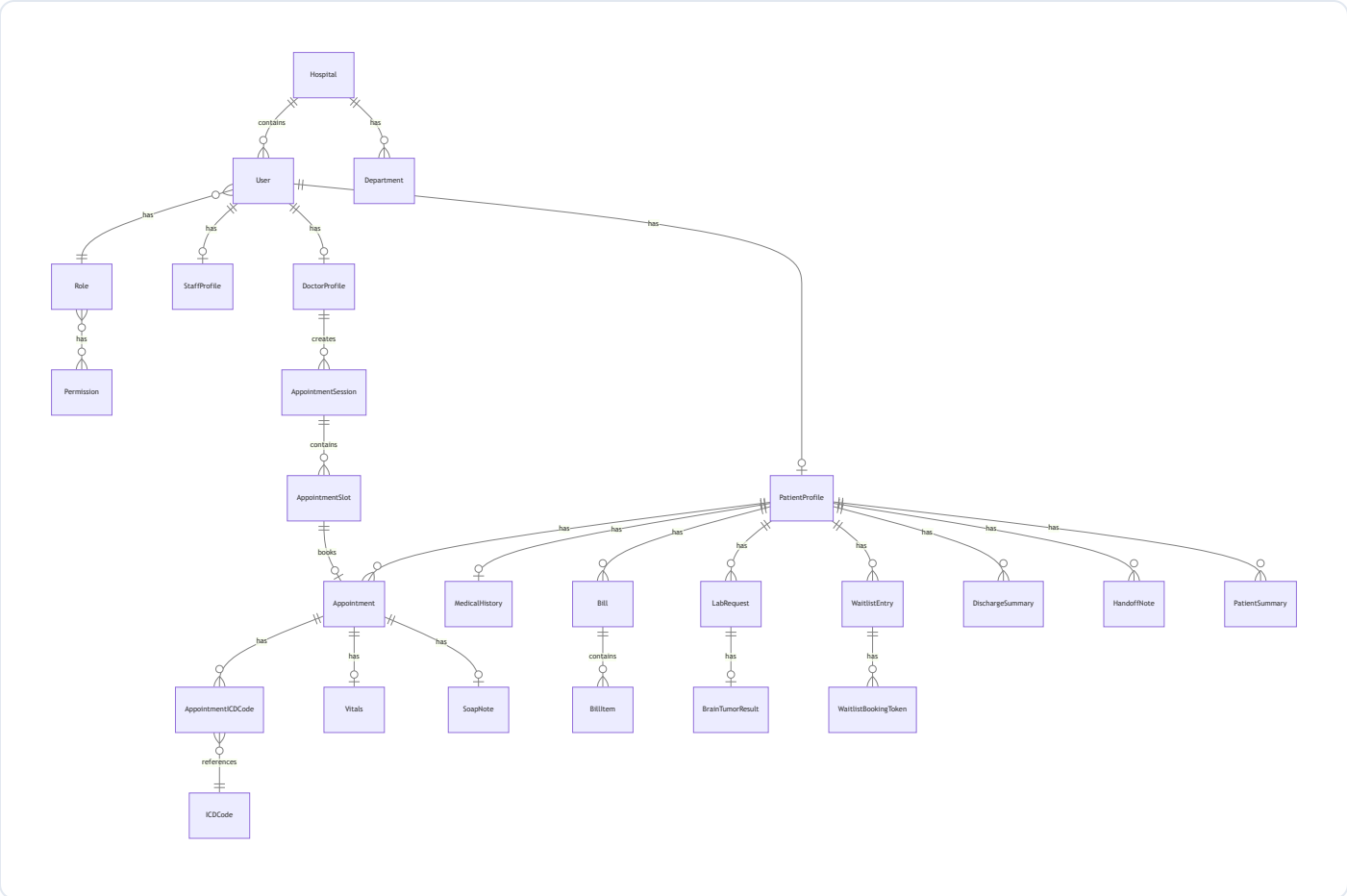
- **Status Indicators** - Color-coded patient states

Key Endpoints:

```
GET    /patients/tracker              # Get tracker data
GET    /tracker/columns               # Get column config
PATCH  /tracker/columns               # Update column config
```

Database Models

Entity Relationship Diagram



Key Models

Model	Description
`User`	Base user account
`PatientProfile`	Patient-specific data
`DoctorProfile`	Doctor credentials/info
`StaffProfile`	Staff member data
`Hospital`	Hospital entity
`Department`	Hospital department
`Role`	User role definition
`Permission`	Granular permission
`AppointmentSession`	Doctor's time block
`AppointmentSlot`	Bookable time unit
`Appointment`	Patient appointment
`Bill`	Patient billing record
`Claim`	Insurance claim
`LabRequest`	Lab test request
`SoapNote`	Clinical SOAP note
`WaitlistEntry`	Waitlist record

API Structure

Route Organization

```
/api
├── /auth           # Authentication
├── /users          # User management
├── /hospitals      # Hospital CRUD
├── /doctors        # Doctor management
├── /patients       # Patient management
├── /staff          # Staff management
├── /departments    # Department CRUD
├── /appointments   # Appointment booking
├── /sessions       # Session management
├── /slots          # Slot management
├── /waitlist       # Waitlist system
├── /billing        # Bills and payments
├── /claims         # Insurance claims
├── /invoices       # Invoice generation
├── /lab-requests   # Lab test requests
├── /lab-results    # Lab result management
├── /soap-notes     # SOAP documentation
├── /handoff-notes  # Shift handoffs
├── /patient-summaries  # Patient summaries
├── /discharge-summaries # Discharge docs
├── /messaging      # Internal messaging
├── /analytics      # Dashboard analytics
├── /signatures     # E-signature management
├── /tracker        # Patient tracker
├── /prescriptions  # Prescription management
├── /clinical-data  # Vitals & medical history
└── /icd-codes      # ICD code lookup
```

Integration Points

External Services

Service	Purpose	Configuration
Google Gemini	AI text generation	`GEMINI_API_KEY`

Service	Purpose	Configuration
Cloudinary	Image/file storage	`CLOUDINARY_*` keys
Stripe	Payment processing	`STRIPE_*` keys
SMTP	Email delivery	`EMAIL_*` config
Google OAuth	Social login	`GOOGLE_*` credentials

AI Services

Brain Tumor Detection API

- **Endpoint**: `http://localhost:8001/predict`
- **Method**: POST with image file
- **Returns**: Classification and confidence

Gemini AI Uses:

1. Audio transcription
2. SOAP note generation
3. Handoff note creation
4. Patient summary generation

Deployment

Docker Deployment

```
# Build and start all services
docker-compose up -d

# View logs
docker-compose logs -f

# Run database migrations
docker-compose exec backend alembic upgrade head
```

Services Deployed:

- **Frontend** - Port 3000
- **Backend** - Port 8000
- **Database** - PostgreSQL
- **Nginx** - Reverse proxy (80/443)

Environment Variables

See `.env.example` for required configuration: - Database credentials - JWT secret key - Encryption key - API keys (Gemini, Cloudinary, Stripe) - Email configuration - Google OAuth credentials

---

Security Features

- 🔐 **JWT Authentication** - Token-based security
- 🔒 **Password Hashing** - Bcrypt encryption
- 🛡️ **RBAC** - Role-based access control
- 📝 **Audit Logging** - All actions logged
- 🔑 **Data Encryption** - Sensitive fields encrypted
- 🚫 **CORS** - Cross-origin protection
- ⏰ **Token Expiry** - Automatic session timeout

Conclusion

NexGenEMR is a comprehensive, enterprise-grade EMR system designed for modern healthcare facilities. It provides:

- **Complete Patient Journey** - From registration to discharge
- **Flexible Scheduling** - Multiple booking methods and waitlist management
- **Financial Management** - Full billing and claims workflow
- **Clinical Excellence** - AI-powered documentation
- **Security First** - HIPAA-ready architecture
- **Scalability** - Multi-hospital support

For detailed implementation guidance, refer to: - `DEPLOYMENT_GUIDE.md` - Production deployment - `WAITLIST_TESTING_GUIDE.md` - Waitlist feature testing - `DOCKER_SETUP.md` - Docker configuration

---

Last Updated: January 29, 2026